

2026 年电工杯数学建模竞赛论文

A 题

绿电直连型电氢氨园区优化运行

参赛编号： 014315

论文题目： 绿电直连型电氢氨园区优化运行

绿电直连型电氢氨园区优化运行

摘要：绿电直连将风光电源与电氢氨等高载能负荷在园区侧耦合，是推动新能源就近消纳和化工过程低碳化的重要路径；但仅按年电量核算容易掩盖小时级源荷错配，并使上网反送、备用互济和储能配置成本转化为系统层面的平衡问题。针对这一矛盾，本文以给定负荷、电价、风光场景和设备参数为基础，构建面向绿电直连电氢氨园区的小时级统一优化框架。模型将绿电直连三项政策指标嵌入运行评价，统一比较刚性满负荷、离散开停机、连续负荷率、离网储能和并网同产量五类工况；以边际成本排序解析求解离散调节问题，以含购售电互斥的混合整数线性规划刻画连续柔性负荷和储能调度；进一步定义公共电网支撑价值，并将前四问数值结果整理为政策证据链。

计算结果表明，刚性 36 t/d 满负荷基准下，典型日新能源自发自用比例和总用电量绿电比例分别为 64.08% 和 69.21%，但新能源上网比例达到 35.92%，超过 20% 约束，揭示了风光出力与制氢制氨刚性负荷之间的关键错配。扩容至 72 t/d 后，离散开停机模型显示，典型场景中满足三项绿电指标的最低成本方案为 54 t/d，吨氨成本为 4591.27 元/t；若仅追求年均吨氨成本，36 t/d 虽降至 4493.82 元/t，却全年无全满足代表日，表明经济性最优并不等价于合规性最优。连续负荷率模型进一步量化柔性调节的增量价值：36 t/d 方案较离散开停机降本 284.35 元/t，全满足代表天数由 0 天提升至 165 天；63 t/d 方案全满足代表天数达到 300 天，表明可调负荷是提高小时级源荷匹配和系统友好性的核心资源。

在离网情形下，无储能运行的年产氨量仅 9781.56 t，产能利用率为 37.74%，年弃风弃光 14188.67 MWh，并存在常规负荷缺供。配置 5 MWh 储能后，年产氨量提高至 9972.26 t，年弃电降至 11280.74 MWh，常规负荷缺供消除，年均吨氨成本为 4046.47 元/t。储能经济-消纳 Pareto 分析显示，5 MWh 是给定成本口径下的经济方案；若最大弃电场景弃电率不超过 10%，需 80 MWh 储能；若要求零弃电，则需 155 MWh，吨氨成本升至 4290.85 元/t。并网同产量对比给出公共电网支撑价值 735.81 元/t。基于上述证据，本文建议建立小时级绿电溯源和最大交换功率考核机制，将柔性负荷纳入准入与补偿范围，以经济性、弃电率和可靠性多目标配置储能，并完善并网项目系统费用分担机制。

关键词：绿电直连；电氢氨园区；柔性负荷；储能配置；公共电网支撑价值；混合整数线性规划

1 问题背景与重述

1.1 问题背景

在高比例新能源接入背景下，风电、光伏出力的波动性与工业负荷的连续性之间存在显著矛盾。绿电直连通过将新能源电源与电解制氢、合成氨等高载能负荷在园区层面耦合，为新能源就地消纳和化工行业低碳转型提供了新的实现路径 [1, 2]。然而，若园区只按年累计电量平衡进行设计，可能掩盖小时尺度上的源荷错配，导致部分时段向公共电网购电，另一些时段又向外部电网反送电。对于电氢氨园区，制氢电解槽、合成氨装置、储能和公共电网共同构成多能耦合系统，其运行优化需要同时兼顾功率平衡、制氨产量、绿电指标、吨氨成本和电力系统友好性。

1.2 问题重述

给定资料包括园区常规负荷曲线、典型日风光出力、24 种风光组合场景、风光发电成本、分时电价、余电上网价格、电解槽与合成氨设备参数以及储能参数。本文需要完成如下任务：第一，计算典型日固定制氢制氨负荷下的购电量、上网电量、绿电指标和吨氨成本；第二，扩容至 72 t/d 后，考虑制氨装置离散开停机并优化不同日产量下的运行时段；第三，将制氨负荷放宽为 10%–100% 连续可调，并与离散开停机模型对比；第四，在离网条件下引入储能，分析弃风弃光、缺供风险、储能容量和并网同产量经济性；第五，基于前四问结果，分析绿电直连园区高渗透对新能源消纳、电网调峰、备用、费用分担和政策机制的影响。

2 问题分析

五项任务分别涉及指标核算、负荷调节、储能配置和政策建议，但共同指向同一核心问题：如何在小时尺度上协调风光出力波动、园区常规负荷、制氢制氨负荷、储能和公共电网交换。若仅从日总电量或年总电量判断绿电直连效果，容易掩盖“部分时段购电、部分时段反送”的时序错配；若仅以吨氨成本最低为目标，又可能牺牲新能源就地消纳和政策合规性。因此，本文将购电量、上网电量、弃风弃光、缺供电量、绿电指标和吨氨成本置于同一评价框架，先分析电量流向，再讨论调节手段，最后评价电力系统影响。

2.1 问题一分析

问题一是全文的基准工况。此时制氢电解槽和合成氨装置全天满负荷运行，工艺负荷没有调节自由度，园区只能通过公共电网吸收短缺电量或外送富余电量。其重点在于逐小时识别风光出力与固定负荷之间的错配，并在统一口径下计算新能源自发自用比例、总用电量绿电比例和新能源上网比例。由于三项指标分别约束本地消纳、用电绿色属性和外送规模，单一指标达标并不代表整体合格；特别是上网比例能够直接反映固定负荷吸纳风光的能力。问题一的结果将作为后续各问的参照，用于衡量开停机、连续调节和储能对源荷匹配的改善程度。

2.2 问题二分析

问题二在扩容至 72 t/d 后引入离散开停机调节。对给定日产量而言，满负荷制氨功率固定，日产量等价于确定一天内的开机小时数，因此决策核心转化为生产时段选择。在未给出启停成本、最小开停时间和爬坡约束的条件下，各时段之间除总开机小时数外不存在跨时段耦合，可将 0-1 开机决策转化为按小时边际成本排序的问题。模型需同时处理两个矛盾：一方面，低日产量可减少高价购电时段的生产负荷，从而降低吨氨成本；另一方面，负荷规模下降会削弱新能源本地消纳能力，使上网比例升高。因此，问题二不仅需要给出最低成本日产量，还应比较不同日产量在 24 种风光场景下的绿电指标合格性，明确经济性与合规性之间的取舍。

2.3 问题三分析

问题三进一步把制氨负荷由“全开或停机”放宽为 10%–100% 连续可调。与问题二相比，连续负荷率可以在风光高出力或低电价时段提高用电，在风光不足或高电价时段降低用电，更符合柔性工业负荷参与源荷匹配的实际含义。此时的建模难点在于购电和上网变量不能任意同时出现：分时购电低谷价低于余电上网电价时，若缺少互斥约束，模型可能形成低价购电再高价售电的非物理套利。因此，问题三需要在连续负荷率、日产量约束和功率平衡之外加入购售电互斥变量，形成混合整数线性规划。为客观比较连续调节价值，问题三沿用问题二的设备容量、日产量集合、场景权重、成本口径和绿电指标口径，仅改变负荷调节方式。

2.4 问题四分析

问题四把系统边界从并网切换为离网，公共电网不再承担功率平衡、余电消纳和备用支撑功能。此时风光富余不能外送，只能由制氨负荷、储能或弃风弃光吸收；风光不足时，也不能通过购电弥补，只能降低制氨负荷、释放储能或记录常规负荷缺供。因此，问题四需要先建立无储能离网调度，判断当前风光装机在完全自给条件下能够支撑的产氨水平和可靠性；再引入储能 SOC 动态、充放电互斥、容量约束和投资成本，分析储能对削峰填谷、减少弃电和保障常规负荷的作用。储能配置不宜用单一“最优容量”概括，还应区分经济性目标与消纳目标：小容量储能可能吨氨成本较低，而低弃电或零弃电要求往往需要显著更大的容量。进一步在相同产氨量下比较离网储能与并网运行成本，可将公共电网的平衡和备用价值转化为可量化指标。

2.5 问题五分析

问题五侧重将前四问的数值结果转化为电力系统影响和政策机制分析。高渗透绿电直连园区具有双重作用：一方面，高载能可调负荷有利于新能源就近消纳，减少远距离外送压力，并为电力系统提供需求侧调节资源；另一方面，若项目按最低吨氨成本运行，可能出现低产量、高上网比例和公共电网平衡成本外部化等问题。问题五围绕“模型证据–系统影响–政策建议”展开：由问题一识别小时错配，由问题二和问题三评价负荷柔性价值，由问题四量化储能和公共电网支撑价值，再据此提出小时级绿电溯源、上网比例和交换功率考核、柔性负荷补偿、储能多目标配置以及系统费用分担等建议。

2.6 整体递进关系

五个问题构成逐步增加系统柔性的递进链条。问题一是不含优化变量的确定性核算，用于暴露刚性负荷下的源荷错配；问题二引入离散开停机变量，用边际成本排序选择生产时段；问题三引入连续负荷率和购售电互斥约束，用混合整数线性规划刻画柔性负荷调节；问题四删除购售电变量并加入储能和缺供变量，用离网调度、容量扫描和同产量比较评价完全依赖风光时的可靠性与成本；问题五则把前四问的成本、指标、交换功率和储能结果整理为系统影响证据。本文由此形成“刚性基准-离散调节-连续调节-储能配置-政策评价”的分析框架。

图1以“源-荷-储-网-氨”的能量耦合为主线，概括本文从风光时序波动、园区柔性调节到五问递进建模的总体逻辑：前端由风光资源和常规负荷形成源荷错配，中端通过制氢制氨负荷、储能和电网交换提供调节手段，后端将成本、绿电指标和电网支撑价值转化为政策证据链。

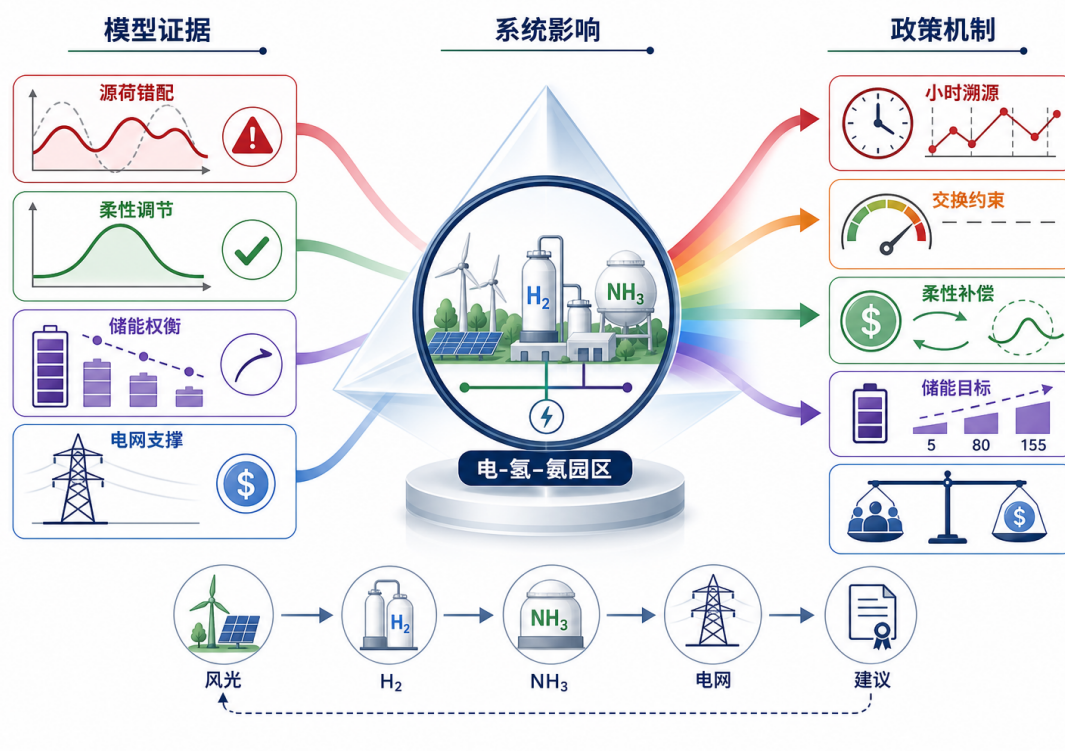


图 1：绿电直连型电氢氨园区模型证据、系统影响与政策机制关系

3 模型假设

- H1. 以小时为基本调度时间尺度，令 $\Delta t = 1 \text{ h}$ 。
- H2. 问题二至四的 24 种风光组合场景由 6 种风电场景和 4 种光伏场景组合得到，每个场景代表 15 天，全年按 360 天加权统计。
- H3. 并网问题中新能源自发自用电量采用主口径 $E^{self} = E^{re} - E^{sell}$ ，给定括号公式仅作为复核口径。
- H4. 扩容后 72 t/d 工况下，碱性电解槽、PEM 电解槽和合成氨装置聚合为同步产氢-制氨单元，满负荷功率为 41.5 MW，满负荷产氨能力为 3 t/h。
- H5. 成本均为给定成本口径下的比较成本，包含风光发电成本、购电成本、余电上网收益、设备运维和储能年化成本，不包含未给定参数的全寿命固定投资。
- H6. 储能按 1C 配置，即 $P^{cap} = E^{cap}$ ；储能 SOC 采用典型日闭环约束。
- H7. 不考虑电网潮流、电压、无功、短路容量等配网约束；不考虑合成氨反应器热惯性、启停成本、最小开停时间和氢氨储罐动态。

4 符号说明与数据处理

4.1 符号说明

符号	含义
s, t, D	风光组合场景、小时编号、日产氨量目标
$P_{s,t}^w, P_{s,t}^{pv}, P_{s,t}^{re}$	风电、光伏和新能源总出力，单位 MW
P_t^L	园区常规电负荷，单位 MW
$u_{s,D,t}, y_{s,D,t}$	连续制氨负荷率、离散开停机变量
$P_{s,t}^{buy}, P_{s,t}^{sell}$	购电功率和上网功率，单位 MW
$P_{s,t}^{cur}, P_{s,t}^{shed}$	弃风弃光功率、常规负荷缺供功率，单位 MW
$P_{s,t}^{ch}, P_{s,t}^{dis}, E_{s,t}$	储能充电功率、放电功率和 SOC
$M^{\text{NH}_3}, C, LCOA$	产氨量、总成本和给定成本口径下的吨氨成本
$R^{self}, R^{green}, R^{sell}$	新能源自发自用比例、总用电量绿电比例和新能源上网比例
R^{cur}, R^{use}	弃风弃光率和风光利用率

4.2 数据说明及预处理

模型输入来自附件 1–8。问题一采用附件 1 和附件 2 构成典型日；问题二至四将 6 种风电场景与 4 种光伏场景组合，形成 24 种风光场景。每个组合场景代表 15 天，年度统计采用

$$X^{year} = \sum_{s=1}^{24} 15X_s.$$

常规负荷由附件 1 标幺曲线乘以 6 MW 得到。问题一采用初始制氨能力 36 t/d，生产单元满负荷功率为

$$P^{prod} = 10 + 10 + 0.75 = 20.75 \text{ MW}, \quad q^{\text{NH}_3} = 1.5 \text{ t/h}.$$

问题二至四采用扩容后 72 t/d 能力：

$$P^{prod} = 20 + 20 + 1.5 = 41.5 \text{ MW}, \quad q^{\text{NH}_3} = 3 \text{ t/h}.$$

此时合成氨每小时需氢 600 kg，两类电解槽满负荷产氢量为 $280 + 320 = 600 \text{ kg/h}$ ，故可将制氢和合成氨聚合为同步生产单元。

5 统一模型框架

5.1 并网功率平衡与绿电指标

并网运行下，园区小时功率平衡为

$$P_{s,t}^{re} + P_{s,t}^{buy} = P_t^L + P_{s,t}^{prod} + P_{s,t}^{sell}.$$

并网型绿电直连指标按

$$E^{self} = E^{re} - E^{sell}, \quad R^{self} = \frac{E^{self}}{E^{re}}, \quad R^{green} = \frac{E^{self}}{E^{load}}, \quad R^{sell} = \frac{E^{sell}}{E^{re}}$$

计算，并与 60%、30% 和 20% 阈值比较。给定括号公式 $E^{load} - E^{sell} - E^{buy}$ 作为复核口径保留。

5.2 成本核算模型

日净成本统一写为

$$C = C^w + C^{pv} + C^{buy} + C^{om} + C^{bat} - I^{sell}.$$

其中 C^w 、 C^{pv} 为风电和光伏发电成本， C^{buy} 为分时购电成本， C^{om} 为电解槽和合成氨装置运维成本， C^{bat} 为储能成本， I^{sell} 为余电上网收益。吨氨成本为

$$LCOA = \frac{C}{M^{\text{NH}_3}}.$$

本文的 $LCOA$ 均为给定成本口径下的比较成本，不代表包含全部固定投资的工程全寿命平准化成本。

5.3 求解算法

各问求解算法按模型结构分别确定，表2给出了模型、算法与关键结果的对应关系。问题一不含优化变量，直接按 24 小时功率平衡核算购电、上网、绿电指标和吨氨成本。问题二在固定日产量下只需选择开机小时；由于未给定启停成本和爬坡约束，目标函数可分解为各小时边际成本之和，按边际成本升序选择开机时段即可得到全局最优解。问题三将制氨负荷率放宽为连续变量，并引入购售电互斥二进制变量，形成小规模混合整数线性规划。问题四在离网条件下删除购售电变量，分别采用无储能规则调度、含储能 MILP、储能容量扫描和并网同产量 MILP 比较离网自平衡与电网支撑价值。问题五不增设调度变量，而是把前四问的成本、指标、交换功率和储能结果整理为系统影响证据。

数值求解后主要检查三类条件：一是功率平衡、日产量、年度权重和 SOC 闭环误差；二是购电/上网互斥、储能充/放电互斥；三是绿电比例、上网比例、弃电率和缺供

量等关键指标是否满足定义。问题三和问题四的最大平衡误差均处于 10^{-14} 量级，表明 MILP 求解结果满足数值精度要求。

表 2：五问模型与求解算法对应关系

问题	模型核心	求解算法	关键结果
Q1	典型日功率平衡、三项绿电指标、吨氨成本	逐小时确定性核算，复杂度 $O(24)$	购售电量、绿电指标、吨氨成本
Q2	离散开停机 0-1 模型，固定开机小时数	边际成本排序，等价于特殊 0-1 线性规划	最优开机时段、年度成本、合格天数
Q3	连续负荷率 MILP，购售电互斥	每个场景和日产量独立求解	成本、购售电量、指标合格性对比
Q4	离网储能 MILP，SOC 闭环、弃电和缺供	规则调度、容量扫描、并网同产量对比	储能 Pareto、离网成本、电网支撑价值
Q5	系统影响指标和证据矩阵	归纳前四问关键结果并建立政策证据链	系统影响、政策依据和建议

上述算法以统一功率平衡、绿电指标和成本口径为基础，再根据各问的变量类型选择确定性核算、边际排序或混合整数线性规划。图2概括了从数据输入、统一口径、分问题求解到结果校验和结果分析的算法主线。



图 2：五问求解算法逻辑

6 问题一：典型日刚性满负荷运行指标核算

6.1 模型建立

在问题一中，制氢制氨负荷全天满负荷运行，工艺负荷为 20.75 MW，日产氨量为 36 t。对每个小时直接由功率平衡得到

$$P_t^{buy} = \max(P_t^L + 20.75 - P_t^{re}, 0),$$

$$P_t^{sell} = \max(P_t^{re} - P_t^L - 20.75, 0).$$

随后按统一绿电指标和成本模型计算典型日结果。

问题一不含优化变量，其作用是建立后续各问的基准核算口径。本文同时计算两类自发自用复核量：主口径 $E^{self} = E^{re} - E^{sell}$ 与负荷侧校验量 $E^{load} - E^{buy}$ 二者一致；给定括号公式 $E^{load} - E^{sell} - E^{buy}$ 仅作为复核列，不参与主结论。这样既保持物理含义清晰，也避免后续问题在并网和离网指标之间混用口径。

6.2 结果分析

表 3: 问题一典型日核心结果

指标	数值	单位
典型日总用电量	558.7200	MWh
新能源发电量	603.4480	MWh
网购电量	172.0438	MWh
上网电量	216.7718	MWh
新能源自发自用比例	64.08	%
总用电量绿电比例	69.21	%
新能源上网电量比例	35.92	%
吨氨成本	4322.34	元/t

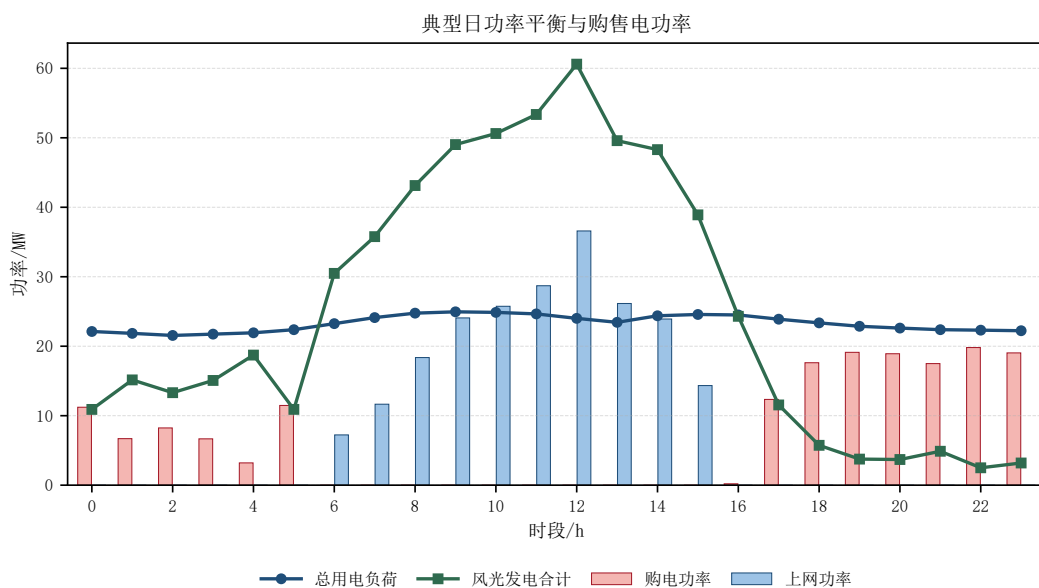


图 3: 典型日风光出力、园区负荷与电网交换功率

表3显示,典型日自发自用比例和绿电比例均满足要求,但新能源上网比例为 35.92%,超过 20% 约束。图3表明,夜间光伏出力为零且风电不足时需要购电,白天风光出力集中时又产生大量上网电量;图4则直接给出三项指标与阈值的对比,突出上网比例是唯一不合格项。该结果表明,刚性制氨负荷无法主动追踪风光出力变化,是后续负荷调节和储能配置的直接动因。

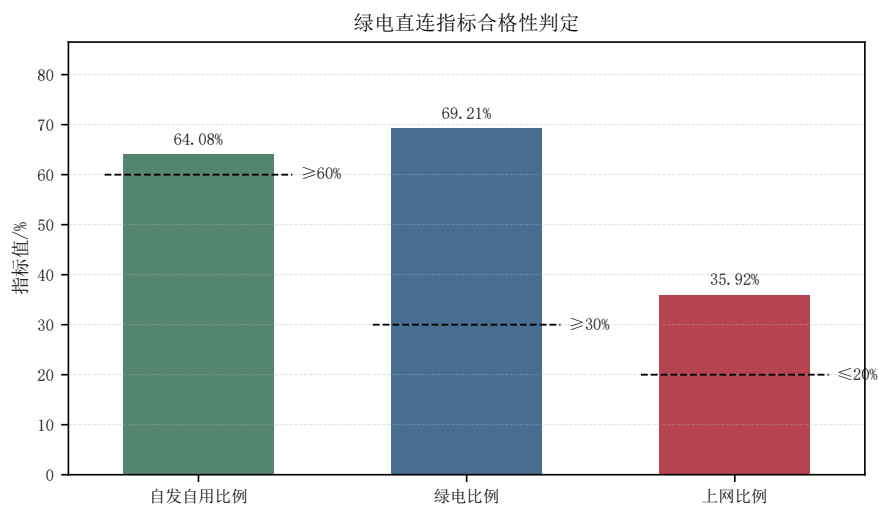


图 4: 问题一三项绿电指标达标情况

7 问题二：离散开停机制氨调节优化

7.1 模型建立

扩容后聚合生产单元满负荷功率为 41.5 MW，满负荷产氨能力为 3 t/h。设 $y_{s,D,t} \in \{0,1\}$ 表示场景 s 、日产量 D 、时段 t 是否满负荷开机，则

$$P_{s,D,t}^{prod} = 41.5 y_{s,D,t}, \quad m_{s,D,t}^{NH_3} = 3 y_{s,D,t}.$$

日产量集合取

$$D \in \{72, 63, 54, 45, 36\} \text{ t/d},$$

对应开机小时数

$$H_D = \frac{D}{3} \in \{24, 21, 18, 15, 12\}.$$

约束为

$$\sum_{t=1}^{24} y_{s,D,t} = H_D.$$

对给定场景和日产量，目标为最小化日净成本：

$$\min C_{s,D} = C_s^w + C_s^{pv} + C_{s,D}^{buy} + C_{s,D}^{om} - I_{s,D}^{sell}.$$

7.2 边际成本排序法

由于未给定启停成本、最小开机时间和爬坡约束，各时段之间仅通过总开机小时数耦合。定义时段 t 由停机变为开机的边际成本：

$$\Delta C_{s,t} = G_{s,t}(P_t^L + 41.5) - G_{s,t}(P_t^L) + C_{s,t}^{om,h},$$

其中

$$G_{s,t}(P) = 1000 \left[c_t^{buy} \max(P - P_{s,t}^{re}, 0) - c_t^{sell} \max(P_{s,t}^{re} - P, 0) \right].$$

将 24 个小时的 $\Delta C_{s,t}$ 从小到大排序，选择前 H_D 个时段开机即可得到全局最优解。

该排序法成立的关键在于，给定条件未包含启停成本、最小开停时间和爬坡率，时段之间除总开机小时数外没有耦合约束。风电和光伏发电成本对同一场景而言为常数，生产运维成本在开机小时内线性叠加，因此总成本可拆成 24 个小时边际成本之和。对每个场景和日产量，只需排序一次即可精确求解，计算复杂度为 $O(|T| \log |T|)$ ；24 个场景和 5 个日产量构成 120 个相互独立的小问题，计算量较小。

7.3 求解结果

典型风光场景结果见表4。以吨氨成本为单一准则时，36 t/d 方案最低；若要求三项绿电指标全部满足，54 t/d 方案是最低成本可行方案，吨氨成本为 4591.27 元/t。

表 4: 问题二典型场景离散开停机结果

日产量 /(t/d)	开机小时 /h	吨氨成本 /(元/t)	自发自用 比例	绿电比例	合格性
72	24	6219.14	93.26%	53.26%	全满足
63	21	5328.06	92.17%	59.66%	全满足
54	18	4591.27	90.08%	67.30%	全满足
45	15	3944.54	75.49%	66.67%	部分满足
36	12	3191.67	55.25%	59.68%	部分满足

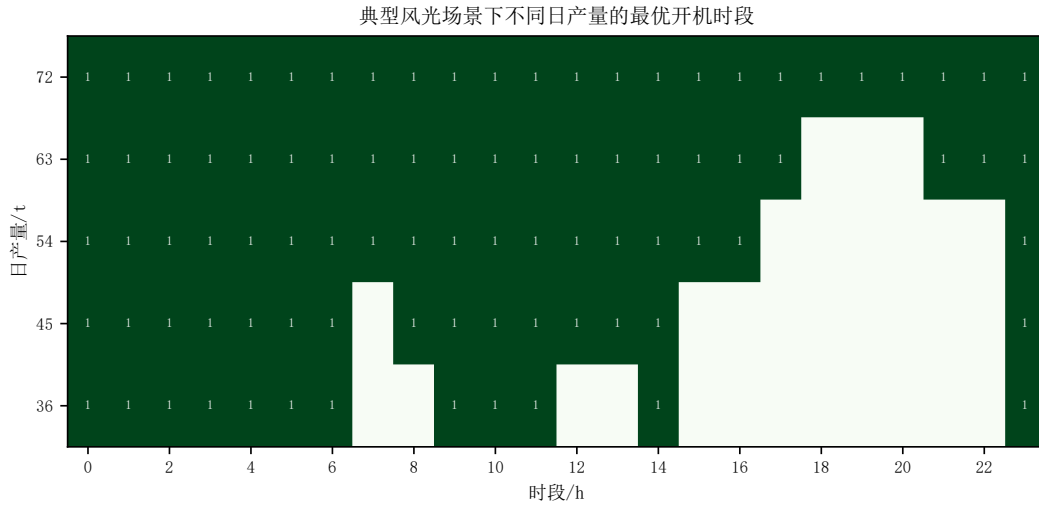


图 5: 典型风光场景下不同日产量的离散开机时段

图5给出了边际成本排序后的开机时段选择。72 t/d 方案必须全天运行，因此缺乏避开低风光或高电价时段的空间；随着日产量下降，模型优先保留夜间风电相对稳定、白天风光合计出力较高或净购电成本较低的小时，而剔除傍晚和部分低出力小时。该图表明，离散开停机调节具有明显的“块状”特征：它能够通过减少开机小时降低成本，但不能在同一小时内细化吸纳富余风光，因此低日产量下仍容易出现负荷规模不足和上网比例偏高的问题。

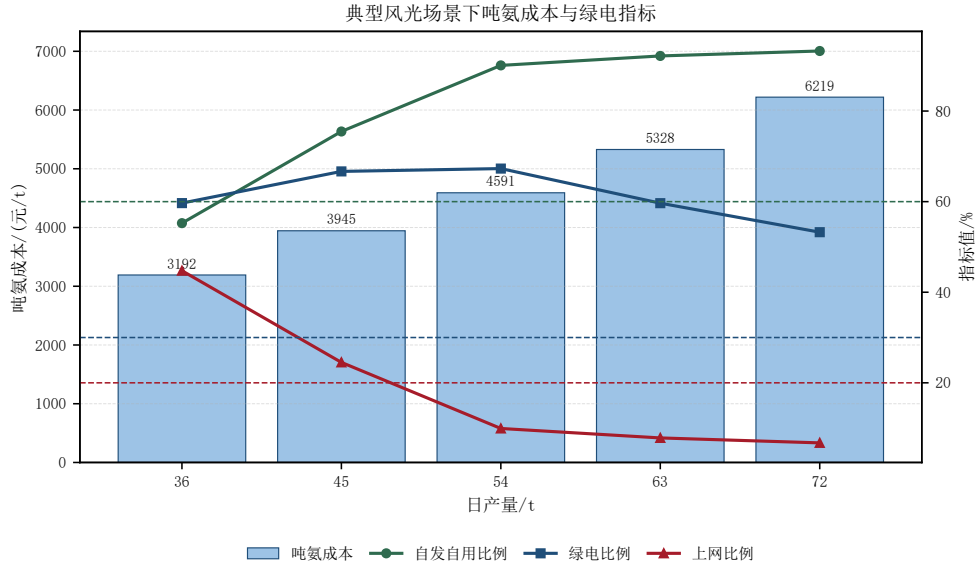


图 6: 离散开停机下典型场景日产量、吨氨成本和绿电指标权衡

图6进一步表明，离散调节的最低成本点并不等于政策合规点。36 t/d 通过减少高成本开机小时降低吨氨成本，但其自发自用比例和上网比例均偏离合规要求；54 t/d 在成本、产能利用率和三项绿电指标之间取得较好折中。24 场景年度加权结果见表5。日产量越低，吨氨成本越低，但年上网电量显著增加，导致全满足天数下降。

表 5: 问题二 24 场景年度加权结果

日产量 /(t/d)	年产氨 /t	年均吨氨成本 /(元/t)	年购电量 /MWh	年上网量 /MWh	全满足天数 /d
72	25920	7182.67	221146.80	12078.72	270
63	22680	6523.27	183462.13	19214.05	210
54	19440	5909.66	150249.86	30821.78	210
45	16200	5290.32	127977.50	53369.41	15
36	12960	4493.82	107277.55	77489.47	0

问题二表明，单纯降低日产量可以降低购电压力和吨氨成本，但也会削弱本地负荷对新能源的吸纳能力，使余电上网比例升高。因此，成本最优与指标合规之间存在明确权衡。

8 问题三：连续可调制氨负荷优化

8.1 模型建立

设 $u_{s,D,t}$ 为连续负荷率。主模型采用连续运行假设：

$$0.1 \leq u_{s,D,t} \leq 1,$$

并满足日产量约束

$$\sum_{t=1}^{24} 3u_{s,D,t} = D.$$

生产负荷为

$$P_{s,D,t}^{prod} = 41.5u_{s,D,t}.$$

并网功率平衡为

$$P_{s,t}^{re} + P_{s,D,t}^{buy} = P_t^L + 41.5u_{s,D,t} + P_{s,D,t}^{sell}.$$

为避免同一时段购电并售电的非物理套利，引入二进制变量 $b_{s,D,t}$ ：

$$P_{s,D,t}^{buy} \leq Mb_{s,D,t}, \quad P_{s,D,t}^{sell} \leq M(1 - b_{s,D,t}).$$

目标函数为

$$\min C_{s,D} = C_s^w + C_s^{pv} + C_{s,D}^{buy} + C_{s,D}^{com} - I_{s,D}^{sell}.$$

该模型为小规模混合整数线性规划，本文采用 HiGHS 求解。对每个场景和日产量，模型含 24 个负荷率变量、24 个购电变量、24 个上网变量和 24 个购售电互斥二进制变量，共 96 个变量；约束包括 24 个功率平衡约束、1 个日产量约束和 48 个购售电互斥约束。120 个场景-日产量组合彼此独立，计算稳定性较好；功率平衡、日产量约束、负荷率上下限和购售电互斥均满足要求，最大功率平衡误差为 1.42×10^{-14} MW。

8.2 结果与对比

典型场景下，连续调节与离散开停具有相同日产量集合。54 t/d 仍是满足三项指标的最低成本典型方案，吨氨成本为 4530.88 元/t。年度结果见表6。

表 6：问题三 24 场景年度加权结果

日产量 /(t/d)	年产氨 /t	年均吨氨成本 /(元/t)	年购电量 /MWh	年上网量 /MWh	上网比例	全满足天数 /d
72	25920	7182.67	221146.80	12078.72	7.05%	270
63	22680	6413.13	176326.80	12078.72	7.05%	300
54	19440	5663.95	137231.50	17803.42	10.39%	240
45	16200	5008.76	109563.64	34955.56	20.40%	240
36	12960	4209.48	91060.62	61272.54	35.76%	165

图7展示了典型风光场景下不同日产量的连续负荷率曲线。与问题二只能选择开机或停机不同，连续模型在风光合计出力较高的 8–16 时段保持较高负荷率，在晚间低出力时段将负荷率压至下限，从而把生产负荷从购电压力较大的时段转移到新能源富余时段。54 t/d 和 63 t/d 方案在白天基本贴近高负荷运行，同时仍能在低出力时段保留必要生产；36 t/d 方案虽调节幅度最大，但因总负荷规模偏小，仍不能完全吸收白天富余新能源，这解释了其成本下降与指标合规改善并不同步的原因。

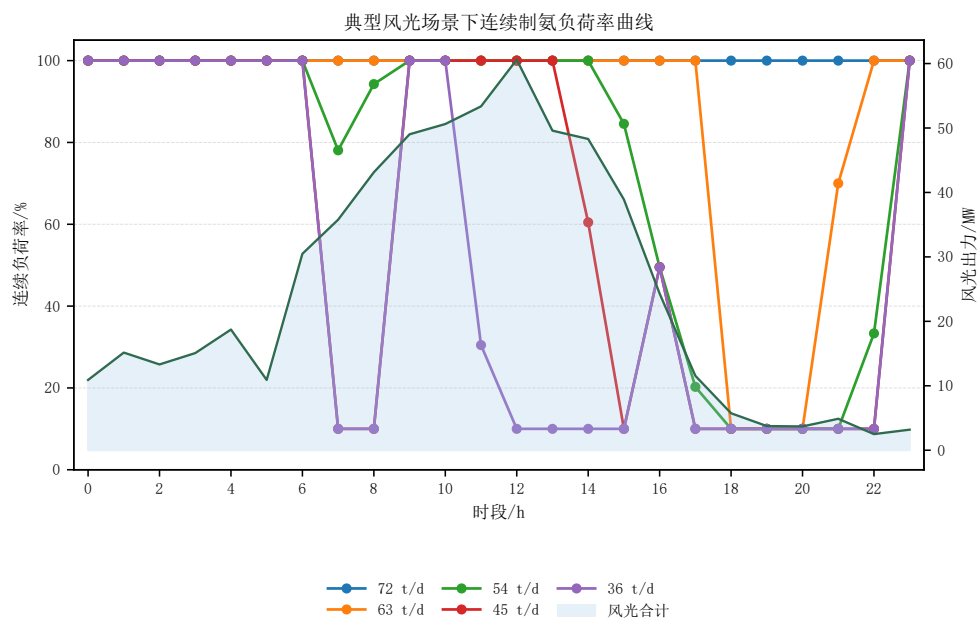


图 7: 典型风光场景下连续制氨负荷率调节曲线

与问题二相比，连续调节在除 72 t/d 外的所有日产量下均降低年均吨氨成本，见表7。36 t/d 方案降本 284.35 元/t，45 t/d 方案全满足天数由 15 天提高到 240 天，表明连续可调负荷能够显著改善源荷时序匹配。

表 7: 问题二与问题三年度结果对比

日产量/(t/d)	Q2 成本	Q3 成本	降本/(元/t)	全满足天数变化
72	7182.67	7182.67	0.00	0
63	6523.27	6413.13	110.15	+90
54	5909.66	5663.95	245.71	+30
45	5290.32	5008.76	281.55	+225
36	4493.82	4209.48	284.35	+165

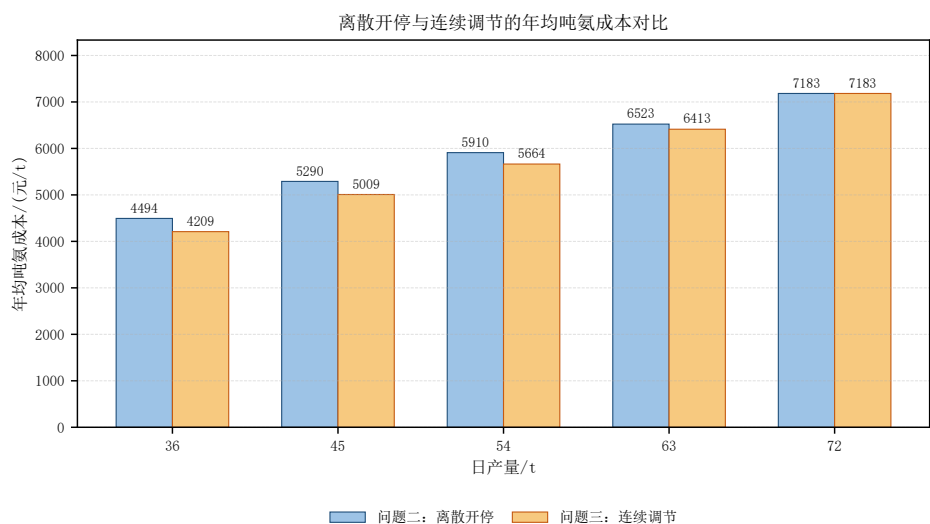


图 8：离散开停机与连续调节的年均吨氨成本对比

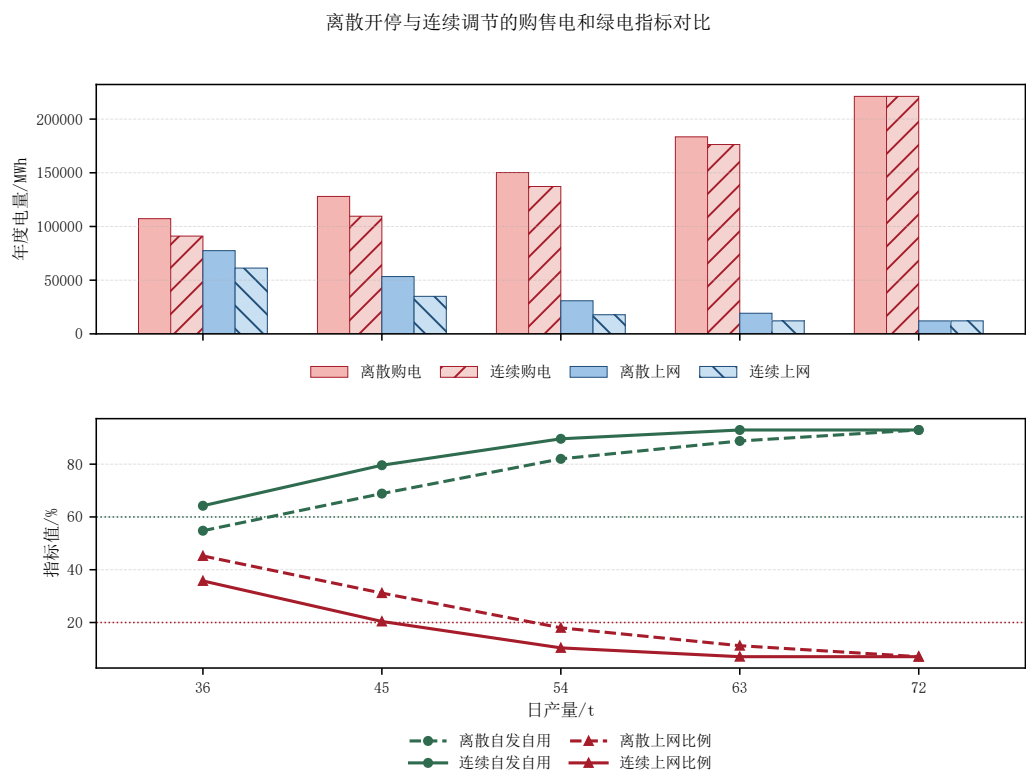


图 9：离散开停机与连续调节的电网交换和指标对比

图8和图9分别从成本和电网交换角度展示连续调节的优势：除 72 t/d 外，连续模型均降低年均吨氨成本，并同步削减部分购电和上网电量。

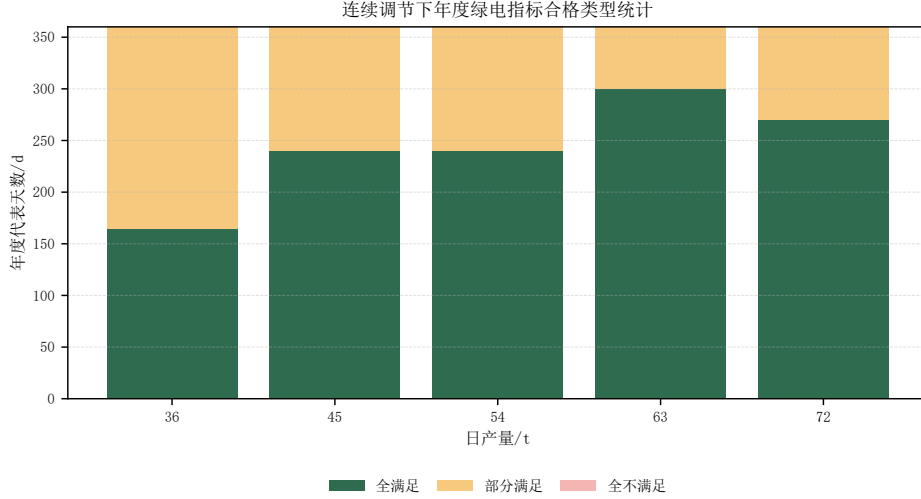


图 10: 连续调节下 24 场景年度绿电指标合格类型统计

72 t/d 因全天满负荷运行，连续模型没有调节自由度，结果与离散模型一致。中低日产量下，连续负荷率可在风光富余或电价较低时提高负荷，在风光不足或电价较高时降低负荷，从而同步减少购电和上网。图10显示，连续调节不仅降低成本，还显著改变合格类型分布：45 t/d 方案全满足天数由问题二的 15 天提升至 240 天，36 t/d 方案由 0 天提升至 165 天。该结果表明，柔性负荷的价值主要体现为小时级源荷匹配能力，而不仅是运行成本下降。

9 问题四：离网运行与储能配置优化

9.1 无储能离网模型

离网运行取消外部购售电。无储能时功率平衡为

$$P_{s,t}^w + P_{s,t}^{pv} + P_{s,t}^{shed} = P_t^L + 41.5u_{s,t} + P_{s,t}^{cur}.$$

若风光扣除常规负荷后的可用功率低于制氨 10% 运行下限，则制氨停机；若风光不足以满足常规负荷，则记录缺供。离网部分不再计算上网比例，而采用

$$R_s^{use} = \frac{E_s^{re} - E_s^{cur}}{E_s^{re}}, \quad R_s^{cur} = \frac{E_s^{cur}}{E_s^{re}}.$$

9.2 含储能离网模型

引入储能后，离网功率平衡为

$$P_{s,t}^w + P_{s,t}^{pv} + P_{s,t}^{dis} + P_{s,t}^{shed} = P_t^L + 41.5u_{s,t} + P_{s,t}^{ch} + P_{s,t}^{cur}.$$

储能 SOC 动态为

$$E_{s,t} = (1 - \rho)E_{s,t-1} + \eta^{ch}P_{s,t}^{ch} - \frac{P_{s,t}^{dis}}{\eta^{dis}},$$

并采用日内闭环 $E_{s,24} = E_{s,0}$ 。容量与充放电互斥约束为

$$0 \leq E_{s,t} \leq E^{cap},$$

$$0 \leq P_{s,t}^{ch} \leq P^{cap} y_{s,t}^b, \quad 0 \leq P_{s,t}^{dis} \leq P^{cap}(1 - y_{s,t}^b),$$

其中 $y_{s,t}^b \in \{0, 1\}$ 。由于未给出独立功率成本参数，储能按 1C 配置取 $P^{cap} = E^{cap}$ 。

调度目标采用字典序近似：优先避免常规负荷缺供，其次提高制氨产量，再减少弃电和储能无效循环：

$$\min \lambda_{shed} \sum_t P_{s,t}^{shed} - \lambda_{NH_3} \sum_t 3u_{s,t} + \lambda_{cur} \sum_t P_{s,t}^{cur} + c_{bat}^{om} \sum_t (P_{s,t}^{ch} + P_{s,t}^{dis}).$$

储能日折算投资成本为

$$C^{bat,d} = \frac{1000 E_{kWh}^{cap}}{15 \times 360}.$$

该目标函数采用足够大的权重模拟工程调度优先级，而非将缺供、产氨和弃电直接加总：常规负荷缺供首先被强惩罚，随后最大化绿氨产量，最后在同等产量下减少弃电和储能无效循环。储能模型含负荷率、开机状态、充电、放电、SOC、弃电、缺供和充放电模式变量，采用日内 SOC 闭环处理代表日循环。数值解满足离网功率平衡、SOC 动态、SOC 上下界和充放电互斥要求；有储能离网功率平衡最大误差为 7.11×10^{-15} MW，SOC 动态最大误差为 6.66×10^{-15} MWh。

9.3 储能容量配置与 Pareto 权衡

针对无储能弃电最大的场景 $W4_PV1$ 进行 0–300 MWh 容量扫描，主经济方案选择能够提高无储能日产氨量的正储能容量中吨氨成本最低者。为避免将储能配置压缩为单点结论，进一步采用 ε 约束：

$$\min_{E^{cap}} LCOA(E^{cap}), \quad \frac{E^{cur}(E^{cap})}{E^{re}} \leq \varepsilon, \quad E^{shed} = 0.$$

9.4 结果分析

离网模式对比见表8。无储能离网时，年产氨量为 9781.56 t，产能利用率仅 37.74%，年弃电 14188.67 MWh，并存在 8.29 MWh 常规负荷缺供。配置 5 MWh 储能后，常规负荷缺供降为 0，年弃电降至 11280.74 MWh，年均吨氨成本为 4046.47 元/t。

表 8：问题四离网、储能与并网同产量对比

模式	储能/MWh	年产氨/t	产能利用率	吨氨成本/(元/t)	年弃电/MWh
离网无储能	0	9781.56	37.74%	4056.31	14188.67
离网 + 储能	5	9972.26	38.47%	4046.47	11280.74
并网同产量	—	9972.26	38.47%	3310.66	—

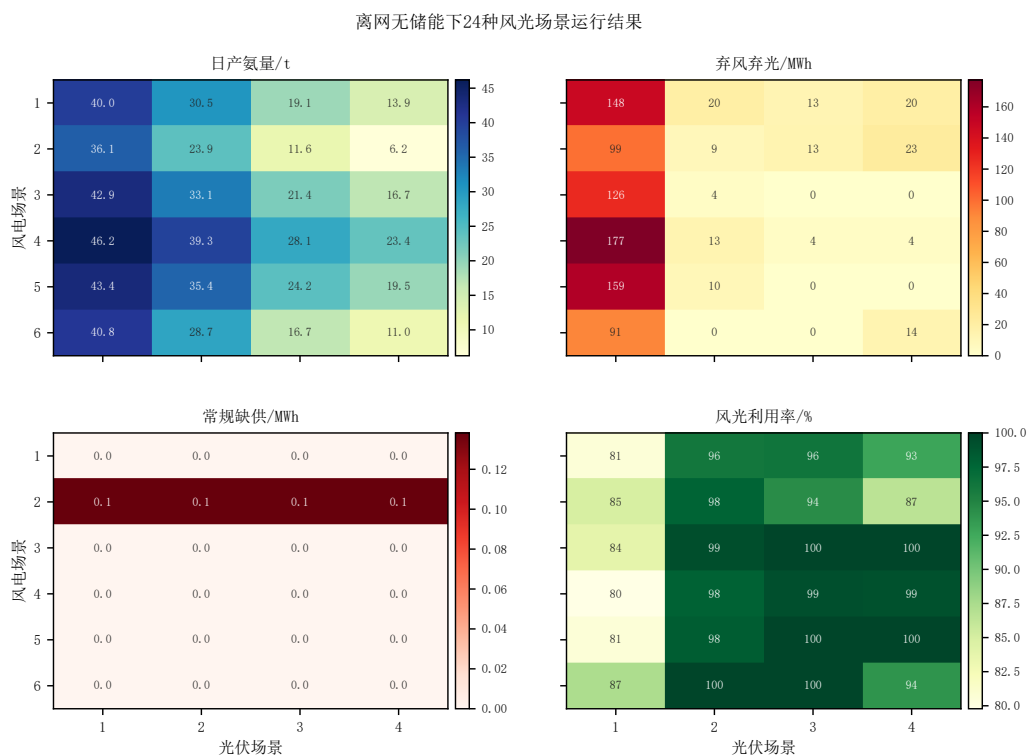


图 11: 无储能离网下 24 种风光组合场景的产氨、弃电、缺供与风光利用率

图11进一步揭示离网无储能结果的场景差异。日产氨量随风电场景增强显著提高，说明夜间和低光伏时段主要依赖风电维持制氨最低运行；光伏增强虽能提高日间风光利用率，但在负荷吸收能力有限时也会推高局部弃电。常规负荷缺供集中出现在低风场景，且绝对电量较小，表明离网风险并非全年平均意义上的电量不足，而是少数小时的瞬时功率不足。因此，储能的首要作用并非简单增加年电量，而是跨小时搬移少量关键电量，用于覆盖低出力边界时段并减少高出力时段弃风弃光。

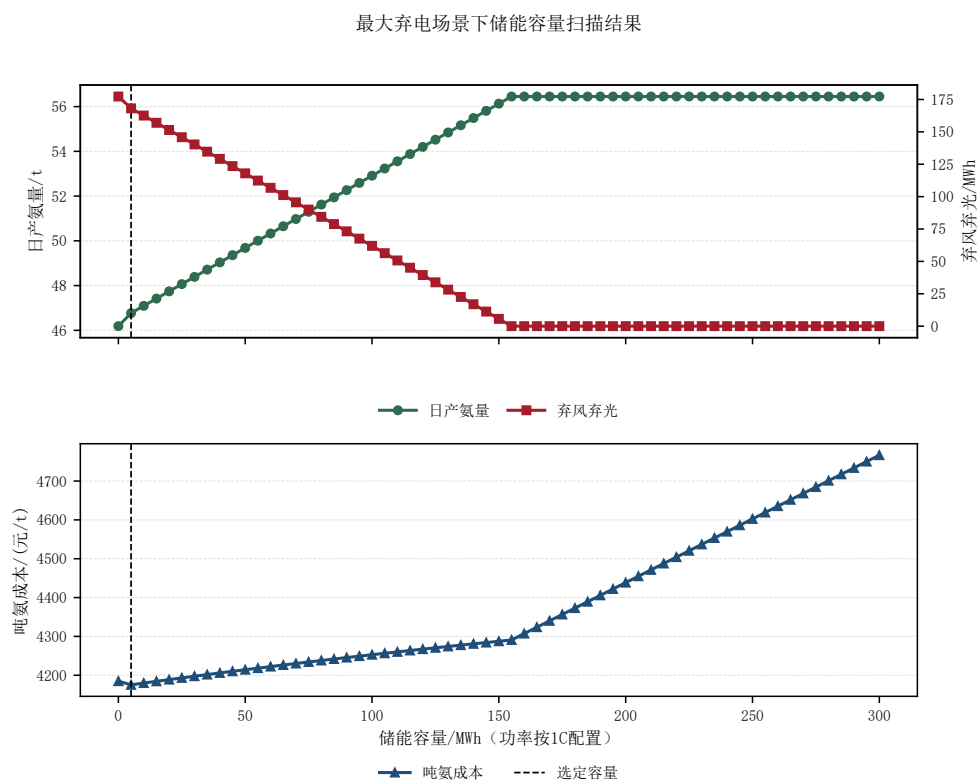


图 12: 最大弃电场景储能容量扫描与边际收益递减

图12显示，储能容量增加后弃电量下降、日产氨量提高，但吨氨成本并非单调下降。小容量储能优先消除关键时段缺供并搬移少量富余风光，边际收益较高；容量继续增大后，新增储能主要服务于降低弃电率，投资折算成本逐渐超过边际产氨收益。最大弃电场景的储能 Pareto 结果见表9和图13。5 MWh 是经济方案，但并非消纳最优方案；若要求弃电率不超过 10%，需 80 MWh；若要求零弃电，需 155 MWh，吨氨成本升至 4290.85 元/t。

表 9: 最大弃电场景储能经济-消纳权衡

方案	储能/MWh	日产氨/t	弃电率	吨氨成本/(元/t)
弃电率不超过 20%	5	46.77	19.17%	4175.30
弃电率不超过 10%	80	51.62	9.63%	4237.95
弃电率不超过 5%	120	54.20	4.50%	4267.18
弃电率不超过 1%	150	56.13	0.64%	4287.50
零弃电	155	56.45	0.00%	4290.85

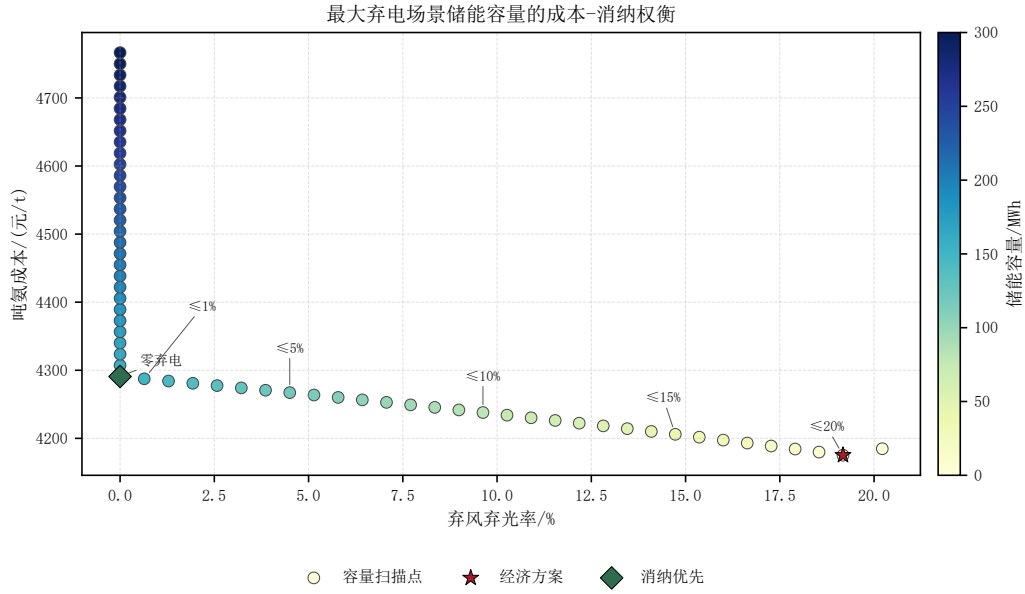


图 13: 最大弃电场景储能容量的成本-消纳权衡

为量化公共电网的平衡支撑价值，本文以离网 + 储能方案的各场景日产氨量为目标，求并网同产量调度。单位产品支撑价值定义为

$$v^{grid} = \frac{C^{off+bat,year} - C^{grid,year}}{M^{NH_3,year}}.$$

计算得

$$v^{grid} = 4046.47 - 3310.66 = 735.81 \text{ 元/t.}$$

该值可理解为公共电网为园区提供备用、互济和余电消纳服务所体现的单位产品经济价值。

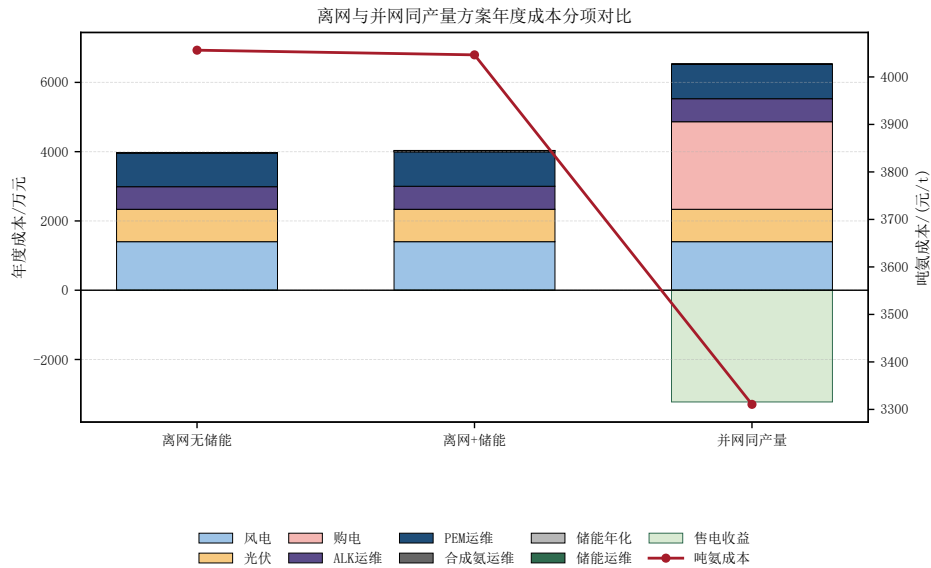


图 14: 离网与并网同产量的成本分项对比

图14进一步表明，并网同产量方案的成本优势主要来自公共电网提供的时空互济、备用和余电消纳能力，并非离网储能方案本身调度失效。

10 问题五：绿电直连高渗透影响分析

10.1 证据矩阵构建

问题五不再建立新的调度模型，而是从前四问提取数值证据，形成“模型依据–系统影响–政策含义”的分析链条。设园区与公共电网的交换功率为

$$P_t^{ex} = P_t^{buy} - P_t^{sell},$$

则高渗透后区域聚合交换功率为 $P_t^{EX} = \sum_i P_{i,t}^{ex}$ 。因此，绿电直连项目不宜仅按年净电量评价，还应关注交换功率峰谷差、反送时段、购售电总量、柔性负荷削减购售电的能力以及公共电网支撑价值。本文用

$$\Delta E^{buy} = E^{buy,discrete} - E^{buy,continuous}, \quad \Delta E^{sell} = E^{sell,discrete} - E^{sell,continuous}$$

度量连续调节相对离散开停的电网交换改善，用 v^{grid} 度量并网模式相对离网自平衡的系统支撑价值。核心证据见表10。

表 10：前四问支撑问题五的关键证据

编号	来源	数值证据	政策含义
E1	Q1	上网比例 35.92%，同时购电 172.04 MWh、上网 216.77 MWh	应考核小时级源荷匹配
E2	Q2	36 t/d 成本最低但 0 天全满足，72 t/d 全满足 270 天但成本最高	运行考核不宜仅看吨成本
E3	Q3	36 t/d 连续调节较离散开停降本 284.35 元/t，购电和上网均减少 16216.93 MWh	应重视柔性负荷价值
E4	Q3	72 t/d 上网比例 7.05%，36 t/d 上网比例 35.76%	新能源装机应以荷定源
E5	Q4	离网无储能产能利用率 37.74%，5 MWh 储能后缺供为 0	离网需评估可靠性与储能
E6	Q4	公共电网支撑价值 735.81 元/t	并网项目应承担系统费用
E7	Q4	满产逐小时自治需 1156.31 MW 至 3006.39 MW 装机	完全离网满产成本高
E8	Q4	5 MWh 为经济方案，80 MWh 满足弃电率不超过 10%，155 MWh 零弃电	储能需多目标评估

10.2 系统影响与政策建议

绿电直连园区的有利影响主要体现在三方面：第一，高耗能绿氢绿氨负荷能够吸收本地风光电量，促进新能源就近消纳；第二，连续可调电解槽和合成氨装置可作为需求侧资源，降低公共电网购售电波动；第三，储能与柔性负荷组合有助于提升离网或弱联网场景下的可靠性。

同时，绿电直连高渗透也会带来风险。若项目只追求最低吨氨成本，可能选择低产量运行，导致上网比例升高并增加外部电网消纳压力。并网型项目若不承担系统运行费、备用和辅助服务费用，可能将平衡成本转移给公共电网。完全离网虽然强化物理绿电属性，但需要更大储能和冗余装机，经济性与土地资源约束突出。

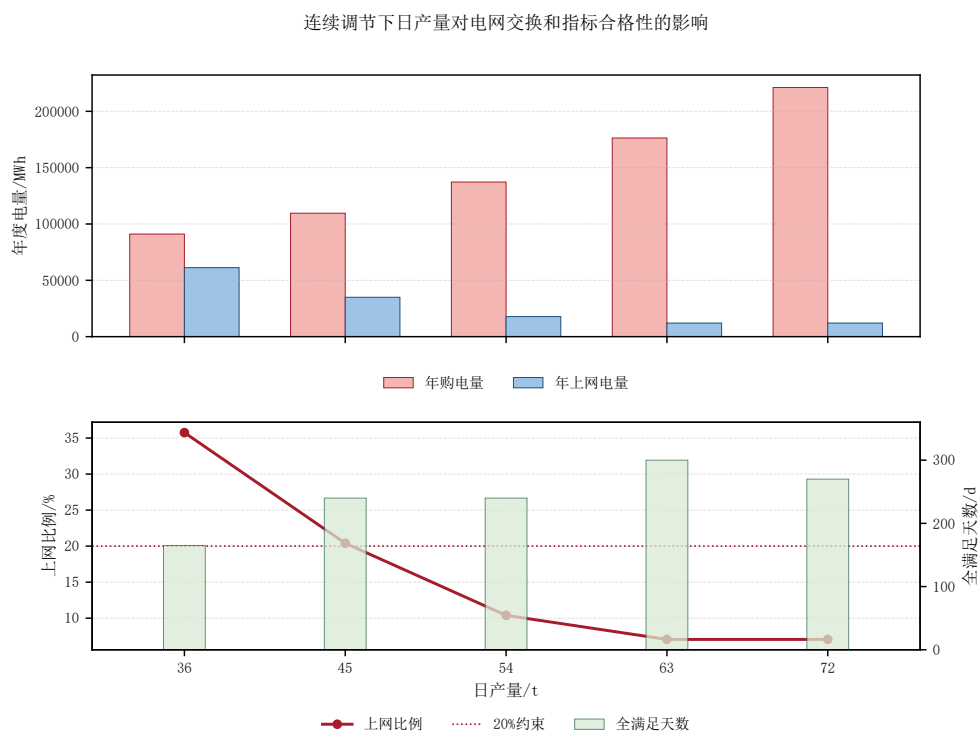


图 15: 连续调节下日产量对电网交换和指标合格性的影响

图15表明，日产量提高会增强本地消纳并降低上网比例，但也会提高购电需求和吨氨成本。基于这一权衡，本文提出六条建议：建立小时级绿电溯源与分表计量制度；将上网比例、最大交换功率和反送时段纳入运行考核；把柔性负荷能力作为准入、评价和补偿对象；建立储能配置的经济性、弃电率和可靠性多目标评估；完善并网型项目的输配电费、系统运行费、备用和辅助服务费用分担机制；推进风光氢氨醇一体化与弱联网示范 [3, 4, 5]。

这些建议均对应前四问的定量证据，具体映射见表11：小时级溯源对应问题一“同日购电与上网并存”的现象；柔性负荷补偿对应问题三购售电同步削减和全满足天数提升；储能多目标评价对应问题四经济容量与零弃电容量的分离；系统费用分担对应735.81 元/t 的公共电网支撑价值。因此，问题五的政策分析由运行模型推导而来，其目标是将数值结果转化为监管指标、市场机制和工程准入规则。

表 11: 政策建议与模型证据对应关系

建议	模型依据	预期作用
小时级绿电溯源与分表计量	Q1 同日电 172.04 MWh、上网 216.77 MWh；Q3 低产量上网比例 35.76%	避免年累计电量掩盖小时错配，提升绿电认证可信度
上网比例与最大交换功率联合考核	Q1 上网比例 35.92% 超标；低负荷运行反送压力上升	抑制局部潮流反转和新能源消纳困难时段反送
柔性负荷准入与补偿	Q3 中 36 t/d 购电和上网均较 Q2 减少 16216.93 MWh	将制氢制氨负荷纳入需求响应和辅助服务资源
储能经济-消纳-可靠性多目标评估	Q4 中 5、80、155 MWh 分别对应经济、低弃电和零弃电方案	避免单一容量比例导致过度储能或储能不足
系统费用和备用责任分担	Q4 测算公共电网支撑价值 735.81 元/t	防止平衡成本向其他用户转移
弱联网示范与源网荷储协同	完全离网满产自治需 1156.31-3006.39 MW 装机	在产业脱碳、系统安全性和经济性之间取得折中

11 结果汇总与综合讨论

前四问结果表明，绿电直连型电氢氨园区的运行优化不能简化为最低吨氨成本问题。问题一显示，刚性满负荷生产下虽然绿电比例较高，但上网比例超标。问题二说明，降低日产量可降低成本，却会削弱新能源就地消纳。问题三表明，连续负荷率调节在相同产量下可进一步降低成本并提高指标合格天数，是改善源荷匹配的关键柔性手段。问题四则表明，离网运行需要储能和冗余容量支撑，公共电网的平衡服务具有可量化经济价值。

从工程决策角度看，若以典型场景合规为目标，54 t/d 是问题二和问题三中较均衡的日产量方案；若以全年全满足天数最大为目标，问题三中 63 t/d 表现最好；若以离网可靠性为目标，5 MWh 储能可消除常规负荷缺供，但若追求低弃电或零弃电，则需显著提高储能容量。表12汇总了五问之间的量化发现和决策含义。上述结果表明，绿电直连项目应在吨氨成本、产能利用率、绿电指标、弃电率和电网支撑成本之间进行综合权衡。

表 12: 五问核心发现与决策含义汇总

问题	核心量化发现	对后续决策的含义
Q1	典型日上网比例 35.92%，超过 20% 约束	固定制氨负荷会造成小时级源荷错配，必须引入负荷调节或储能
Q2	36 t/d 成本最低但 0 天全满足；54 t/d 为典型合规最低成本方案	离散降负荷可降本，但产能利用率和本地消纳能力同步下降
Q3	36 t/d 较 Q2 降本 284.35 元/t，45 t/d 全满足天数提高 225 天	连续可调负荷是同时改善成本和合规性的关键柔性资源
Q4	5 MWh 消除缺供，80 MWh 满足最大弃电场景弃电率不超过 10%，155 MWh 零弃电	储能配置必须采用经济性、消纳率和可靠性的多目标权衡
Q5	公共电网支撑价值 735.81 元/t	并网绿电直连项目应承担系统平衡费用，并以小时级指标监管

12 模型评价与推广

本文模型具有三方面特点。第一，五问均服从同一小时级功率平衡、同一场景权重、同一给定成本口径和同一绿电指标体系，因此结果可横向比较。第二，模型没有停留在“成本最小”单目标层面，而是持续跟踪自发自用比例、绿电比例、上网比例、弃电率、缺供量、产能利用率和公共电网支撑价值。第三，问题三和问题四引入互斥二进制变量，避免低谷购电价低于上网价时出现同小时购电并上网的非物理解，也避免储能同时充放电；问题四将储能经济最优与消纳最优分离，问题五将模型输出转换为政策证据，使数学模型具有工程和监管解释能力。

与前沿研究相比，本文属于给定数据与约束范围内的确定性规划调度模型。国际绿氨优化文献通常进一步采用设计-运行协同优化，同时优化风光、电解槽、Haber-Bosch 装置、电池、氢储罐和氨储罐容量；也常用长期气象序列、鲁棒优化、机会约束或 CVaR 度量风光不确定性 [6, 7, 8, 9, 10, 11, 12, 13]。本文的主要贡献在于指标口径统一、五问结果可比、模型证据能够支撑政策讨论；不足是设备容量大多固定，风光不确定性仅由 24 个代表场景近似，且未刻画合成氨反应回路的热惯性和压力动态。因此，本文结论适

合作为给定条件下的规划调度和政策分析依据，而非直接替代工程可研中的全寿命成本和动态安全校核。

模型局限主要包括四类。第一，真实 Haber-Bosch 合成氨系统受温度、压力、循环气、催化剂寿命和安全联锁约束，不能像一般电负荷一样快速波动。第二，本文将电解槽和合成氨装置同步聚合，未显式配置氢储罐和氨储罐，因而没有刻画制氢与制氨之间的物料缓冲能力。第三，吨氨成本未纳入给定资料未包含的电解槽、空分、压缩、储氢储氨和合成氨固定投资，因此不能视为完整工程 LCOA。第四，并网模型只在园区关口比较购售电量和成本，尚未考虑潮流、电压、无功、短路容量、保护配合和接入容量约束。

后续可在当前 MILP 框架基础上加入负荷爬坡率、启停成本和最小开停时间：

$$0.1z_t \leq u_t \leq z_t, \quad v_t \geq z_t - z_{t-1}, \quad -R^{down} \leq u_t - u_{t-1} \leq R^{up}.$$

也可引入氢储罐动态

$$H_t = (1 - \rho^H)H_{t-1} + Q_t^{H2,prod} - Q_t^{H2,use},$$

或采用 CVaR 刻画场景风险：

$$\min \sum_s p_s C_s + \lambda \left[\eta + \frac{1}{1 - \alpha} \sum_s p_s \xi_s \right], \quad \xi_s \geq C_s - \eta.$$

进一步结合配电网潮流、电压约束、辅助服务市场、容量电费、绿证和碳价机制，可将本文模型推广为源网荷储氢氨协同规划与运行评估工具。

13 结论

本文围绕绿电直连型电氢氨园区建立了统一的小时级优化分析框架。主要结论如下：

- (1) 刚性满负荷制氨下，典型日新能源自发自用比例为 64.08%，总用电量绿电比例为 69.21%，但上网比例为 35.92%，不满足不超过 20% 的要求，表明固定负荷存在显著源荷错配。
- (2) 离散开停机能够降低运行成本，但低日产量方案会提高上网电量。典型场景中，若要求三项指标全部满足，54 t/d 为最低成本离散方案。
- (3) 连续负荷率调节相对离散开停机优势突出。36 t/d 方案降本 284.35 元/t，全满足天数由 0 天提高至 165 天；45 t/d 全满足天数由 15 天提高至 240 天。
- (4) 离网运行下当前风光装机难以支撑高利用率生产。5 MWh 储能可消除常规负荷缺供并降低弃电，但若要求最大弃电场景弃电率不超过 10%，需 80 MWh 储能；若要求零弃电，需 155 MWh 储能。
- (5) 并网同产量方案的年均吨氨成本为 3310.66 元/t，离网 + 储能为 4046.47 元/t，公共电网支撑价值为 735.81 元/t，表明并网型绿电直连项目应合理承担系统平衡费用。
- (6) 政策上应坚持小时级源荷匹配、以荷定源、柔性负荷补偿、储能多目标评估和系统费用分担，促进新能源消纳、产业脱碳和电力系统安全协调统一。

参考文献

- [1] 国家发展改革委, 国家能源局. 关于有序推动多用户绿电直连发展有关事项的通知 [Z]. 2026.
- [2] 国家发展改革委, 国家能源局. 关于有序推动绿电直连发展有关事项的通知 [Z]. 2025.
- [3] 国家发展改革委, 国家能源局. 关于深化新能源上网电价市场化改革促进新能源高质量发展的通知 [Z]. 2025.
- [4] 国家发展改革委等. 关于大力实施可再生能源替代行动的指导意见 [Z]. 2024.
- [5] 国家发展改革委, 国家能源局. 关于推进电力源网荷储一体化和多能互补发展的指导意见 [Z]. 2021.
- [6] International Renewable Energy Agency. Innovation Outlook: Renewable Ammonia[R]. 2022.
- [7] International Energy Agency. Global Hydrogen Review[R]. 2025.
- [8] European Commission. Commission Delegated Regulation (EU) 2023/1184[R]. 2023.
- [9] Nayak-Luke R, Bañares-Alcántara R. Techno-economic viability of islanded green ammonia as a carbon-free energy vector and as a substitute for conventional production[J]. Energy & Environmental Science, 2020.
- [10] Rosa L, Tonelli M. Optimal design of decentralized ammonia production via electric Haber-Bosch systems[J]. Green Chemistry, 2026.
- [11] Hochhaus L, et al. Optimal scheduling of a large-scale power-to-ammonia process[J]. Computers & Chemical Engineering, 2023.
- [12] 乐生铜等. 考虑能量分配的电-氢-氨耦合系统多主体混合时间尺度优化调度 [J]. 电网技术, 2025.
- [13] 胡利等. 考虑氢氨储运的综合能源系统两阶段鲁棒优化经济调度 [J]. 电力建设, 2026.

A 附录 A：数值校验说明

问题三连续调节模型校验显示，最大功率平衡误差为 1.42×10^{-14} MW，同时购电与售电小时数为 0，日产量约束误差为 1.42×10^{-14} t。问题四离网模型校验显示，无储能与有储能功率平衡最大误差均为 7.11×10^{-15} MW，储能 SOC 动态最大误差为 6.66×10^{-15} MWh，储能同时充放电乘积最大值为 6.35×10^{-14} ，均为数值优化容差量级。

B 附录 B：补充图件

正文保留支撑主要结论的核心图件。本附录按“基准核算-离散调节-连续调节-离网储能-政策证据”的逻辑列出补充图件，用于展示未在正文展开的成本分布、场景差异、储能调度和电网支撑价值等结果。

B.1 问题一补充图件

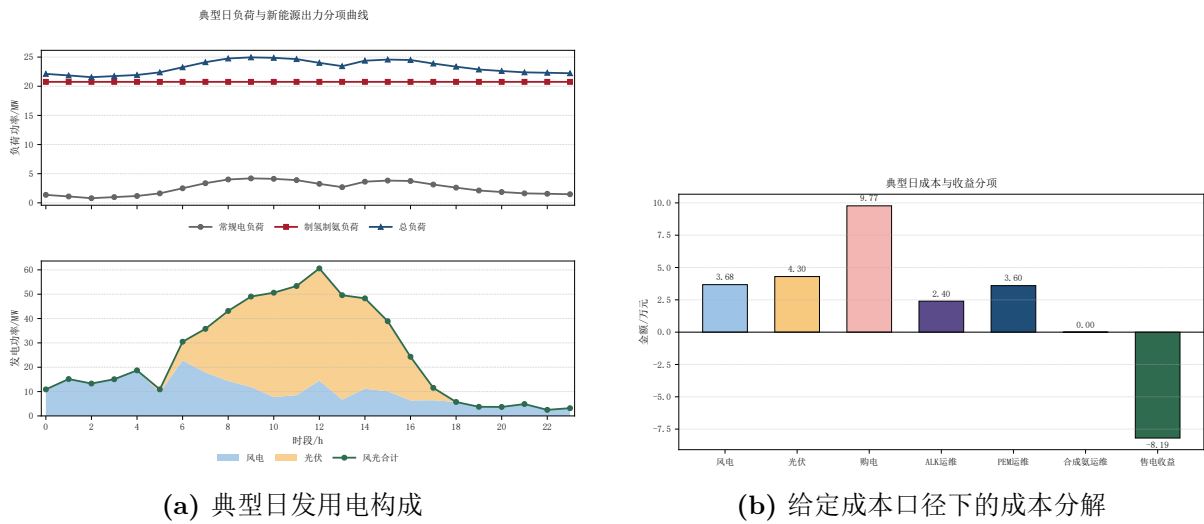
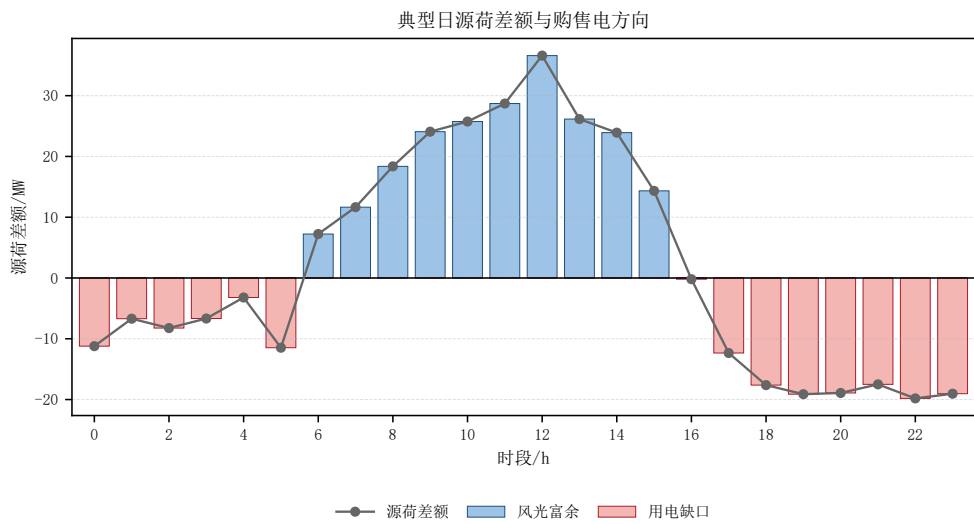


图 16：问题一基准工况补充结果



B.2 问题二补充图件

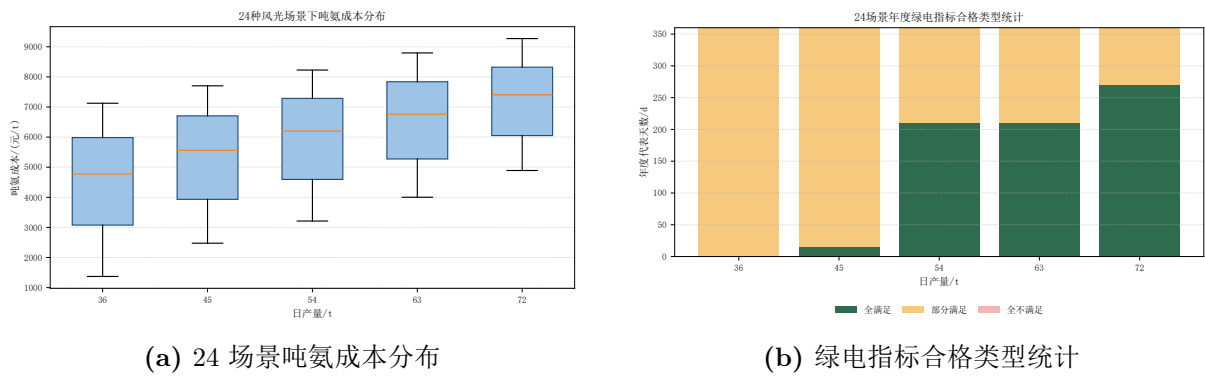


图 18: 问题二离散开停机的成本分布与合规性补充结果

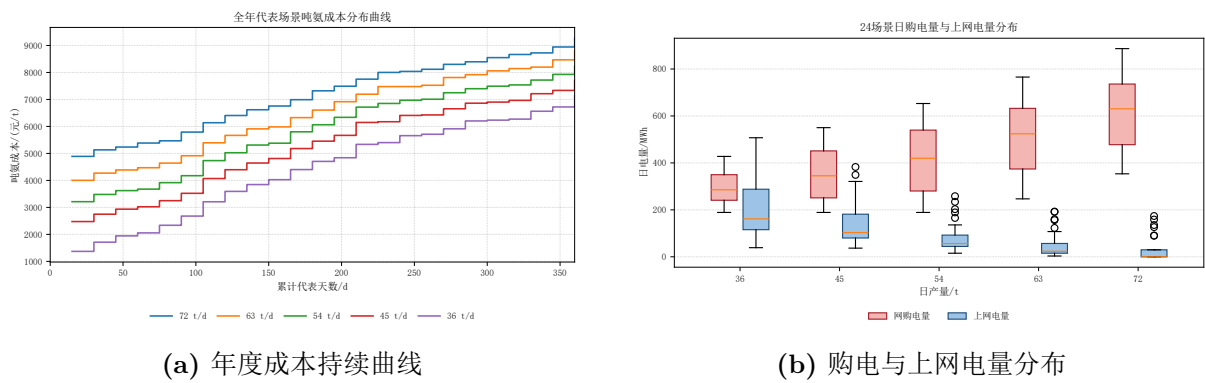


图 19: 问题二离散调节的年度经济性与电网交换补充结果

离散开停模式下24场景吨氨成本热力图（格内为千元/t）

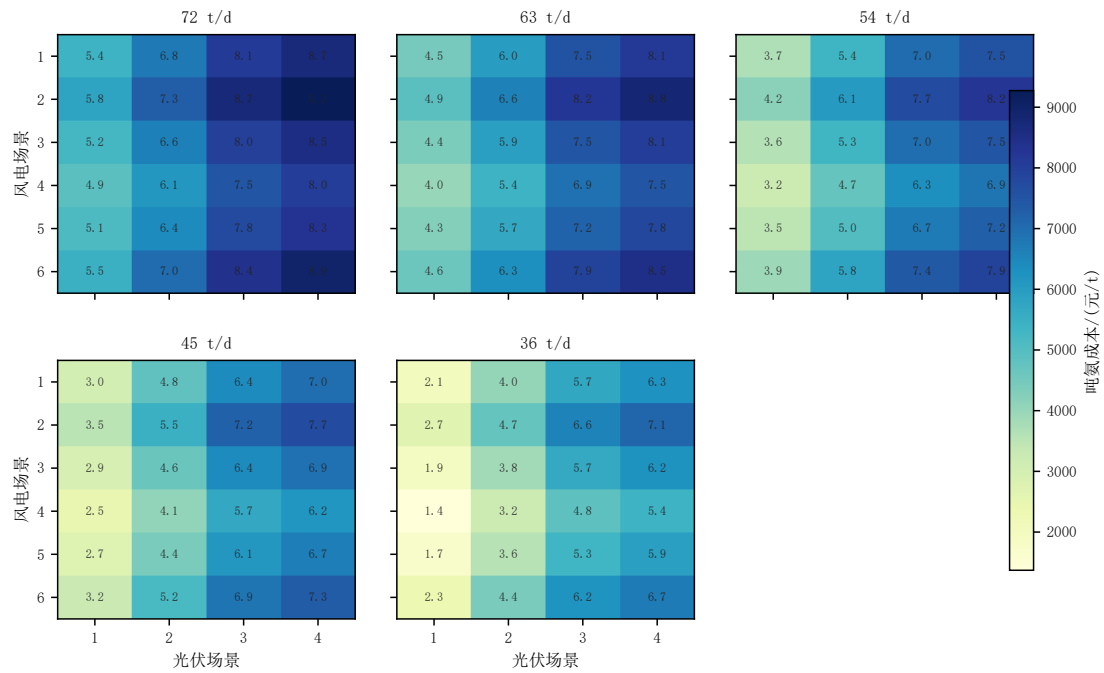


图 20: 问题二 24 种风光组合场景下的吨氨成本热力图

B.3 问题三补充图件

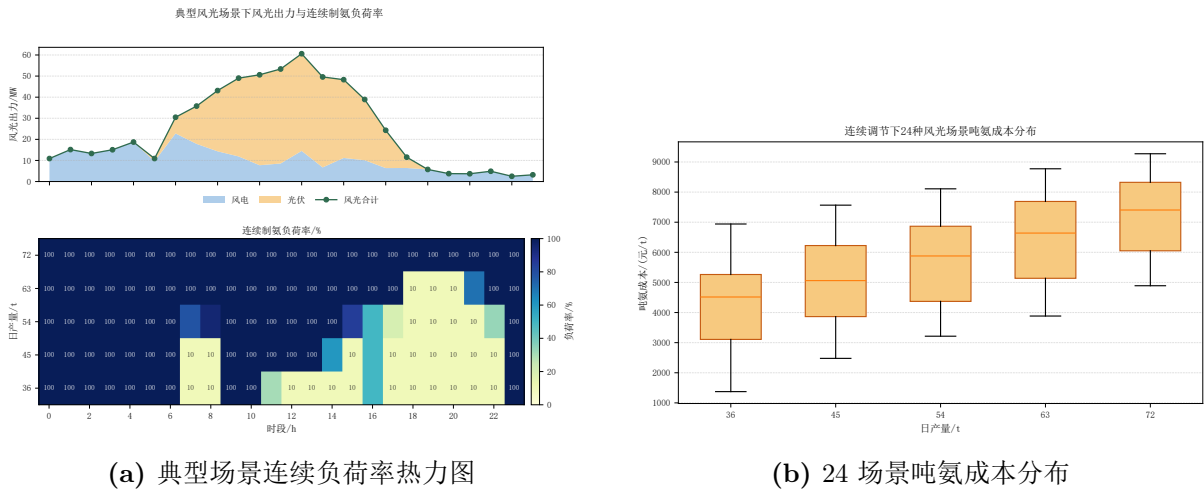
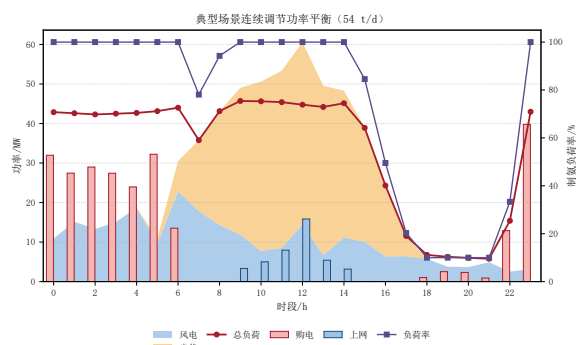
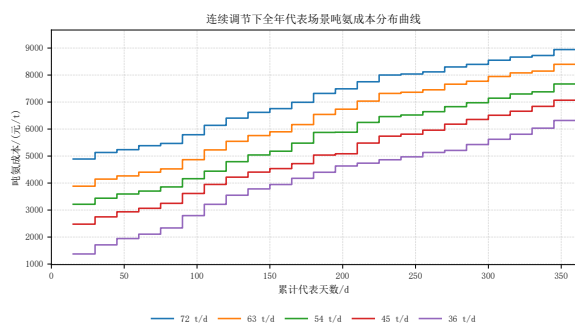


图 21: 问题三连续调节的负荷率与成本分布补充结果



(a) 典型场景功率平衡



(b) 年度成本持续曲线

图 22: 问题三连续调节的小时级运行和年度经济性补充结果

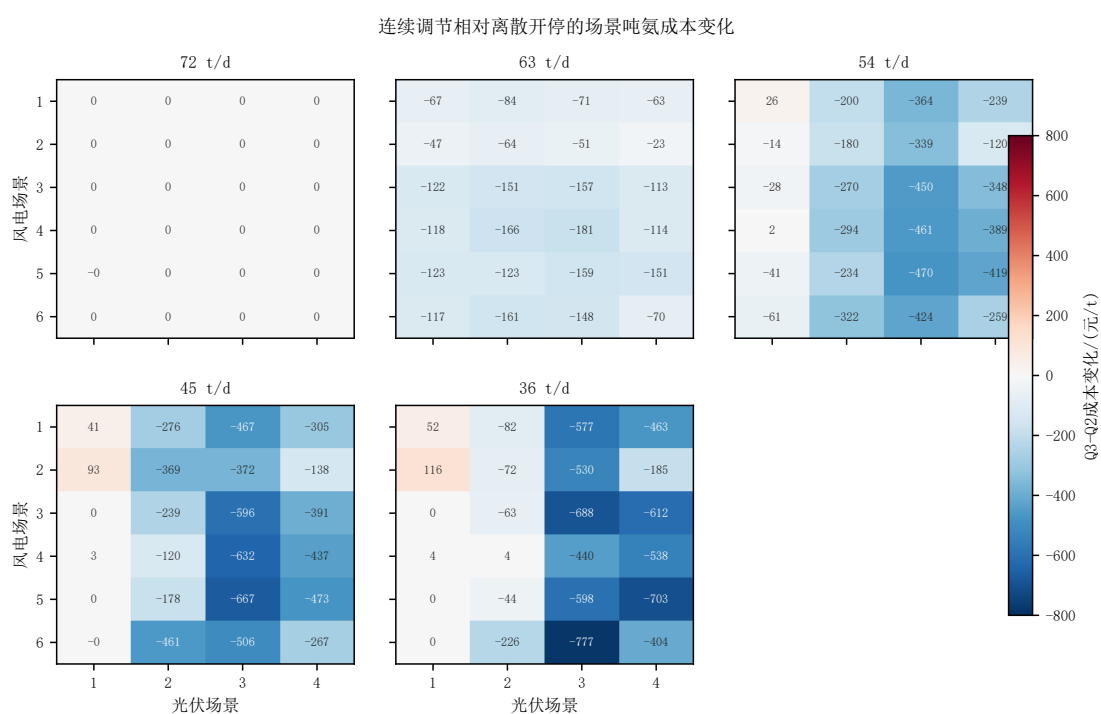


图 23: 问题三相对问题二的吨氨成本变化热力图

B.4 问题四补充图件

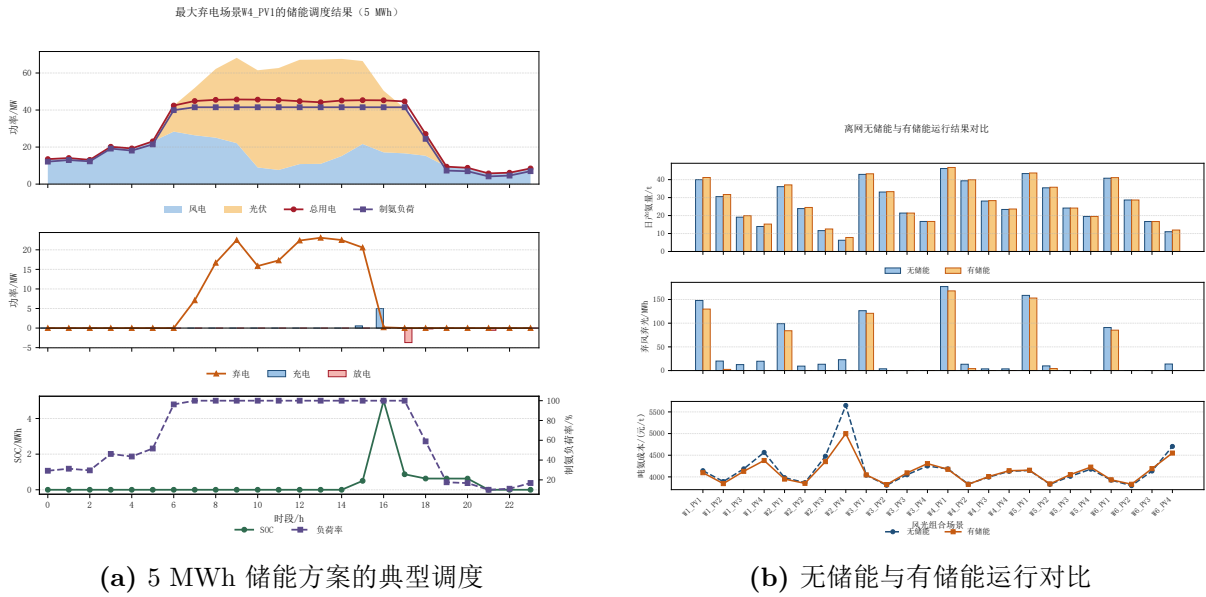


图 24: 问题四离网储能调度补充结果

B.5 问题五补充图件

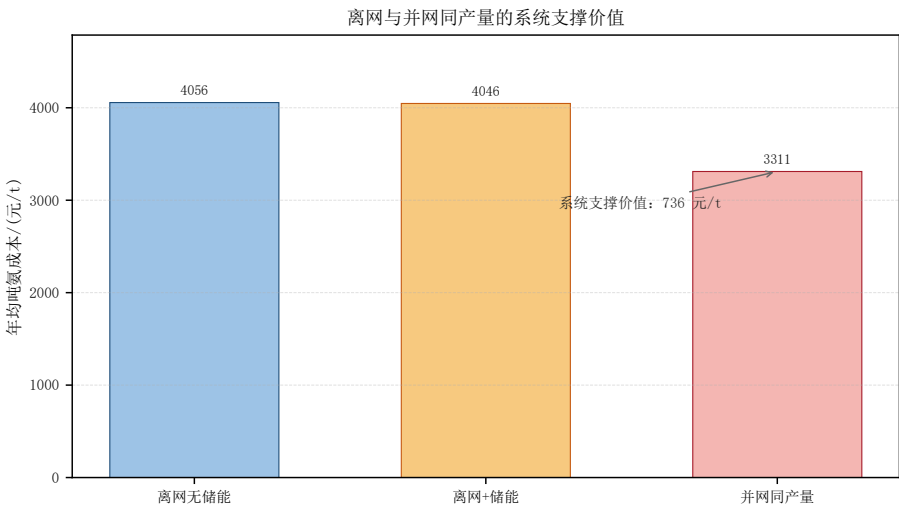


图 25: 问题五离网与并网同产量对比下的公共电网支撑价值

C 附录 C：结果表与明细数据

各问的完整数值表随支撑材料一并给出。正文和附录只选取关键汇总表与补充图件，小时级调度、24 场景明细和容量扫描数据以表格文件保存，便于对主要结论进行核验。

- Q1** q1_summary.csv: 典型日能量、指标和成本汇总。
q1_hourly_results.csv: 24 小时发电、负荷、购电、上网和电价明细。
- Q2** q2_typical_summary.csv: 典型场景汇总。
q2_annual_summary.csv: 年度加权结果。
q2_distribution_summary.csv: 成本分布统计。
q2_typical_hourly_schedule.csv: 典型场景小时级开停机调度。
q2_all_scenario_hourly_schedule.csv: 24 场景小时级开停机调度。
- Q3** q3_typical_summary.csv: 典型场景连续调节结果。
q3_annual_summary.csv: 年度加权结果。
q3_compare_q2_summary.csv: 相对问题二的增量价值对比。
q3_distribution_summary.csv: 成本分布统计。
q3_typical_hourly_schedule.csv: 典型场景小时级负荷率调度。
q3_all_scenario_hourly_schedule.csv: 24 场景小时级连续调节调度。
- Q4** q4_no_storage_annual_summary.csv: 离网无储能年度结果。
q4_storage_annual_summary.csv: 离网有储能年度结果。
q4_mode_comparison_summary.csv: 离网、储能与并网同产量对比。
q4_storage_capacity_scan.csv: 储能容量扫描。
q4_storage_pareto_summary.csv: 储能经济-消纳权衡。
q4_validation_summary.csv: 数值校验结果。
- Q5** q5_model_evidence_summary.csv: E1-E8 模型证据。
q5_policy_recommendation_matrix.csv: 政策建议矩阵。
q5_policy_reference_matrix.csv: 政策与报告引用依据。

D 附录 D：公共程序模块

本附录列出计算所用的公共 Python 程序。程序按照依赖关系排列：首先是包初始化文件，其次是数据读取与场景构造模块，再是指标核算模块，最后是统一绘图模块。各问题的求解程序均通过这些模块共享数据口径、绿电指标、成本指标和图件风格。

D.1 公共模块入口

文件：code/model_common/__init__.py

说明：声明公共工具包，使各问题程序能够统一导入数据、指标和绘图函数。

```
1 """Reusable modelling utilities for A problem."""
2
```

D.2 数据读取与场景构造模块

文件：code/model_common/data.py

说明：读取附件中的负荷、风电、光伏、电价和设备参数，并构造问题二至问题四使用的 24 个风光组合场景。

```
1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from pathlib import Path
5
6 import numpy as np
7 import pandas as pd
8
9
10 PROJECT_ROOT = Path(__file__).resolve().parents[1]
11
12
13 @dataclass(frozen=True)
14 class SystemParams:
15     load_peak_mw: float = 6.0
16     wind_capacity_mw: float = 40.0
17     pv_capacity_mw: float = 64.0
18     alk_power_mw_36t: float = 10.0
19     pem_power_mw_36t: float = 10.0
20     ammonia_power_mw_36t: float = 0.75
21     ammonia_output_t_per_h_36t: float = 1.5
22     alk_h2_kg_per_h_36t: float = 140.0
23     pem_h2_kg_per_h_36t: float = 160.0
24     wind_lcoe_yuan_per_kwh: float = 0.15
25     pv_lcoe_yuan_per_kwh: float = 0.12
26     alk_om_yuan_per_kwh: float = 0.10
27     pem_om_yuan_per_kwh: float = 0.15
28     ammonia_om_yuan_per_kwh: float = 0.002
29     sell_price_yuan_per_kwh: float = 0.3779
30     storage_capex_yuan_per_kwh: float = 1000.0
31     storage_life_year: float = 15.0
32     storage_charge_eff: float = 0.90
33     storage_discharge_eff: float = 0.90
34     storage_self_loss_ratio: float = 0.002
35
36     def scale(self, daily_ammonia_t: float) -> float:
37         return daily_ammonia_t / 36.0
38
39     def process_power_mw(self, daily_ammonia_t: float = 36.0) -> float:
40         k = self.scale(daily_ammonia_t)
41         return (self.alk_power_mw_36t + self.pem_power_mw_36t + self.ammonia_power_mw_36t) * k
42
43     def alk_power_mw(self, daily_ammonia_t: float = 36.0) -> float:
44         return self.alk_power_mw_36t * self.scale(daily_ammonia_t)
```

```

45
46 def pem_power_mw(self, daily_ammonia_t: float = 36.0) -> float:
47     return self.pem_power_mw_36t * self.scale(daily_ammonia_t)
48
49 def ammonia_power_mw(self, daily_ammonia_t: float = 36.0) -> float:
50     return self.ammonia_power_mw_36t * self.scale(daily_ammonia_t)
51
52 def ammonia_output_t_per_h(self, daily_ammonia_t: float = 36.0) -> float:
53     return self.ammonia_output_t_per_h_36t * self.scale(daily_ammonia_t)
54
55
56 def find_input_file(keyword: str, root: Path = PROJECT_ROOT) -> Path:
57     matches = sorted(root.glob(f"*{keyword}*.xlsx"))
58     if not matches:
59         raise FileNotFoundError(f"Cannot find Excel file containing keyword: {keyword}")
60     return matches[0]
61
62
63 def normalize_hour_label(label: object) -> str:
64     return str(label).strip().replace(" ", ":")
65
66
67 def purchase_price(hour: int) -> float:
68     if 10 <= hour < 15 or 18 <= hour < 21:
69         return 0.8024
70     if 7 <= hour < 10 or 15 <= hour < 18 or 21 <= hour < 23:
71         return 0.6074
72     return 0.3424
73
74
75 def load_typical_day(params: SystemParams | None = None, root: Path = PROJECT_ROOT) -> pd.DataFrame:
76     params = params or SystemParams()
77     load_raw = pd.read_excel(find_input_file("附件 1", root))
78     renew_raw = pd.read_excel(find_input_file("附件 2", root))
79
80     load_raw.columns = ["time", "load_pu"]
81     renew_raw.columns = ["time", "wind_pu", "pv_pu"]
82
83     df = pd.merge(load_raw, renew_raw, on="time", how="inner")
84     if len(df) != 24:
85         raise ValueError(f"Expected 24 hourly rows for typical day, got {len(df)}")
86
87     df["hour"] = np.arange(24)
88     df["time"] = df["time"].map(normalize_hour_label)
89     df["regular_load_mw"] = df["load_pu"].astype(float) * params.load_peak_mw
90     df["wind_mw"] = df["wind_pu"].astype(float) * params.wind_capacity_mw
91     df["pv_mw"] = df["pv_pu"].astype(float) * params.pv_capacity_mw
92     df["renewable_mw"] = df["wind_mw"] + df["pv_mw"]
93     df["purchase_price_yuan_per_kwh"] = df["hour"].map(purchase_price)
94     df["sell_price_yuan_per_kwh"] = params.sell_price_yuan_per_kwh
95     return df
96
97
98 def load_wind_solar_scenarios(params: SystemParams | None = None, root: Path = PROJECT_ROOT) ->
99     ↪ pd.DataFrame:
100     params = params or SystemParams()
101     wind_raw = pd.read_excel(find_input_file("附件 3", root))
102     pv_raw = pd.read_excel(find_input_file("附件 4", root))
103
104     wind_raw = wind_raw.rename(columns={wind_raw.columns[0]: "time"})
105     pv_raw = pv_raw.rename(columns={pv_raw.columns[0]: "time"})
106     wind_raw["hour"] = np.arange(24)
107     pv_raw["hour"] = np.arange(24)
108
109     rows: list[dict[str, float | int | str]] = []
110     wind_cols = [c for c in wind_raw.columns if str(c).startswith("风电场景")]
111     pv_cols = [c for c in pv_raw.columns if str(c).startswith("光伏场景")]
112     for wi, w_col in enumerate(wind_cols, 1):
113         for pi, pv_col in enumerate(pv_cols, 1):
114             for h in range(24):
115                 rows.append(
116                     {
117                         "scenario": f"W{wi}_PV{pi}",
118                         "wind_scenario": wi,
119                         "pv_scenario": pi,
120                         "hour": h,
121                         "time": normalize_hour_label(wind_raw.loc[h, "time"]),
122                         "wind_pu": float(wind_raw.loc[h, w_col]),
123                         "pv_pu": float(pv_raw.loc[h, pv_col]),
124                         "wind_mw": float(wind_raw.loc[h, w_col]) * params.wind_capacity_mw,
125                         "pv_mw": float(pv_raw.loc[h, pv_col]) * params.pv_capacity_mw,

```

```

125         }
126     )
127     df = pd.DataFrame(rows)
128     df["renewable_mw"] = df["wind_mw"] + df["pv_mw"]
129     df["purchase_price_yuan_per_kwh"] = df["hour"].map(purchase_price)
130     df["sell_price_yuan_per_kwh"] = params.sell_price_yuan_per_kwh
131     return df
132

```

D.3 指标与成本核算模块

文件：code/model_common/metrics.py

说明：统一计算并网绿电直连指标、离网消纳指标、给定成本口径下的吨氨成本及合格性判定。

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4
5  import pandas as pd
6
7  from .data import SystemParams
8
9
10 @dataclass(frozen=True)
11 class IndicatorThresholds:
12     min_self_use_ratio: float = 0.60
13     min_green_power_ratio: float = 0.30
14     max_sale_ratio: float = 0.20
15
16
17 def add_process_load(
18     df: pd.DataFrame,
19     params: SystemParams,
20     daily_ammonia_t: float = 36.0,
21     load_factor_col: str | None = None,
22 ) -> pd.DataFrame:
23     out = df.copy()
24     if load_factor_col is None:
25         factor = 1.0
26         out["process_load_factor"] = factor
27     else:
28         out["process_load_factor"] = out[load_factor_col].astype(float)
29         out["alk_load_mw"] = params.alk_power_mw(daily_ammonia_t) * out["process_load_factor"]
30         out["pem_load_mw"] = params.pem_power_mw(daily_ammonia_t) * out["process_load_factor"]
31         out["ammonia_load_mw"] = params.ammonia_power_mw(daily_ammonia_t) * out["process_load_factor"]
32         out["process_load_mw"] = out["alk_load_mw"] + out["pem_load_mw"] + out["ammonia_load_mw"]
33         out["ammonia_output_t"] = params.ammonia_output_t_per_h(daily_ammonia_t) *
34             ↪ out["process_load_factor"]
35         out["total_load_mw"] = out["regular_load_mw"] + out["process_load_mw"]
36         return out
37
38 def add_grid_exchange(df: pd.DataFrame) -> pd.DataFrame:
39     out = df.copy()
40     net_deficit = out["total_load_mw"] - out["renewable_mw"]
41     out["grid_purchase_mw"] = net_deficit.clip(lower=0)
42     out["grid_sale_mw"] = (-net_deficit).clip(lower=0)
43     out["grid_exchange_mw"] = out["grid_purchase_mw"] - out["grid_sale_mw"]
44     return out
45
46
47 def mwh(df: pd.DataFrame, column: str) -> float:
48     return float(df[column].sum())
49
50
51 def q1_summary(df: pd.DataFrame, params: SystemParams, thresholds: IndicatorThresholds | None =
52     ↪ None) -> pd.DataFrame:
53     thresholds = thresholds or IndicatorThresholds()
54
55     total_load_mwh = mwh(df, "total_load_mw")
56     regular_load_mwh = mwh(df, "regular_load_mw")

```

```

56 process_load_mwh = mwh(df, "process_load_mwh")
57 wind_mwh = mwh(df, "wind_mw")
58 pv_mwh = mwh(df, "pv_mw")
59 renewable_mwh = mwh(df, "renewable_mw")
60 purchase_mwh = mwh(df, "grid_purchase_mw")
61 sale_mwh = mwh(df, "grid_sale_mw")
62 ammonia_t = mwh(df, "ammonia_output_t")
63
64 renewable_self_use_mwh = renewable_mwh - sale_mwh
65 load_side_self_use_mwh = total_load_mwh - purchase_mwh
66 statement_formula_self_use_mwh = total_load_mwh - sale_mwh - purchase_mwh
67
68 self_use_ratio = renewable_self_use_mwh / renewable_mwh
69 green_power_ratio = renewable_self_use_mwh / total_load_mwh
70 sale_ratio = sale_mwh / renewable_mwh
71 statement_formula_ratio = statement_formula_self_use_mwh / renewable_mwh
72
73 wind_cost = wind_mwh * 1000 * params.wind_lcoe_yuan_per_kwh
74 pv_cost = pv_mwh * 1000 * params.pv_lcoe_yuan_per_kwh
75 purchase_cost = float((df["grid_purchase_mw"] * 1000 * df["purchase_price_yuan_per_kwh"]).sum())
76 alk_om = float((df["alk_load_mw"] * 1000 * params.alk_om_yuan_per_kwh).sum())
77 pem_om = float((df["pem_load_mw"] * 1000 * params.pem_om_yuan_per_kwh).sum())
78 ammonia_om = float((df["ammonia_load_mw"] * 1000 * params.ammonia_om_yuan_per_kwh).sum())
79 sale_revenue = sale_mwh * 1000 * params.sell_price_yuan_per_kwh
80 gross_cost = wind_cost + pv_cost + purchase_cost + alk_om + pem_om + ammonia_om
81 net_cost = gross_cost - sale_revenue
82 cost_per_ton = net_cost / ammonia_t
83
84 rows = [
85     ("典型日总用电量", total_load_mwh, "MWh"),
86     ("其中: 常规电负荷用电量", regular_load_mwh, "MWh"),
87     ("其中: 制氢制氨工艺用电量", process_load_mwh, "MWh"),
88     ("风电发电量", wind_mwh, "MWh"),
89     ("光伏发电量", pv_mwh, "MWh"),
90     ("新能源发电量", renewable_mwh, "MWh"),
91     ("网购电量", purchase_mwh, "MWh"),
92     ("上网电量", sale_mwh, "MWh"),
93     ("新能源自发自用电量(物理/政策口径)", renewable_self_use_mwh, "MWh"),
94     ("新能源自发自用电量(由负荷侧校验)", load_side_self_use_mwh, "MWh"),
95     ("题面括号公式复核量: 总用电量-上网电量-网购电量", statement_formula_self_use_mwh, "MWh"),
96     ("新能源自发自用比例(物理/政策口径)", self_use_ratio, "ratio"),
97     ("总用电量绿电比例", green_power_ratio, "ratio"),
98     ("新能源上网电量比例", sale_ratio, "ratio"),
99     ("题面括号公式比例复核", statement_formula_ratio, "ratio"),
100     ("新能源自发自用比例是否达标", float(self_use_ratio >= thresholds.min_self_use_ratio),
101         → "bool"),
102     ("总用电量绿电比例是否达标", float(green_power_ratio >= thresholds.min_green_power_ratio),
103         → "bool"),
104     ("新能源上网电量比例是否达标", float(sale_ratio <= thresholds.max_sale_ratio), "bool"),
105     ("日产氨量", ammonia_t, "t"),
106     ("风电成本", wind_cost, "yuan"),
107     ("光伏成本", pv_cost, "yuan"),
108     ("购电成本", purchase_cost, "yuan"),
109     ("碱性电解槽运维成本", alk_om, "yuan"),
110     ("PEM 电解槽运维成本", pem_om, "yuan"),
111     ("合成氨装置运维成本", ammonia_om, "yuan"),
112     ("余电上网收益", sale_revenue, "yuan"),
113     ("日总成本(扣除售电收益后)", net_cost, "yuan"),
114     ("吨氨成本", cost_per_ton, "yuan/t"),
115 ]
116 return pd.DataFrame(rows, columns=["metric", "value", "unit"])
117
118 def get_metric(summary: pd.DataFrame, metric: str) -> float:
119     return float(summary.loc[summary["metric"] == metric, "value"].iloc[0])
120
121 def format_metric(value: float, unit: str) -> str:
122     if unit == "ratio":
123         return f"{value:.4%}"
124     if unit == "bool":
125         return "是" if value >= 0.5 else "否"
126     if unit in {"yuan", "yuan/t"}:
127         return f"{value:,.2f}"
128     return f"{value:,.4f}"
129

```

D.4 统一绘图模块

文件: code/model_common/plotting.py

说明: 封装论文图件的字号、配色、坐标轴、图例和导出格式, 保证各问图表风格一致。

```
1 from __future__ import annotations
2
3 from pathlib import Path
4 from typing import Iterable
5
6 import matplotlib.pyplot as plt
7 import numpy as np
8 import pandas as pd
9
10
11 CSEE_COLORS = {
12     "blue": "#1f4e79",
13     "red": "#a61c2a",
14     "green": "#2f6b4f",
15     "orange": "#c55a11",
16     "purple": "#5b4b8a",
17     "gray": "#666666",
18     "light_blue": "#9dc3e6",
19     "light_red": "#f4b6b2",
20 }
21
22
23 def set_csee_style() -> None:
24     plt.rcParams.update(
25         {
26             "font.sans-serif": ["SimSun", "Microsoft YaHei", "SimHei", "Arial Unicode MS", "DejaVu
↳ Sans"],
27             "font.serif": ["Times New Roman", "SimSun", "DejaVu Serif"],
28             "axes.unicode_minus": False,
29             "figure.dpi": 160,
30             "savefig.dpi": 600,
31             "axes.linewidth": 0.8,
32             "axes.labelsize": 9,
33             "axes.titlesize": 10,
34             "xtick.labelsize": 8,
35             "ytick.labelsize": 8,
36             "legend.fontsize": 8,
37             "lines.linewidth": 1.6,
38             "lines.markersize": 4.2,
39             "grid.linewidth": 0.45,
40             "pdf.fonttype": 42,
41             "ps.fonttype": 42,
42             "svg.fonttype": "none",
43         }
44     )
45
46
47 def save_figure(fig: plt.Figure, output_base: Path, formats: Iterable[str] = ("png", "pdf", "svg"))
↳ -> None:
48     output_base.parent.mkdir(parents=True, exist_ok=True)
49     for fmt in formats:
50         kwargs = {"bbox_inches": "tight"}
51         if fmt == "png":
52             kwargs["dpi"] = 600
53         fig.savefig(output_base.with_suffix(f".{fmt}"), **kwargs)
54
55
56 def legend_below(ax: plt.Axes, ncol: int, y: float = -0.22) -> None:
57     ax.legend(
58         frameon=False,
59         ncol=ncol,
60         loc="upper center",
61         bbox_to_anchor=(0.5, y),
62         borderaxespad=0,
63         handlelength=2.4,
64         columnspacing=1.2,
65     )
66
67
68 def figure_legend_below(fig: plt.Figure, handles: list, labels: list, ncol: int, y: float = 0.01) ->
↳ None:
69     fig.legend(
70         handles,
71         labels,
```

```

72         frameon=False,
73         ncol=ncol,
74         loc="lower center",
75         bbox_to_anchor=(0.5, y),
76         borderaxespad=0,
77         handlelength=2.4,
78         columnspacing=1.2,
79     )
80
81
82 def annotate_bars(ax: plt.Axes, bars, fmt: str = "{:.2f}", dy_frac: float = 0.015) -> None:
83     ymin, ymax = ax.get_ylim()
84     dy = (ymax - ymin) * dy_frac
85     for bar in bars:
86         value = bar.get_height()
87         if value >= 0:
88             ax.text(bar.get_x() + bar.get_width() / 2, value + dy, fmt.format(value), ha="center",
89                     ↪ va="bottom", fontsize=7.5)
89         else:
90             ax.text(bar.get_x() + bar.get_width() / 2, value - dy, fmt.format(value), ha="center",
91                     ↪ va="top", fontsize=7.5)
92
93 def plot_q1_power_balance(df: pd.DataFrame, output_dir: Path) -> None:
94     set_csee_style()
95     x = df["hour"]
96     fig, ax = plt.subplots(figsize=(7.4, 4.6))
97     ax.plot(x, df["total_load_mw"], color=CSEE_COLORS["blue"], marker="o", label=" 总用电负荷")
98     ax.plot(x, df["renewable_mw"], color=CSEE_COLORS["green"], marker="s", label=" 风光发电合计")
99     ax.bar(x - 0.18, df["grid_purchase_mw"], width=0.34, color=CSEE_COLORS["light_red"],
100           ↪ edgecolor=CSEE_COLORS["red"], linewidth=0.5, label="购电功率")
101     ax.bar(x + 0.18, df["grid_sale_mw"], width=0.34, color=CSEE_COLORS["light_blue"],
102           ↪ edgecolor=CSEE_COLORS["blue"], linewidth=0.5, label="上网功率")
103     ax.set_xlabel(" 时段/h")
104     ax.set_ylabel(" 功率/MW")
105     ax.set_title(" 典型日功率平衡与购售电功率")
106     ax.set_xticks(range(0, 24, 2))
107     ax.set_xlim(-0.6, 23.6)
108     ax.grid(True, axis="y", linestyle="--", alpha=0.45)
109     legend_below(ax, ncol=4, y=-0.18)
110     fig.subplots_adjust(left=0.10, right=0.98, top=0.88, bottom=0.24)
111     save_figure(fig, output_dir / "q1_fig1_power_balance_csee")
112     plt.close(fig)
113
114 def plot_q1_components(df: pd.DataFrame, output_dir: Path) -> None:
115     set_csee_style()
116     x = df["hour"]
117     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7.4, 6.2), sharex=True)
118     ax1.plot(x, df["regular_load_mw"], color=CSEE_COLORS["gray"], marker="o", label=" 常规电负荷")
119     ax1.plot(x, df["process_load_mw"], color=CSEE_COLORS["red"], marker="s", label=" 制氢制氨负荷")
120     ax1.plot(x, df["total_load_mw"], color=CSEE_COLORS["blue"], marker="^", label=" 总负荷")
121     ax1.set_ylabel(" 负荷功率/MW")
122     ax1.grid(True, axis="y", linestyle="--", alpha=0.45)
123     legend_below(ax1, ncol=3, y=-0.09)
124
125     ax2.stackplot(
126         x,
127         df["wind_mw"],
128         df["pv_mw"],
129         colors=[CSEE_COLORS["light_blue"], "#f7c97f"],
130         labels=[" 风电", " 光伏"],
131         alpha=0.85,
132     )
133     ax2.plot(x, df["renewable_mw"], color=CSEE_COLORS["green"], marker="o", label=" 风光合计")
134     ax2.set_xlabel(" 时段/h")
135     ax2.set_ylabel(" 发电功率/MW")
136     ax2.set_xticks(range(0, 24, 2))
137     ax2.set_xlim(-0.2, 23.2)
138     ax2.grid(True, axis="y", linestyle="--", alpha=0.45)
139     legend_below(ax2, ncol=3, y=-0.22)
140     fig.suptitle(" 典型日负荷与新能源出力分项曲线", y=0.98, fontsize=10)
141     fig.subplots_adjust(left=0.10, right=0.98, top=0.90, bottom=0.18, hspace=0.36)
142     save_figure(fig, output_dir / "q1_fig2_components_csee")
143     plt.close(fig)
144
145 def plot_q1_indicator_bars(summary: pd.DataFrame, output_dir: Path) -> None:
146     set_csee_style()
147     metrics = [
148         (" 新能源自发自用比例 (物理/政策口径)", 0.60, "≥60%"),

```

```

149         (" 总用电量绿电比例", 0.30, ">30%"),
150         (" 新能源上网电量比例", 0.20, "<20%"),
151     ]
152     values = [float(summary.loc[summary["metric"] == m, "value"].iloc[0]) for m, _, _ in metrics]
153     thresholds = [thr for _, thr, _ in metrics]
154     labels = [" 自发自用比例", " 绿电比例", " 上网比例"]
155
156     fig, ax = plt.subplots(figsize=(6.4, 4.0))
157     colors = [CSEE_COLORS["green"], CSEE_COLORS["blue"], CSEE_COLORS["red"]]
158     bars = ax.bar(labels, [v * 100 for v in values], color=colors, alpha=0.82, width=0.56)
159     for bar, value in zip(bars, values):
160         ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 1.5, f"{value:.2%}",
161                 ↪ ha="center", va="bottom", fontsize=8)
162     for i, (_, thr, text) in enumerate(metrics):
163         ax.hlines(thr * 100, i - 0.35, i + 0.35, colors="black", linestyle="--", linewidth=0.9)
164         ax.text(i + 0.38, thr * 100, text, va="center", fontsize=8)
165     ax.set_ylabel(" 指标值/%")
166     ax.set_ylim(0, max(82, max(values) * 125))
167     ax.set_title(" 绿电直连指标合格性判定")
168     ax.grid(True, axis="y", linestyle="--", alpha=0.4)
169     fig.subplots_adjust(left=0.12, right=0.96, top=0.88, bottom=0.15)
170     save_figure(fig, output_dir / "q1_fig3_indicator_check_csee")
171     plt.close(fig)
172
173 def plot_q1_cost_breakdown(summary: pd.DataFrame, output_dir: Path) -> None:
174     set_csee_style()
175     cost_metrics = [
176         " 风电成本",
177         " 光伏成本",
178         " 购电成本",
179         " 碱性电解槽运维成本",
180         " PEM 电解槽运维成本",
181         " 合成氨装置运维成本",
182         " 余电上网收益",
183     ]
184     data = summary[summary["metric"].isin(cost_metrics)].copy()
185     data["plot_value"] = data["value"]
186     data.loc[data["metric"] == " 余电上网收益", "plot_value"] *= -1
187     labels = [" 风电", " 光伏", " 购电", " ALK 运维", " PEM 运维", " 合成氨运维", " 售电收益"]
188     colors = [
189         CSEE_COLORS["light_blue"],
190         "#f7c97f",
191         CSEE_COLORS["light_red"],
192         CSEE_COLORS["purple"],
193         CSEE_COLORS["blue"],
194         CSEE_COLORS["gray"],
195         CSEE_COLORS["green"],
196     ]
197
198     fig, ax = plt.subplots(figsize=(7.4, 4.4))
199     bars = ax.bar(labels, data["plot_value"] / 10000, color=colors, edgecolor="black",
200                 ↪ linewidth=0.45, width=0.62)
201     ax.axhline(0, color="black", linewidth=0.8)
202     for bar, value in zip(bars, data["plot_value"] / 10000):
203         va = "bottom" if value >= 0 else "top"
204         offset = 0.25 if value >= 0 else -0.25
205         ax.text(bar.get_x() + bar.get_width() / 2, value + offset, f"{value:.2f}", ha="center",
206                 ↪ va=va, fontsize=8)
207     ax.set_ylabel(" 金额/万元")
208     ax.set_title(" 典型日成本与收益分项")
209     ax.grid(True, axis="y", linestyle="--", alpha=0.4)
210     fig.subplots_adjust(left=0.10, right=0.98, top=0.88, bottom=0.16)
211     save_figure(fig, output_dir / "q1_fig4_cost_breakdown_csee")
212     plt.close(fig)
213
214 def plot_q1_net_exchange(df: pd.DataFrame, output_dir: Path) -> None:
215     set_csee_style()
216     x = df["hour"]
217     net_surplus = df["renewable_mw"] - df["total_load_mw"]
218     fig, ax = plt.subplots(figsize=(7.4, 4.2))
219     ax.axhline(0, color="black", linewidth=0.8)
220     ax.bar(
221         x,
222         net_surplus.clip(lower=0),
223         color=CSEE_COLORS["light_blue"],
224         edgecolor=CSEE_COLORS["blue"],
225         linewidth=0.5,
226         label=" 风光富余",
227     )

```

```

227 ax.bar(
228     x,
229     net_surplus.clip(upper=0),
230     color=CSEE_COLORS["light_red"],
231     edgecolor=CSEE_COLORS["red"],
232     linewidth=0.5,
233     label=" 用电缺口",
234 )
235 ax.plot(x, net_surplus, color=CSEE_COLORS["gray"], marker="o", linewidth=1.2, label=" 源荷差额")
236 ax.set_xlabel(" 时段/h")
237 ax.set_ylabel(" 源荷差额/MW")
238 ax.set_title(" 典型日源荷差额与购售电方向")
239 ax.set_xticks(range(0, 24, 2))
240 ax.set_xlim(-0.6, 23.6)
241 ax.grid(True, axis="y", linestyle="--", alpha=0.4)
242 legend_below(ax, ncol=3, y=-0.22)
243 fig.subplots_adjust(left=0.11, right=0.98, top=0.88, bottom=0.24)
244 save_figure(fig, output_dir / "q1_fig5_net_exchange_csee")
245 plt.close(fig)
246
247
248 def plot_q3_typical_load_factor(typical_hourly: pd.DataFrame, output_dir: Path) -> None:
249     set_csee_style()
250     renewable = typical_hourly.drop_duplicates("hour").sort_values("hour")
251     heat = typical_hourly.pivot_table(index="production_t_per_day", columns="hour",
252     ↪ values="production_load_factor", aggfunc="first")
253     heat = heat.sort_index(ascending=False) * 100
254
255     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7.6, 5.8), sharex=True,
256     ↪ gridspec_kw={"height_ratios": [1.05, 1.3]})
257     ax1.stackplot(
258         renewable["hour"],
259         renewable["wind_mw"],
260         renewable["pv_mw"],
261         colors=[CSEE_COLORS["light_blue"], "#f7c97f"],
262         labels=[" 风电", " 光伏"],
263         alpha=0.82,
264     )
265     ax1.plot(
266         renewable["hour"],
267         renewable["renewable_mw"],
268         color=CSEE_COLORS["green"],
269         marker="o",
270         linewidth=1.2,
271         label=" 风光合计",
272     )
273     ax1.set_ylabel(" 风光出力/MW")
274     ax1.grid(True, axis="y", linestyle="--", alpha=0.4)
275     legend_below(ax1, ncol=3, y=-0.08)
276
277     im = ax2.imshow(heat.values, aspect="auto", cmap="YlGnBu", vmin=0, vmax=100)
278     ax2.set_yticks(np.arange(len(heat.index)))
279     ax2.set_yticklabels([f"{int(v)}" for v in heat.index])
280     ax2.set_xticks(range(0, 24, 2))
281     ax2.set_xlabel(" 时段/h")
282     ax2.set_ylabel(" 日产量/t")
283     ax2.set_title(" 连续制氨负荷率/%", fontsize=9, pad=4)
284     for i in range(heat.shape[0]):
285         for j in range(heat.shape[1]):
286             value = heat.values[i, j]
287             if value <= 15 or value >= 95:
288                 text_color = "white" if value >= 60 else "#222222"
289                 ax2.text(j, i, f"{value:.0f}", ha="center", va="center", fontsize=6.5,
290                 ↪ color=text_color)
291     cbar = fig.colorbar(im, ax=ax2, orientation="vertical", fraction=0.024, pad=0.014)
292     cbar.set_label(" 负荷率/%")
293     fig.suptitle(" 典型风光场景下风光出力与连续制氨负荷率", y=0.985, fontsize=10)
294     fig.subplots_adjust(left=0.10, right=0.94, top=0.90, bottom=0.12, hspace=0.38)
295     save_figure(fig, output_dir / "q3_fig1_typical_load_factor_csee")
296     plt.close(fig)
297
298 def plot_q3_typical_load_factor_lines(typical_hourly: pd.DataFrame, output_dir: Path) -> None:
299     set_csee_style()
300     fig, ax1 = plt.subplots(figsize=(7.6, 4.8))
301     for production, group in typical_hourly.groupby("production_t_per_day", sort=False):
302         group = group.sort_values("hour")
303         ax1.plot(group["hour"], group["production_load_factor"] * 100, marker="o", linewidth=1.4,
304         ↪ label=f"{int(production)} t/d")
305     ax1.set_xlabel(" 时段/h")
306     ax1.set_ylabel(" 连续负荷率/%")

```

```

304 ax1.set_xticks(range(0, 24, 2))
305 ax1.set_xlim(-0.4, 23.4)
306 ax1.set_ylim(0, 105)
307 ax1.grid(True, axis="y", linestyle="--", alpha=0.4)
308
309 renewable = typical_hourly.drop_duplicates("hour").sort_values("hour")
310 ax2 = ax1.twinx()
311 ax2.fill_between(renewable["hour"], renewable["renewable_mw"], color=CSEE_COLORS["light_blue"],
312 ↪ alpha=0.25, label="风光合计")
313 ax2.plot(renewable["hour"], renewable["renewable_mw"], color=CSEE_COLORS["green"],
314 ↪ linewidth=1.2)
315 ax2.set_ylabel(" 风光出力/MW")
316
317 handles1, labels1 = ax1.get_legend_handles_labels()
318 handles2, labels2 = ax2.get_legend_handles_labels()
319 figure_legend_below(fig, handles1 + handles2[:1], labels1 + labels2[:1], ncol=3, y=0.01)
320 ax1.set_title(" 典型风光场景下连续制氨负荷率曲线")
321 fig.subplots_adjust(left=0.10, right=0.90, top=0.88, bottom=0.24)
322 save_figure(fig, output_dir / "q3_fig1b_typical_load_factor_lines_csee")
323 plt.close(fig)
324
325 def plot_q3_q2_cost_compare(compare: pd.DataFrame, output_dir: Path) -> None:
326     set_csee_style()
327     df = compare.sort_values("production_t_per_day")
328     x = np.arange(len(df))
329     width = 0.34
330     fig, ax = plt.subplots(figsize=(7.4, 4.5))
331     bars1 = ax.bar(
332         x - width / 2,
333         df["q2_annual_average_ton_cost_yuan_per_t"],
334         width=width,
335         color=CSEE_COLORS["light_blue"],
336         edgecolor=CSEE_COLORS["blue"],
337         linewidth=0.6,
338         label=" 问题二: 离散开停",
339     )
340     bars2 = ax.bar(
341         x + width / 2,
342         df["q3_annual_average_ton_cost_yuan_per_t"],
343         width=width,
344         color="#f7c97f",
345         edgecolor=CSEE_COLORS["orange"],
346         linewidth=0.6,
347         label=" 问题三: 连续调节",
348     )
349     ax.set_xticks(x)
350     ax.set_xticklabels([f"{int(v)}" for v in df["production_t_per_day"]])
351     ax.set_xlabel(" 日产量/t")
352     ax.set_ylabel(" 年均吨氨成本/(元/t)")
353     ax.set_title(" 离散开停与连续调节的年均吨氨成本对比")
354     ax.set_ylim(0, max(df["q2_annual_average_ton_cost_yuan_per_t"].max(),
355 ↪ df["q3_annual_average_ton_cost_yuan_per_t"].max()) * 1.16)
356     ax.grid(True, axis="y", linestyle="--", alpha=0.4)
357     annotate_bars(ax, bars1, fmt="{:.0f}", dy_frac=0.01)
358     annotate_bars(ax, bars2, fmt="{:.0f}", dy_frac=0.01)
359     legend_below(ax, ncol=2, y=-0.20)
360     fig.subplots_adjust(left=0.11, right=0.98, top=0.88, bottom=0.23)
361     save_figure(fig, output_dir / "q3_fig2_compare_q2_cost_csee")
362     plt.close(fig)
363
364 def plot_q3_grid_indicator_compare(compare: pd.DataFrame, output_dir: Path) -> None:
365     set_csee_style()
366     df = compare.sort_values("production_t_per_day")
367     x = np.arange(len(df))
368     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7.6, 6.0), sharex=True)
369
370     width = 0.18
371     ax1.bar(x - 1.5 * width, df["q2_annual_purchase_mwh"], width=width,
372 ↪ color=CSEE_COLORS["light_red"], edgecolor=CSEE_COLORS["red"], linewidth=0.5,
373 ↪ label="离散购电")
374     ax1.bar(x - 0.5 * width, df["q3_annual_purchase_mwh"], width=width, color="#f4d2cf",
375 ↪ edgecolor=CSEE_COLORS["red"], linewidth=0.5, hatch="//", label="连续购电")
376     ax1.bar(x + 0.5 * width, df["q2_annual_sale_mwh"], width=width, color=CSEE_COLORS["light_blue"],
377 ↪ edgecolor=CSEE_COLORS["blue"], linewidth=0.5, label="离散上网")
378     ax1.bar(x + 1.5 * width, df["q3_annual_sale_mwh"], width=width, color="#c9dfff2",
379 ↪ edgecolor=CSEE_COLORS["blue"], linewidth=0.5, hatch="\\\\", label="连续上网")
380     ax1.set_ylabel(" 年度电量/MWh")
381     ax1.grid(True, axis="y", linestyle="--", alpha=0.4)
382     legend_below(ax1, ncol=4, y=-0.10)

```

```

377 ax2.plot(x, df["q2_annual_self_use_ratio"] * 100, marker="o", color=CSEE_COLORS["green"],
378 ↪ linestyle="--", label="离散自发自用")
379 ax2.plot(x, df["q3_annual_self_use_ratio"] * 100, marker="o", color=CSEE_COLORS["green"],
380 ↪ label="连续自发自用")
381 ax2.plot(x, df["q2_annual_sale_ratio"] * 100, marker="^", color=CSEE_COLORS["red"],
382 ↪ linestyle="--", label="离散上网比例")
383 ax2.plot(x, df["q3_annual_sale_ratio"] * 100, marker="^", color=CSEE_COLORS["red"],
384 ↪ label="连续上网比例")
385 ax2.axhline(60, color=CSEE_COLORS["green"], linestyle=":", linewidth=0.8)
386 ax2.axhline(20, color=CSEE_COLORS["red"], linestyle=":", linewidth=0.8)
387 ax2.set_xticks(x)
388 ax2.set_xticklabels([f"{int(v)}" for v in df["production_t_per_day"]])
389 ax2.set_xlabel(" 日产量/t")
390 ax2.set_ylabel(" 指标值/%")
391 ax2.grid(True, axis="y", linestyle="--", alpha=0.4)
392 legend_below(ax2, ncol=2, y=-0.24)
393
394 fig.suptitle(" 离散开停与连续调节的购售电和绿电指标对比", y=0.985, fontsize=10)
395 fig.subplots_adjust(left=0.10, right=0.98, top=0.90, bottom=0.20, hspace=0.34)
396 save_figure(fig, output_dir / "q3_fig3_compare_q2_grid_indicator_csee")
397 plt.close(fig)
398
399 def plot_q3_scenario_cost_box(scenario_summary: pd.DataFrame, output_dir: Path, production_levels:
400 ↪ Iterable[int]) -> None:
401     set_csee_style()
402     ordered = sorted(production_levels)
403     data = [scenario_summary.loc[scenario_summary["production_t_per_day"] == d, "吨氨成本"] for d in
404 ↪ ordered]
405     fig, ax = plt.subplots(figsize=(7.4, 4.4))
406     bp = ax.boxplot(data, tick_labels=[str(d) for d in ordered], patch_artist=True, widths=0.58)
407     for patch in bp["boxes"]:
408         patch.set_facecolor("#f7c97f")
409         patch.set_edgecolor(CSEE_COLORS["orange"])
410     ax.set_xlabel(" 日产量/t")
411     ax.set_ylabel(" 吨氨成本/(元/t)")
412     ax.set_title(" 连续调节下 24 种风光场景吨氨成本分布")
413     ax.grid(True, axis="y", linestyle="--", alpha=0.4)
414     fig.subplots_adjust(left=0.11, right=0.98, top=0.88, bottom=0.15)
415     save_figure(fig, output_dir / "q3_fig4_scenario_cost_box_csee")
416     plt.close(fig)
417
418 def plot_q3_cost_duration_curve(
419     scenario_summary: pd.DataFrame,
420     output_dir: Path,
421     production_levels: Iterable[int],
422     scenario_days: int = 15,
423 ) -> None:
424     set_csee_style()
425     fig, ax = plt.subplots(figsize=(7.4, 4.5))
426     for production in sorted(production_levels, reverse=True):
427         vals = np.sort(scenario_summary.loc[scenario_summary["production_t_per_day"] == production,
428 ↪ "吨氨成本"].to_numpy())
429         x = np.arange(1, len(vals) + 1) * scenario_days
430         ax.step(x, vals, where="post", linewidth=1.4, label=f"{production} t/d")
431     ax.set_xlabel(" 累计代表天数/d")
432     ax.set_ylabel(" 吨氨成本/(元/t)")
433     ax.set_title(" 连续调节下全年代表场景吨氨成本分布曲线")
434     ax.set_xlim(0, max(360, len(vals) * scenario_days))
435     ax.grid(True, linestyle="--", alpha=0.4)
436     legend_below(ax, ncol=5, y=-0.20)
437     fig.subplots_adjust(left=0.11, right=0.98, top=0.88, bottom=0.24)
438     save_figure(fig, output_dir / "q3_fig7_annual_cost_duration_csee")
439     plt.close(fig)
440
441 def plot_q3_cost_delta_heatmaps(q2_summary: pd.DataFrame, q3_summary: pd.DataFrame, output_dir:
442 ↪ Path, production_levels: Iterable[int]) -> None:
443     set_csee_style()
444     key = ["wind_scenario", "pv_scenario", "production_t_per_day"]
445     merged = q2_summary[key + [" 吨氨成本"]].merge(
446         q3_summary[key + [" 吨氨成本"]],
447         on=key,
448         suffixes=("_q2", "_q3"),
449     )
450     merged["cost_delta"] = merged[" 吨氨成本_q3"] - merged[" 吨氨成本_q2"]
451     vmax = float(np.ceil(max(abs(merged["cost_delta"].min()), abs(merged["cost_delta"].max())) / 50)
452 ↪ * 50)

```

```

448 levels = sorted(production_levels, reverse=True)
449 fig, axes = plt.subplots(2, 3, figsize=(8.8, 5.4), sharex=True, sharey=True)
450 axes_flat = axes.ravel()
451 image = None
452 for ax, production in zip(axes_flat, levels):
453     matrix = (
454         merged.loc[merged["production_t_per_day"] == production]
455         .pivot_table(index="wind_scenario", columns="pv_scenario", values="cost_delta",
456           ↪ aggfunc="first")
457         .sort_index()
458     )
459     image = ax.imshow(matrix.values, cmap="RdBu_r", vmin=-vmax, vmax=vmax, aspect="auto")
460     ax.set_title(f"{production} t/d", fontsize=9)
461     ax.set_xticks(np.arange(matrix.shape[1]))
462     ax.set_xticklabels([str(int(c)) for c in matrix.columns])
463     ax.set_yticks(np.arange(matrix.shape[0]))
464     ax.set_yticklabels([str(int(i)) for i in matrix.index])
465     for i in range(matrix.shape[0]):
466         for j in range(matrix.shape[1]):
467             value = matrix.values[i, j]
468             text_color = "white" if abs(value) > 0.55 * vmax else "#222222"
469             ax.text(j, i, f"{value:.0f}", ha="center", va="center", fontsize=6.5,
470               ↪ color=text_color)
471 axes_flat[-1].axis("off")
472 for ax in axes[:, 0]:
473     ax.set_ylabel(" 风电场景")
474 for ax in axes[-1, :2]:
475     ax.set_xlabel(" 光伏场景")
476 if image is not None:
477     cbar = fig.colorbar(image, ax=axes_flat[-1], orientation="vertical", fraction=0.026,
478       ↪ pad=0.018)
479     cbar.set_label("Q3-Q2 成本变化/(元/t)")
480 fig.suptitle(" 连续调节相对离散开停的场景吨氨成本变化", y=0.985, fontsize=10)
481 fig.subplots_adjust(left=0.08, right=0.90, top=0.90, bottom=0.10, wspace=0.14, hspace=0.26)
482 save_figure(fig, output_dir / "q3_fig8_cost_delta_heatmap_csee")
483 plt.close(fig)
484
485 def plot_q3_satisfaction_stacked(annual: pd.DataFrame, output_dir: Path) -> None:
486     set_csee_style()
487     df = annual.sort_values("production_t_per_day")
488     x = np.arange(len(df))
489     fig, ax = plt.subplots(figsize=(7.4, 4.4))
490     ax.bar(x, df["full_satisfied_days"], color=CSEE_COLORS["green"], label=" 全满足")
491     ax.bar(x, df["partial_satisfied_days"], bottom=df["full_satisfied_days"], color="#f7c97f",
492       ↪ label="部分满足")
493     bottom = df["full_satisfied_days"] + df["partial_satisfied_days"]
494     ax.bar(x, df["none_satisfied_days"], bottom=bottom, color=CSEE_COLORS["light_red"],
495       ↪ label="全不满足")
496     ax.set_xticks(x)
497     ax.set_xticklabels([f"{int(v)}" for v in df["production_t_per_day"]])
498     ax.set_xlabel(" 日产量/t")
499     ax.set_ylabel(" 年度代表天数/d")
500     ax.set_title(" 连续调节下年度绿电指标合格类型统计")
501     legend_below(ax, ncol=3, y=-0.20)
502     ax.grid(True, axis="y", linestyle="--", alpha=0.4)
503     fig.subplots_adjust(left=0.10, right=0.98, top=0.88, bottom=0.24)
504     save_figure(fig, output_dir / "q3_fig5_satisfaction_stacked_csee")
505     plt.close(fig)
506
507 def plot_q3_typical_power_balance(selected_hourly: pd.DataFrame, output_dir: Path,
508   ↪ production_t_per_day: float) -> None:
509     set_csee_style()
510     df = selected_hourly.sort_values("hour")
511     x = df["hour"]
512     fig, ax1 = plt.subplots(figsize=(7.6, 4.8))
513     ax1.stackplot(
514         x,
515         df["wind_mw"],
516         df["pv_mw"],
517         colors=[CSEE_COLORS["light_blue"], "#f7c97f"],
518         labels=[" 风电", " 光伏"],
519         alpha=0.82,
520     )
521     ax1.plot(x, df["total_load_mw"], color=CSEE_COLORS["red"], marker="o", label=" 总负荷")
522     ax1.bar(x - 0.18, df["grid_purchase_mw"], width=0.34, color=CSEE_COLORS["light_red"],
523       ↪ edgecolor=CSEE_COLORS["red"], linewidth=0.5, label="购电")
524     ax1.bar(x + 0.18, df["grid_sale_mw"], width=0.34, color=CSEE_COLORS["light_blue"],
525       ↪ edgecolor=CSEE_COLORS["blue"], linewidth=0.5, label="上网")

```

```

520 ax1.set_xlabel(" 时段/h")
521 ax1.set_ylabel(" 功率/MW")
522 ax1.set_xticks(range(0, 24, 2))
523 ax1.set_xlim(-0.5, 23.5)
524 ax1.grid(True, axis="y", linestyle="--", alpha=0.4)
525
526 ax2 = ax1.twinx()
527 ax2.plot(x, df["production_load_factor"] * 100, color=CSEE_COLORS["purple"], marker="s",
↪ linewidth=1.2, label="负荷率")
528 ax2.set_ylabel(" 制氮负荷率/%")
529 ax2.set_ylim(0, 105)
530
531 handles1, labels1 = ax1.get_legend_handles_labels()
532 handles2, labels2 = ax2.get_legend_handles_labels()
533 ax1.legend(
534     handles1 + handles2,
535     labels1 + labels2,
536     frameon=False,
537     ncol=5,
538     loc="upper center",
539     bbox_to_anchor=(0.5, -0.18),
540     borderaxespad=0,
541     columnspacing=1.1,
542 )
543 ax1.set_title(f" 典型场景连续调节功率平衡 ({production_t_per_day:.0f} t/d) ")
544 fig.subplots_adjust(left=0.10, right=0.90, top=0.88, bottom=0.24)
545 save_figure(fig, output_dir / "q3_fig6_typical_power_balance_csee")
546 plt.close(fig)

```

E 附录 E：问题一至问题五计算程序

本附录按五个问题的递进顺序列出各计算程序。各程序均从附件和公共模块读取数据，完成指标核算、优化求解、结果统计和图表绘制。问题一为基准核算，问题二为离散开停机优化，问题三为连续负荷率 MILP，问题四为离网储能和并网同产量对比，问题五为模型证据与政策矩阵整理。

E.1 问题一：典型风光场景指标核算

文件：code/questions/solve_q1.py

说明：计算刚性满负荷基准下的小时级功率平衡、三项绿电指标、购售电量和吨氨成本，并输出典型日结果图表。

```
1 from __future__ import annotations
2
3 import sys
4 from pathlib import Path
5
6 import pandas as pd
7
8 PROJECT_ROOT = Path(__file__).resolve().parents[1]
9 if str(PROJECT_ROOT) not in sys.path:
10     sys.path.insert(0, str(PROJECT_ROOT))
11
12 from model_common.data import SystemParams, load_typical_day
13 from model_common.metrics import add_grid_exchange, add_process_load, format_metric, get_metric,
14     ↪ q1_summary
15 from model_common.plotting import (
16     plot_q1_components,
17     plot_q1_cost_breakdown,
18     plot_q1_indicator_bars,
19     plot_q1_net_exchange,
20     plot_q1_power_balance,
21 )
22
23 OUT_DIR = Path(__file__).resolve().parent / "outputs"
24
25
26 def build_q1_hourly_table(params: SystemParams) -> pd.DataFrame:
27     hourly = load_typical_day(params=params, root=PROJECT_ROOT)
28     hourly = add_process_load(hourly, params=params, daily_ammonia_t=36.0)
29     hourly = add_grid_exchange(hourly)
30     return hourly
31
32
33 def write_result_summary(summary: pd.DataFrame, output_path: Path) -> None:
34     lines = [
35         "# 问题一-计算结果摘要",
36         "",
37         "| 指标 | 数值 | 单位 |",
38         "| ---|---:|---|",
39     ]
40     for _, row in summary.iterrows():
41         lines.append(f"| {row['metric']} | {format_metric(float(row['value'])), row['unit']} |"
42             ↪ f" {row['unit']} |")
43
44     lines.extend(
45         [
46             "",
47             "## 合格性判定（物理/政策口径）",
48             "",
49             f"- 新能源自发自用比例: {get_metric(summary, '新能源自发自用比例（物理/政策口径）'):.2%},
50             ↪ 要求不低于60%。",
51             f"- 总用电量绿电比例: {get_metric(summary, '总用电量绿电比例'):.2%}, 要求不低于 30%。",
52             f"- 新能源上网电量比例: {get_metric(summary, '新能源上网电量比例'):.2%}, 要求不高于 20%。",
53             "",
54             "说明：本结果将新能源自发自用电量定义为新能源发电量中未上网、用于园区负荷的电量；
55             ↪ 同时输出题面括号公式复核量，便于论文中说明口径差异。",
56         ]
57     )
58 )
```

```

55     output_path.write_text("\n".join(lines), encoding="utf-8")
56
57
58 def main() -> None:
59     OUT_DIR.mkdir(parents=True, exist_ok=True)
60     params = SystemParams()
61     hourly = build_q1_hourly_table(params)
62     summary = q1_summary(hourly, params)
63
64     hourly_columns = [
65         "hour",
66         "time",
67         "load_pu",
68         "wind_pu",
69         "pv_pu",
70         "regular_load_mw",
71         "alk_load_mw",
72         "pem_load_mw",
73         "ammonia_load_mw",
74         "process_load_mw",
75         "total_load_mw",
76         "wind_mw",
77         "pv_mw",
78         "renewable_mw",
79         "grid_purchase_mw",
80         "grid_sale_mw",
81         "grid_exchange_mw",
82         "purchase_price_yuan_per_kwh",
83         "sell_price_yuan_per_kwh",
84     ]
85     hourly.loc[:, hourly_columns].to_csv(OUT_DIR / "q1_hourly_results.csv", index=False,
86     ↪ encoding="utf-8-sig")
87     summary.to_csv(OUT_DIR / "q1_summary.csv", index=False, encoding="utf-8-sig")
88     write_result_summary(summary, OUT_DIR / "q1_result_summary.md")
89
90     plot_q1_power_balance(hourly, OUT_DIR)
91     plot_q1_components(hourly, OUT_DIR)
92     plot_q1_indicator_bars(summary, OUT_DIR)
93     plot_q1_cost_breakdown(summary, OUT_DIR)
94     plot_q1_net_exchange(hourly, OUT_DIR)
95
96     print(summary.to_string(index=False))
97     print(f"\nOutputs written to: {OUT_DIR}")
98
99 if __name__ == "__main__":
100     main()

```

E.2 问题二：离散开停机优化

文件：code/questions/solve_q2.py

说明：在扩容至 72 t/d 后，采用边际成本排序法求解不同日产量下的最优开机时段，并完成 24 场景年度加权统计。

```

1  from __future__ import annotations
2
3  import sys
4  from pathlib import Path
5
6  import matplotlib.pyplot as plt
7  import numpy as np
8  import pandas as pd
9
10 PROJECT_ROOT = Path(__file__).resolve().parents[1]
11 if str(PROJECT_ROOT) not in sys.path:
12     sys.path.insert(0, str(PROJECT_ROOT))
13
14 from model_common.data import SystemParams, load_typical_day, load_wind_solar_scenarios
15 from model_common.metrics import get_metric, q1_summary
16 from model_common.plotting import CSEE_COLORS, annotate_bars, legend_below, save_figure,
17 ↪ set_csee_style

```

```

18
19 OUT_DIR = Path(__file__).resolve().parent / "outputs"
20 PRODUCTION_LEVELS_T = [72, 63, 54, 45, 36]
21 SCENARIO_DAYS = 15
22 CAPACITY_DAILY_AMMONIA_T = 72.0
23
24
25 def add_regular_load_to_scenarios(params: SystemParams) -> pd.DataFrame:
26     scenario_df = load_wind_solar_scenarios(params=params, root=PROJECT_ROOT)
27     regular = load_typical_day(params=params, root=PROJECT_ROOT)[["hour", "regular_load_mw"]]
28     return scenario_df.merge(regular, on="hour", how="left")
29
30
31 def net_grid_cost_yuan(load_mw: pd.Series, renewable_mw: pd.Series, buy_price: pd.Series,
    ↪ sell_price: float) -> pd.Series:
32     deficit = (load_mw - renewable_mw).clip(lower=0)
33     surplus = (renewable_mw - load_mw).clip(lower=0)
34     return deficit * 1000 * buy_price - surplus * 1000 * sell_price
35
36
37 def optimize_discrete_schedule(
38     base_df: pd.DataFrame,
39     params: SystemParams,
40     production_t_per_day: float,
41     case_name: str,
42     wind_scenario: int | None = None,
43     pv_scenario: int | None = None,
44 ) -> tuple[pd.DataFrame, dict[str, float | int | str]]:
45     hourly = base_df.copy().sort_values("hour").reset_index(drop=True)
46     full_output_t_per_h = params.ammonia_output_t_per_h(CAPACITY_DAILY_AMMONIA_T)
47     required_hours = production_t_per_day / full_output_t_per_h
48     if abs(required_hours - round(required_hours)) > 1e-9:
49         raise ValueError(f"Production {production_t_per_day} t/d cannot be represented by full-load
    ↪ hours.")
50     required_hours = int(round(required_hours))
51
52     full_alk_mw = params.alk_power_mw(CAPACITY_DAILY_AMMONIA_T)
53     full_pem_mw = params.pem_power_mw(CAPACITY_DAILY_AMMONIA_T)
54     full_ammonia_mw = params.ammonia_power_mw(CAPACITY_DAILY_AMMONIA_T)
55     full_process_mw = full_alk_mw + full_pem_mw + full_ammonia_mw
56     process_om_yuan_per_h = (
57         full_alk_mw * 1000 * params.alk_om_yuan_per_kwh
58         + full_pem_mw * 1000 * params.pem_om_yuan_per_kwh
59         + full_ammonia_mw * 1000 * params.ammonia_om_yuan_per_kwh
60     )
61
62     off_load = hourly["regular_load_mw"]
63     on_load = hourly["regular_load_mw"] + full_process_mw
64     renewable = hourly["renewable_mw"]
65     buy_price = hourly["purchase_price_yuan_per_kwh"]
66
67     off_grid_cost = net_grid_cost_yuan(off_load, renewable, buy_price,
    ↪ params.sell_price_yuan_per_kwh)
68     on_grid_cost = net_grid_cost_yuan(on_load, renewable, buy_price, params.sell_price_yuan_per_kwh)
69     marginal_on_cost = on_grid_cost - off_grid_cost + process_om_yuan_per_h
70
71     order = np.lexsort((hourly["hour"].to_numpy(), marginal_on_cost.to_numpy()))
72     on = np.zeros(len(hourly), dtype=int)
73     on[order[:required_hours]] = 1
74
75     hourly["production_on"] = on
76     hourly["marginal_on_cost_yuan"] = marginal_on_cost
77     hourly["alk_load_mw"] = full_alk_mw * hourly["production_on"]
78     hourly["pem_load_mw"] = full_pem_mw * hourly["production_on"]
79     hourly["ammonia_load_mw"] = full_ammonia_mw * hourly["production_on"]
80     hourly["process_load_mw"] = full_process_mw * hourly["production_on"]
81     hourly["ammonia_output_t"] = full_output_t_per_h * hourly["production_on"]
82     hourly["total_load_mw"] = hourly["regular_load_mw"] + hourly["process_load_mw"]
83
84     net_deficit = hourly["total_load_mw"] - hourly["renewable_mw"]
85     hourly["grid_purchase_mw"] = net_deficit.clip(lower=0)
86     hourly["grid_sale_mw"] = (-net_deficit).clip(lower=0)
87     hourly["grid_exchange_mw"] = hourly["grid_purchase_mw"] - hourly["grid_sale_mw"]
88     hourly["case_name"] = case_name
89     hourly["production_t_per_day"] = production_t_per_day
90     hourly["required_on_hours"] = required_hours
91     hourly["equipment_utilization"] = production_t_per_day / CAPACITY_DAILY_AMMONIA_T
92     hourly["wind_scenario"] = -1 if wind_scenario is None else wind_scenario
93     hourly["pv_scenario"] = -1 if pv_scenario is None else pv_scenario
94
95     summary = q1_summary(hourly, params)
96     record = {

```

```

97         "case_name": case_name,
98         "wind_scenario": -1 if wind_scenario is None else wind_scenario,
99         "pv_scenario": -1 if pv_scenario is None else pv_scenario,
100         "production_t_per_day": production_t_per_day,
101         "required_on_hours": required_hours,
102         "equipment_utilization": production_t_per_day / CAPACITY_DAILY_AMMONIA_T,
103         "on_hours": ", ".join(str(int(h)) for h in hourly.loc[hourly["production_on"] == 1, "hour"]),
104     }
105     for _, row in summary.iterrows():
106         record[str(row["metric"])] = float(row["value"])
107     record["satisfied_count"] = int(
108         record["新能源自发自用比例是否达标"] + record["总用电量绿电比例是否达标"] +
        ↪ record["新能源上网电量比例是否达标"]
109     )
110     if record["satisfied_count"] == 3:
111         record["satisfaction_type"] = " 全满足"
112     elif record["satisfied_count"] == 0:
113         record["satisfaction_type"] = " 全不满足"
114     else:
115         record["satisfaction_type"] = " 部分满足"
116     return hourly, record
117
118
119 def run_typical_cases(params: SystemParams) -> tuple[pd.DataFrame, pd.DataFrame]:
120     typical = load_typical_day(params=params, root=PROJECT_ROOT)
121     hourly_parts = []
122     records = []
123     for production in PRODUCTION_LEVELS_T:
124         hourly, record = optimize_discrete_schedule(typical, params, production, "typical")
125         hourly_parts.append(hourly)
126         records.append(record)
127     return pd.concat(hourly_parts, ignore_index=True), pd.DataFrame(records)
128
129
130 def run_scenario_cases(params: SystemParams) -> tuple[pd.DataFrame, pd.DataFrame]:
131     scenarios = add_regular_load_to_scenarios(params)
132     hourly_parts = []
133     records = []
134     for (scenario, wind_id, pv_id), group in scenarios.groupby(["scenario", "wind_scenario",
        ↪ "pv_scenario"], sort=True):
135         for production in PRODUCTION_LEVELS_T:
136             hourly, record = optimize_discrete_schedule(
137                 group,
138                 params,
139                 production,
140                 str(scenario),
141                 wind_scenario=int(wind_id),
142                 pv_scenario=int(pv_id),
143             )
144             hourly_parts.append(hourly)
145             records.append(record)
146     return pd.concat(hourly_parts, ignore_index=True), pd.DataFrame(records)
147
148
149 def build_annual_summary(scenario_summary: pd.DataFrame) -> pd.DataFrame:
150     rows = []
151     for production, group in scenario_summary.groupby("production_t_per_day", sort=False):
152         annual_total_cost = float((group[" 日总成本 (扣除售电收益后)"] * SCENARIO_DAYS).sum())
153         annual_ammonia = float((group[" 日产氨量"] * SCENARIO_DAYS).sum())
154         annual_purchase = float((group[" 网购电量"] * SCENARIO_DAYS).sum())
155         annual_sale = float((group[" 上网电量"] * SCENARIO_DAYS).sum())
156         type_counts = group["satisfaction_type"].value_counts().to_dict()
157         rows.append(
158             {
159                 "production_t_per_day": production,
160                 "annual_days": int(len(group) * SCENARIO_DAYS),
161                 "annual_ammonia_t": annual_ammonia,
162                 "annual_total_cost_yuan": annual_total_cost,
163                 "annual_average_ton_cost_yuan_per_t": annual_total_cost / annual_ammonia,
164                 "annual_purchase_mwh": annual_purchase,
165                 "annual_sale_mwh": annual_sale,
166                 "full_satisfied_scenarios": int(type_counts.get(" 全满足", 0)),
167                 "partial_satisfied_scenarios": int(type_counts.get(" 部分满足", 0)),
168                 "none_satisfied_scenarios": int(type_counts.get(" 全不满足", 0)),
169                 "full_satisfied_days": int(type_counts.get(" 全满足", 0) * SCENARIO_DAYS),
170                 "partial_satisfied_days": int(type_counts.get(" 部分满足", 0) * SCENARIO_DAYS),
171                 "none_satisfied_days": int(type_counts.get(" 全不满足", 0) * SCENARIO_DAYS),
172             }
173         )
174     annual = pd.DataFrame(rows)
175     return annual.sort_values("production_t_per_day", ascending=False).reset_index(drop=True)

```

```

176
177
178 def build_distribution_summary(scenario_summary: pd.DataFrame) -> pd.DataFrame:
179     rows = []
180     for production, group in scenario_summary.groupby("production_t_per_day", sort=False):
181         rows.append(
182             {
183                 "production_t_per_day": production,
184                 "ton_cost_min": float(group["吨氨成本"].min()),
185                 "ton_cost_mean": float(group["吨氨成本"].mean()),
186                 "ton_cost_median": float(group["吨氨成本"].median()),
187                 "ton_cost_max": float(group["吨氨成本"].max()),
188                 "purchase_mwh_mean": float(group["网购电量"].mean()),
189                 "purchase_mwh_min": float(group["网购电量"].min()),
190                 "purchase_mwh_max": float(group["网购电量"].max()),
191                 "sale_mwh_mean": float(group["上网电量"].mean()),
192                 "sale_mwh_min": float(group["上网电量"].min()),
193                 "sale_mwh_max": float(group["上网电量"].max()),
194                 "self_use_ratio_mean": float(group["新能源自发自用比例（物理/政策口径）"].mean()),
195                 "green_power_ratio_mean": float(group["总用电量绿电比例"].mean()),
196                 "sale_ratio_mean": float(group["新能源上网电量比例"].mean()),
197             }
198         )
199     return pd.DataFrame(rows).sort_values("production_t_per_day",
200     ↪ ascending=False).reset_index(drop=True)
201
202 def write_markdown_summary(typical_summary: pd.DataFrame, scenario_summary: pd.DataFrame, annual:
203     ↪ pd.DataFrame) -> None:
204     best_typical = typical_summary.loc[typical_summary["吨氨成本"].idxmin()]
205     best_annual = annual.loc[annual["annual_average_ton_cost_yuan_per_t"].idxmin()]
206
207     lines = [
208         "# 问题二结果摘要",
209         "",
210         "## 典型风光场景",
211         "",
212         "| 日产量/t | 开机小时数 | 设备利用率 | 吨氨成本/(元/t) | 自发自用比例 | 绿电比例 | 上网比例 |",
213         ↪ "合格类型 | 开机时段 |",
214         "|---:|---:|---:|---:|---:|---:|---:|",
215     ]
216     for _, row in typical_summary.sort_values("production_t_per_day", ascending=False).iterrows():
217         lines.append(
218             " | {production:.0f} | {hours:.0f} | {util:.2%} | {cost:,.2f} | {self:.2%} | {green:.2%} |",
219             ↪ " | {sale:.2%} | {stype} | {on_hours} | ".format(
220                 production=row["production_t_per_day"],
221                 hours=row["required_on_hours"],
222                 util=row["equipment_utilization"],
223                 cost=row["吨氨成本"],
224                 self=row["新能源自发自用比例（物理/政策口径）"],
225                 green=row["总用电量绿电比例"],
226                 sale=row["新能源上网电量比例"],
227                 stype=row["satisfaction_type"],
228                 on_hours=row["on_hours"],
229             )
230         )
231     lines.extend(
232         [
233             "",
234             f"典型风光场景下吨氨成本最低的日产量为 **{best_typical['production_t_per_day']:.0f} t/d**,"
235             ↪ "吨氨成本为 **{best_typical['吨氨成本']:.2f} 元/t**.",
236             "",
237             "## 24 种风光场景年度加权统计",
238             "",
239             "| 日产量/t | 年产氨量/t | 年总成本/万元 | 年均吨氨成本/(元/t) | 年购电量/MWh |",
240             ↪ "年上网电量/MWh | 全满足/天 | 部分满足/天 | 全不满足/天 |",
241             "|---:|---:|---:|---:|---:|---:|---:|",
242         ]
243     )
244     for _, row in annual.iterrows():
245         lines.append(
246             " | {production:.0f} | {ammonia:.0f} | {total_cost:,.2f} | {avg_cost:,.2f} |",
247             ↪ " | {purchase:,.2f} | {sale:,.2f} | {full_days} | {partial_days} | {none_days} |",
248             ↪ " | ".format(
249                 production=row["production_t_per_day"],
250                 ammonia=row["annual_ammonia_t"],
251                 total_cost=row["annual_total_cost_yuan"] / 10000,
252                 avg_cost=row["annual_average_ton_cost_yuan_per_t"],

```

```

247         purchase=row["annual_purchase_mwh"],
248         sale=row["annual_sale_mwh"],
249         full_days=int(row["full_satisfied_days"]),
250         partial_days=int(row["partial_satisfied_days"]),
251         none_days=int(row["none_satisfied_days"]),
252     )
253 )
254
255 lines.extend(
256     [
257         """
258         f"24场景年度加权口径下，年均吨氨成本最低的日产量为
        ↪ **{best_annual['production_t_per_day']:.0f} t/d**，年均吨氨成本为
        ↪ **{best_annual['annual_average_ton_cost_yuan_per_t']:.2f} 元/t**。”，
        """
259         "说明：每种风光场景代表15天，年度统计按360天计算；年均吨氨成本按全年总成本除以全年产氨量计算。”，
260     ]
261 )
262
263 (OUT_DIR / "q2_result_summary.md").write_text("\n".join(lines), encoding="utf-8")
264
265
266 def plot_typical_schedule(typical_hourly: pd.DataFrame) -> None:
267     set_csee_style()
268     pivot = typical_hourly.pivot_table(index="production_t_per_day", columns="hour",
        ↪ values="production_on", aggfunc="first")
269     pivot = pivot.sort_index(ascending=False)
270     fig, ax = plt.subplots(figsize=(7.4, 3.8))
271     ax.imshow(pivot.values, cmap=plt.cm.Greens, aspect="auto", vmin=0, vmax=1)
272     ax.set_yticks(np.arange(len(pivot.index)))
273     ax.set_yticklabels([f"{int(v)}" for v in pivot.index])
274     ax.set_xticks(range(0, 24, 2))
275     ax.set_xlabel(" 时段/h")
276     ax.set_ylabel(" 日产量/t")
277     ax.set_title(" 典型风光场景下不同日产量的最优开机时段")
278     for i in range(pivot.shape[0]):
279         for j in range(pivot.shape[1]):
280             if pivot.values[i, j] > 0.5:
281                 ax.text(j, i, "1", ha="center", va="center", fontsize=6, color="white")
282     fig.subplots_adjust(left=0.10, right=0.98, top=0.88, bottom=0.16)
283     save_figure(fig, OUT_DIR / "q2_fig1_typical_schedule_heatmap_csee")
284     plt.close(fig)
285
286
287 def plot_typical_cost_indicator(typical_summary: pd.DataFrame) -> None:
288     set_csee_style()
289     df = typical_summary.sort_values("production_t_per_day", ascending=True)
290     x = np.arange(len(df))
291     fig, ax1 = plt.subplots(figsize=(7.5, 4.8))
292     bars = ax1.bar(x, df["吨氨成本"], color=CSEE_COLORS["light_blue"], edgecolor=CSEE_COLORS["blue"],
        ↪ linewidth=0.6, label="吨氨成本")
293     ax1.set_ylabel(" 吨氨成本/(元/t)")
294     ax1.set_xticks(x)
295     ax1.set_xticklabels([f"{int(v)}" for v in df["production_t_per_day"]])
296     ax1.set_xlabel(" 日产量/t")
297     ax1.set_ylim(0, df["吨氨成本"].max() * 1.18)
298     ax1.grid(True, axis="y", linestyle="--", alpha=0.4)
299     annotate_bars(ax1, bars, fmt="{:.0f}", dy_frac=0.01)
300
301     ax2 = ax1.twinx()
302     ax2.plot(x, df["新能源自发自用比例（物理/政策口径）"] * 100, marker="o", color=CSEE_COLORS["green"],
        ↪ label="自发自用比例")
303     ax2.plot(x, df["总用电量绿电比例"] * 100, marker="s", color=CSEE_COLORS["blue"], label="绿电比例")
304     ax2.plot(x, df["新能源上网电量比例"] * 100, marker="^", color=CSEE_COLORS["red"],
        ↪ label="上网比例")
305     ax2.axhline(60, color=CSEE_COLORS["green"], linestyle="--", linewidth=0.8)
306     ax2.axhline(30, color=CSEE_COLORS["blue"], linestyle="--", linewidth=0.8)
307     ax2.axhline(20, color=CSEE_COLORS["red"], linestyle="--", linewidth=0.8)
308     ax2.set_ylabel(" 指标值/%")
309
310     handles1, labels1 = ax1.get_legend_handles_labels()
311     handles2, labels2 = ax2.get_legend_handles_labels()
312     ax1.legend(
313         handles1 + handles2,
314         labels1 + labels2,
315         frameon=False,
316         ncol=4,
317         loc="upper center",
318         bbox_to_anchor=(0.5, -0.18),
319         borderaxespad=0,
320         columnspacing=1.1,

```

```

321 )
322 ax1.set_title(" 典型风光场景下吨氨成本与绿电指标")
323 fig.subplots_adjust(left=0.10, right=0.90, top=0.88, bottom=0.24)
324 save_figure(fig, OUT_DIR / "q2_fig2_typical_cost_indicator_csee")
325 plt.close(fig)
326
327
328 def plot_scenario_cost_box(scenario_summary: pd.DataFrame) -> None:
329     set_csee_style()
330     ordered = sorted(PRODUCTION_LEVELS_T)
331     data = [scenario_summary.loc[scenario_summary["production_t_per_day"] == d, "吨氨成本"] for d in
332             ↪ ordered]
333     fig, ax = plt.subplots(figsize=(7.4, 4.4))
334     bp = ax.boxplot(data, tick_labels=[str(d) for d in ordered], patch_artist=True, widths=0.58)
335     for patch in bp["boxes"]:
336         patch.set_facecolor(CSEE_COLORS["light_blue"])
337         patch.set_edgecolor(CSEE_COLORS["blue"])
338     ax.set_xlabel(" 日产量/t")
339     ax.set_ylabel(" 吨氨成本/(元/t)")
340     ax.set_title("24 种风光场景下吨氨成本分布")
341     ax.grid(True, axis="y", linestyle="--", alpha=0.4)
342     fig.subplots_adjust(left=0.11, right=0.98, top=0.88, bottom=0.15)
343     save_figure(fig, OUT_DIR / "q2_fig3_scenario_cost_box_csee")
344     plt.close(fig)
345
346 def plot_satisfaction_stacked(annual: pd.DataFrame) -> None:
347     set_csee_style()
348     df = annual.sort_values("production_t_per_day", ascending=True)
349     x = np.arange(len(df))
350     fig, ax = plt.subplots(figsize=(7.4, 4.4))
351     ax.bar(x, df["full_satisfied_days"], color=CSEE_COLORS["green"], label=" 全满足")
352     ax.bar(x, df["partial_satisfied_days"], bottom=df["full_satisfied_days"], color="#f7c97f",
353             ↪ label="部分满足")
354     bottom = df["full_satisfied_days"] + df["partial_satisfied_days"]
355     ax.bar(x, df["none_satisfied_days"], bottom=bottom, color=CSEE_COLORS["light_red"],
356             ↪ label="全不满足")
357     ax.set_xticks(x)
358     ax.set_xticklabels([f"{int(v)}" for v in df["production_t_per_day"]])
359     ax.set_xlabel(" 日产量/t")
360     ax.set_ylabel(" 年度代表天数/d")
361     ax.set_title("24 场景年度绿电指标合格类型统计")
362     legend_below(ax, ncol=3, y=-0.20)
363     ax.grid(True, axis="y", linestyle="--", alpha=0.4)
364     fig.subplots_adjust(left=0.10, right=0.98, top=0.88, bottom=0.24)
365     save_figure(fig, OUT_DIR / "q2_fig4_satisfaction_stacked_csee")
366     plt.close(fig)
367
368 def plot_cost_duration_curve(scenario_summary: pd.DataFrame) -> None:
369     set_csee_style()
370     fig, ax = plt.subplots(figsize=(7.4, 4.5))
371     for production in sorted(PRODUCTION_LEVELS_T, reverse=True):
372         vals = np.sort(scenario_summary.loc[scenario_summary["production_t_per_day"] == production,
373             ↪ "吨氨成本"].to_numpy())
374         x = np.arange(1, len(vals) + 1) * SCENARIO_DAYS
375         ax.step(x, vals, where="post", linewidth=1.4, label=f"{production} t/d")
376     ax.set_xlabel(" 累计代表天数/d")
377     ax.set_ylabel(" 吨氨成本/(元/t)")
378     ax.set_title(" 全年代表场景吨氨成本分布曲线")
379     ax.set_xlim(0, 360)
380     ax.grid(True, linestyle="--", alpha=0.4)
381     legend_below(ax, ncol=5, y=-0.20)
382     fig.subplots_adjust(left=0.11, right=0.98, top=0.88, bottom=0.24)
383     save_figure(fig, OUT_DIR / "q2_fig5_annual_cost_duration_csee")
384     plt.close(fig)
385
386 def plot_purchase_sale_distribution(scenario_summary: pd.DataFrame) -> None:
387     set_csee_style()
388     df = scenario_summary.sort_values("production_t_per_day")
389     levels = sorted(PRODUCTION_LEVELS_T)
390     purchase = [df.loc[df["production_t_per_day"] == d, "网购电量"] for d in levels]
391     sale = [df.loc[df["production_t_per_day"] == d, "上网电量"] for d in levels]
392     x = np.arange(len(levels))
393     fig, ax = plt.subplots(figsize=(7.4, 4.4))
394     bp1 = ax.boxplot(purchase, positions=x - 0.16, widths=0.26, patch_artist=True)
395     bp2 = ax.boxplot(sale, positions=x + 0.16, widths=0.26, patch_artist=True)
396     for patch in bp1["boxes"]:
397         patch.set_facecolor(CSEE_COLORS["light_red"])

```

```

397     patch.set_edgecolor(CSEE_COLORS["red"])
398 for patch in bp2["boxes"]:
399     patch.set_facecolor(CSEE_COLORS["light_blue"])
400     patch.set_edgecolor(CSEE_COLORS["blue"])
401 ax.set_xticks(x)
402 ax.set_xticklabels([str(d) for d in levels])
403 ax.set_xlabel(" 日产量/t")
404 ax.set_ylabel(" 日电量/MWh")
405 ax.set_title("24 场景日购电量与上网电量分布")
406 ax.legend(
407     [bp1["boxes"][0], bp2["boxes"][0]],
408     [" 网购电量", " 上网电量"],
409     frameon=False,
410     loc="upper center",
411     bbox_to_anchor=(0.5, -0.18),
412     ncol=2,
413     borderaxespad=0,
414 )
415 ax.grid(True, axis="y", linestyle="--", alpha=0.4)
416 fig.subplots_adjust(left=0.10, right=0.98, top=0.88, bottom=0.24)
417 save_figure(fig, OUT_DIR / "q2_fig6_purchase_sale_box_csee")
418 plt.close(fig)
419
420
421 def plot_scenario_cost_heatmaps(scenario_summary: pd.DataFrame) -> None:
422     set_csee_style()
423     levels = sorted(PRODUCTION_LEVELS_T, reverse=True)
424     vmin = float(scenario_summary[" 吨氨成本"].min())
425     vmax = float(scenario_summary[" 吨氨成本"].max())
426     fig, axes = plt.subplots(2, 3, figsize=(8.8, 5.4), sharex=True, sharey=True)
427     axes_flat = axes.ravel()
428     image = None
429     for ax, production in zip(axes_flat, levels):
430         matrix = (
431             scenario_summary.loc[scenario_summary["production_t_per_day"] == production]
432             .pivot_table(index="wind_scenario", columns="pv_scenario", values="吨氨成本",
433                 ↪ aggfunc="first")
434             .sort_index()
435         )
436         image = ax.imshow(matrix.values, cmap="YlGnBu", vmin=vmin, vmax=vmax, aspect="auto")
437         ax.set_title(f"{production} t/d", fontsize=9)
438         ax.set_xticks(np.arange(matrix.shape[1]))
439         ax.set_xticklabels([str(int(c)) for c in matrix.columns])
440         ax.set_yticks(np.arange(matrix.shape[0]))
441         ax.set_yticklabels([str(int(i)) for i in matrix.index])
442         for i in range(matrix.shape[0]):
443             for j in range(matrix.shape[1]):
444                 value = matrix.values[i, j]
445                 text_color = "white" if (value - vmin) / (vmax - vmin) > 0.62 else "#222222"
446                 ax.text(j, i, f"{value / 1000:.1f}", ha="center", va="center", fontsize=6.5,
447                     ↪ color=text_color)
448     axes_flat[-1].axis("off")
449     for ax in axes[:, 0]:
450         ax.set_ylabel(" 风电场景")
451     for ax in axes[-1, :2]:
452         ax.set_xlabel(" 光伏场景")
453     if image is not None:
454         cbar = fig.colorbar(image, ax=axes_flat[-1], orientation="vertical", fraction=0.026,
455             ↪ pad=0.018)
456         cbar.set_label(" 吨氨成本/(元/t)")
457     fig.suptitle(" 离散开停模式下 24 场景吨氨成本热力图 (格内为千元/t)", y=0.985, fontsize=10)
458     fig.subplots_adjust(left=0.08, right=0.90, top=0.90, bottom=0.10, wspace=0.14, hspace=0.26)
459     save_figure(fig, OUT_DIR / "q2_fig7_scenario_cost_heatmap_csee")
460     plt.close(fig)
461
462
463 def main() -> None:
464     OUT_DIR.mkdir(parents=True, exist_ok=True)
465     params = SystemParams()
466
467     typical_hourly, typical_summary = run_typical_cases(params)
468     scenario_hourly, scenario_summary = run_scenario_cases(params)
469     annual = build_annual_summary(scenario_summary)
470     distribution = build_distribution_summary(scenario_summary)
471
472     typical_hourly.to_csv(OUT_DIR / "q2_typical_hourly_schedule.csv", index=False,
473         ↪ encoding="utf-8-sig")
474     typical_summary.to_csv(OUT_DIR / "q2_typical_summary.csv", index=False, encoding="utf-8-sig")
475     scenario_hourly.to_csv(OUT_DIR / "q2_all_scenario_hourly_schedule.csv", index=False,
476         ↪ encoding="utf-8-sig")

```

```

472     scenario_summary.to_csv(OUT_DIR / "q2_all_scenario_summary.csv", index=False,
473                             ↪ encoding="utf-8-sig")
474     annual.to_csv(OUT_DIR / "q2_annual_summary.csv", index=False, encoding="utf-8-sig")
475     distribution.to_csv(OUT_DIR / "q2_distribution_summary.csv", index=False, encoding="utf-8-sig")
476
477     write_markdown_summary(typical_summary, scenario_summary, annual)
478     plot_typical_schedule(typical_hourly)
479     plot_typical_cost_indicator(typical_summary)
480     plot_scenario_cost_box(scenario_summary)
481     plot_satisfaction_stacked(annual)
482     plot_cost_duration_curve(scenario_summary)
483     plot_purchase_sale_distribution(scenario_summary)
484     plot_scenario_cost_heatmaps(scenario_summary)
485
486     print("Typical summary:")
487     print(typical_summary[["production_t_per_day", "required_on_hours", "吨氨成本",
488                             ↪ "新能源自发自用比例（物理/政策口径）", "总用电量绿电比例", "新能源上网电量比例",
489                             ↪ "satisfaction_type", "on_hours"]].to_string(index=False))
490     print("\nAnnual summary:")
491     print(annual.to_string(index=False))
492     print(f"\nOutputs written to: {OUT_DIR}")
493
494 if __name__ == "__main__":
495     main()

```

E.3 问题三：连续负荷率优化

文件：code/questions/solve_q3.py

说明：建立含购售电互斥约束的混合整数线性规划模型，量化连续可调制氨负荷相对离散开停机的经济性和合规性增益。

```

1  from __future__ import annotations
2
3  import sys
4  from pathlib import Path
5
6  import numpy as np
7  import pandas as pd
8  from scipy.optimize import Bounds, LinearConstraint, milp
9
10 PROJECT_ROOT = Path(__file__).resolve().parents[1]
11 if str(PROJECT_ROOT) not in sys.path:
12     sys.path.insert(0, str(PROJECT_ROOT))
13
14 from model_common.data import SystemParams, load_typical_day, load_wind_solar_scenarios
15 from model_common.metrics import add_process_load, q1_summary
16 from model_common.plotting import (
17     plot_q3_cost_duration_curve,
18     plot_q3_cost_delta_heatmaps,
19     plot_q3_grid_indicator_compare,
20     plot_q3_q2_cost_compare,
21     plot_q3_satisfaction_stacked,
22     plot_q3_scenario_cost_box,
23     plot_q3_typical_load_factor,
24     plot_q3_typical_load_factor_lines,
25     plot_q3_typical_power_balance,
26 )
27
28
29 OUT_DIR = Path(__file__).resolve().parent / "outputs"
30 Q2_OUT_DIR = PROJECT_ROOT / " 问题 2_ 离散制氨调节优化" / "outputs"
31 PRODUCTION_LEVELS_T = [72, 63, 54, 45, 36]
32 SCENARIO_DAYS = 15
33 CAPACITY_DAILY_AMMONIA_T = 72.0
34 MIN_LOAD_FACTOR = 0.10
35
36
37 def add_regular_load_to_scenarios(params: SystemParams) -> pd.DataFrame:
38     scenario_df = load_wind_solar_scenarios(params=params, root=PROJECT_ROOT)
39     regular = load_typical_day(params=params, root=PROJECT_ROOT)[["hour", "regular_load_mw"]]
40     return scenario_df.merge(regular, on="hour", how="left")

```

```

41
42
43 def solve_load_factor_milp(
44     hourly: pd.DataFrame,
45     params: SystemParams,
46     production_t_per_day: float,
47     big_m_mw: float = 200.0,
48 ) -> np.ndarray:
49     n_hour = len(hourly)
50     if n_hour != 24:
51         raise ValueError(f"Expected 24 hourly rows, got {n_hour}.")
52
53     full_process_mw = params.process_power_mw(CAPACITY_DAILY_AMMONIA_T)
54     full_output_t_per_h = params.ammonia_output_t_per_h(CAPACITY_DAILY_AMMONIA_T)
55     required_load_factor_sum = production_t_per_day / full_output_t_per_h
56
57     u = np.arange(0, n_hour)
58     buy = np.arange(n_hour, 2 * n_hour)
59     sell = np.arange(2 * n_hour, 3 * n_hour)
60     binary = np.arange(3 * n_hour, 4 * n_hour)
61     n_var = 4 * n_hour
62
63     c = np.zeros(n_var)
64     full_om_yuan_per_h = (
65         params.alk_power_mw(CAPACITY_DAILY_AMMONIA_T) * 1000 * params.alk_om_yuan_per_kwh
66         + params.pem_power_mw(CAPACITY_DAILY_AMMONIA_T) * 1000 * params.pem_om_yuan_per_kwh
67         + params.ammonia_power_mw(CAPACITY_DAILY_AMMONIA_T) * 1000 * params.ammonia_om_yuan_per_kwh
68     )
69     c[u] = full_om_yuan_per_h
70     c[buy] = hourly["purchase_price_yuan_per_kwh"].to_numpy() * 1000
71     c[sell] = -params.sell_price_yuan_per_kwh * 1000
72
73     lower = np.zeros(n_var)
74     upper = np.full(n_var, np.inf)
75     lower[u] = MIN_LOAD_FACTOR
76     upper[u] = 1.0
77     upper[buy] = big_m_mw
78     upper[sell] = big_m_mw
79     upper[binary] = 1.0
80
81     rows = []
82     lb = []
83     ub = []
84
85     renewable = hourly["renewable_mw"].to_numpy()
86     regular = hourly["regular_load_mw"].to_numpy()
87     for t in range(n_hour):
88         row = np.zeros(n_var)
89         row[u[t]] = full_process_mw
90         row[buy[t]] = -1.0
91         row[sell[t]] = 1.0
92         rows.append(row)
93         rhs = renewable[t] - regular[t]
94         lb.append(rhs)
95         ub.append(rhs)
96
97     row = np.zeros(n_var)
98     row[u] = 1.0
99     rows.append(row)
100     lb.append(required_load_factor_sum)
101     ub.append(required_load_factor_sum)
102
103     for t in range(n_hour):
104         row = np.zeros(n_var)
105         row[buy[t]] = 1.0
106         row[binary[t]] = -big_m_mw
107         rows.append(row)
108         lb.append(-np.inf)
109         ub.append(0.0)
110
111     for t in range(n_hour):
112         row = np.zeros(n_var)
113         row[sell[t]] = 1.0
114         row[binary[t]] = big_m_mw
115         rows.append(row)
116         lb.append(-np.inf)
117         ub.append(big_m_mw)
118
119     integrality = np.zeros(n_var)
120     integrality[binary] = 1
121
122     constraints = LinearConstraint(np.vstack(rows), np.array(lb), np.array(ub))

```

```

123     result = milp(
124         c=c,
125         integrality=integrality,
126         bounds=Bounds(lower, upper),
127         constraints=constraints,
128         options={"time_limit": 30, "mip_rel_gap": 1e-9},
129     )
130     if not result.success:
131         raise RuntimeError(f"MILP failed for {production_t_per_day} t/d: {result.message}")
132
133     load_factor = np.asarray(result.x[u], dtype=float)
134     load_factor[np.isclose(load_factor, MIN_LOAD_FACTOR, atol=1e-8)] = MIN_LOAD_FACTOR
135     load_factor[np.isclose(load_factor, 1.0, atol=1e-8)] = 1.0
136     return load_factor
137
138
139 def classify_satisfaction(record: dict[str, float | int | str]) -> None:
140     record["satisfied_count"] = int(
141         record["新能源自发自用比例是否达标"] + record["总用电量绿电比例是否达标"] +
142         ↪ record["新能源上网电量比例是否达标"]
143     )
144     if record["satisfied_count"] == 3:
145         record["satisfaction_type"] = " 全满足"
146     elif record["satisfied_count"] == 0:
147         record["satisfaction_type"] = " 全不满足"
148     else:
149         record["satisfaction_type"] = " 部分满足"
150
151 def optimize_continuous_schedule(
152     base_df: pd.DataFrame,
153     params: SystemParams,
154     production_t_per_day: float,
155     case_name: str,
156     wind_scenario: int | None = None,
157     pv_scenario: int | None = None,
158 ) -> tuple[pd.DataFrame, dict[str, float | int | str]]:
159     hourly = base_df.copy().sort_values("hour").reset_index(drop=True)
160     load_factor = solve_load_factor_milp(hourly, params, production_t_per_day)
161
162     hourly["production_load_factor"] = load_factor
163     hourly = add_process_load(
164         hourly,
165         params=params,
166         daily_ammonia_t=CAPACITY_DAILY_AMMONIA_T,
167         load_factor_col="production_load_factor",
168     )
169     net_deficit = hourly["total_load_mw"] - hourly["renewable_mw"]
170     hourly["grid_purchase_mw"] = net_deficit.clip(lower=0)
171     hourly["grid_sale_mw"] = (-net_deficit).clip(lower=0)
172     hourly["grid_exchange_mw"] = hourly["grid_purchase_mw"] - hourly["grid_sale_mw"]
173     hourly["case_name"] = case_name
174     hourly["production_t_per_day"] = production_t_per_day
175     hourly["full_load_equivalent_hours"] = production_t_per_day /
176     ↪ params.ammonia_output_t_per_h(CAPACITY_DAILY_AMMONIA_T)
177     hourly["equipment_utilization"] = production_t_per_day / CAPACITY_DAILY_AMMONIA_T
178     hourly["wind_scenario"] = -1 if wind_scenario is None else wind_scenario
179     hourly["pv_scenario"] = -1 if pv_scenario is None else pv_scenario
180
181     summary = q1_summary(hourly, params)
182     record: dict[str, float | int | str] = {
183         "case_name": case_name,
184         "wind_scenario": -1 if wind_scenario is None else wind_scenario,
185         "pv_scenario": -1 if pv_scenario is None else pv_scenario,
186         "production_t_per_day": production_t_per_day,
187         "full_load_equivalent_hours": production_t_per_day /
188         ↪ params.ammonia_output_t_per_h(CAPACITY_DAILY_AMMONIA_T),
189         "equipment_utilization": production_t_per_day / CAPACITY_DAILY_AMMONIA_T,
190         "load_factor_min": float(hourly["production_load_factor"].min()),
191         "load_factor_mean": float(hourly["production_load_factor"].mean()),
192         "load_factor_max": float(hourly["production_load_factor"].max()),
193         "load_factor_std": float(hourly["production_load_factor"].std(ddof=0)),
194     }
195     for _, row in summary.iterrows():
196         record[str(row["metric"])] = float(row["value"])
197     classify_satisfaction(record)
198     return hourly, record
199
200 def run_typical_cases(params: SystemParams) -> tuple[pd.DataFrame, pd.DataFrame]:
201     typical = load_typical_day(params=params, root=PROJECT_ROOT)

```

```

201 hourly_parts = []
202 records = []
203 for production in PRODUCTION_LEVELS_T:
204     hourly, record = optimize_continuous_schedule(typical, params, production, "typical")
205     hourly_parts.append(hourly)
206     records.append(record)
207 return pd.concat(hourly_parts, ignore_index=True), pd.DataFrame(records)
208
209
210 def run_scenario_cases(params: SystemParams) -> tuple[pd.DataFrame, pd.DataFrame]:
211     scenarios = add_regular_load_to_scenarios(params)
212     hourly_parts = []
213     records = []
214     for (scenario, wind_id, pv_id), group in scenarios.groupby(["scenario", "wind_scenario",
215     ↪ "pv_scenario"], sort=True):
216         for production in PRODUCTION_LEVELS_T:
217             hourly, record = optimize_continuous_schedule(
218                 group,
219                 params,
220                 production,
221                 str(scenario),
222                 wind_scenario=int(wind_id),
223                 pv_scenario=int(pv_id),
224             )
225             hourly_parts.append(hourly)
226             records.append(record)
227 return pd.concat(hourly_parts, ignore_index=True), pd.DataFrame(records)
228
229 def build_annual_summary(scenario_summary: pd.DataFrame) -> pd.DataFrame:
230     rows = []
231     for production, group in scenario_summary.groupby("production_t_per_day", sort=False):
232         annual_total_cost = float((group["日总成本 (扣除售电收益后)"] * SCENARIO_DAYS).sum())
233         annual_ammonia = float((group["日产氨量"] * SCENARIO_DAYS).sum())
234         annual_total_load = float((group["典型日总用电量"] * SCENARIO_DAYS).sum())
235         annual_renewable = float((group["新能源发电量"] * SCENARIO_DAYS).sum())
236         annual_purchase = float((group["网购电量"] * SCENARIO_DAYS).sum())
237         annual_sale = float((group["上网电量"] * SCENARIO_DAYS).sum())
238         annual_self_use = annual_renewable - annual_sale
239         type_counts = group["satisfaction_type"].value_counts().to_dict()
240         rows.append(
241             {
242                 "production_t_per_day": production,
243                 "annual_days": int(len(group) * SCENARIO_DAYS),
244                 "annual_ammonia_t": annual_ammonia,
245                 "annual_total_cost_yuan": annual_total_cost,
246                 "annual_average_ton_cost_yuan_per_t": annual_total_cost / annual_ammonia,
247                 "annual_total_load_mwh": annual_total_load,
248                 "annual_renewable_mwh": annual_renewable,
249                 "annual_purchase_mwh": annual_purchase,
250                 "annual_sale_mwh": annual_sale,
251                 "annual_self_use_mwh": annual_self_use,
252                 "annual_self_use_ratio": annual_self_use / annual_renewable,
253                 "annual_green_power_ratio": annual_self_use / annual_total_load,
254                 "annual_sale_ratio": annual_sale / annual_renewable,
255                 "equipment_utilization": production / CAPACITY_DAILY_AMMONIA_T,
256                 "full_satisfied_scenarios": int(type_counts.get("全满足", 0)),
257                 "partial_satisfied_scenarios": int(type_counts.get("部分满足", 0)),
258                 "none_satisfied_scenarios": int(type_counts.get("全不满足", 0)),
259                 "full_satisfied_days": int(type_counts.get("全满足", 0) * SCENARIO_DAYS),
260                 "partial_satisfied_days": int(type_counts.get("部分满足", 0) * SCENARIO_DAYS),
261                 "none_satisfied_days": int(type_counts.get("全不满足", 0) * SCENARIO_DAYS),
262             }
263         )
264     return pd.DataFrame(rows).sort_values("production_t_per_day",
265     ↪ ascending=False).reset_index(drop=True)
266
267 def build_distribution_summary(scenario_summary: pd.DataFrame) -> pd.DataFrame:
268     rows = []
269     for production, group in scenario_summary.groupby("production_t_per_day", sort=False):
270         rows.append(
271             {
272                 "production_t_per_day": production,
273                 "ton_cost_min": float(group["吨氨成本"].min()),
274                 "ton_cost_mean": float(group["吨氨成本"].mean()),
275                 "ton_cost_median": float(group["吨氨成本"].median()),
276                 "ton_cost_max": float(group["吨氨成本"].max()),
277                 "purchase_mwh_mean": float(group["网购电量"].mean()),
278                 "purchase_mwh_min": float(group["网购电量"].min()),
279                 "purchase_mwh_max": float(group["网购电量"].max()),

```

```

280         "sale_mwh_mean": float(group[" 上网电量"].mean()),
281         "sale_mwh_min": float(group[" 上网电量"].min()),
282         "sale_mwh_max": float(group[" 上网电量"].max()),
283         "self_use_ratio_mean": float(group[" 新能源自发自用比例（物理/政策口径）"].mean()),
284         "green_power_ratio_mean": float(group[" 总用电量绿电比例"].mean()),
285         "sale_ratio_mean": float(group[" 新能源上网电量比例"].mean()),
286     }
287 )
288 return pd.DataFrame(rows).sort_values("production_t_per_day",
↳ ascending=False).reset_index(drop=True)

289
290
291 def build_weighted_annual_from_scenario(scenario_summary: pd.DataFrame, prefix: str) ->
↳ pd.DataFrame:
292     rows = []
293     for production, group in scenario_summary.groupby("production_t_per_day", sort=False):
294         total_cost = float((group[" 日总成本（扣除售电收益后）"] * SCENARIO_DAYS).sum())
295         ammonia = float((group[" 日产氨量"] * SCENARIO_DAYS).sum())
296         total_load = float((group[" 典型日总用电量"] * SCENARIO_DAYS).sum())
297         renewable = float((group[" 新能源发电量"] * SCENARIO_DAYS).sum())
298         purchase = float((group[" 网购电量"] * SCENARIO_DAYS).sum())
299         sale = float((group[" 上网电量"] * SCENARIO_DAYS).sum())
300         self_use = renewable - sale
301         type_counts = group["satisfaction_type"].value_counts().to_dict()
302         rows.append(
303             {
304                 "production_t_per_day": production,
305                 f"{prefix}_annual_total_cost_yuan": total_cost,
306                 f"{prefix}_annual_average_ton_cost_yuan_per_t": total_cost / ammonia,
307                 f"{prefix}_annual_purchase_mwh": purchase,
308                 f"{prefix}_annual_sale_mwh": sale,
309                 f"{prefix}_annual_self_use_ratio": self_use / renewable,
310                 f"{prefix}_annual_green_power_ratio": self_use / total_load,
311                 f"{prefix}_annual_sale_ratio": sale / renewable,
312                 f"{prefix}_full_satisfied_days": int(type_counts.get(" 全满足", 0) * SCENARIO_DAYS),
313                 f"{prefix}_partial_satisfied_days": int(type_counts.get("部分满足", 0) *
↳ SCENARIO_DAYS),
314                 f"{prefix}_none_satisfied_days": int(type_counts.get("全不满足", 0) * SCENARIO_DAYS),
315             }
316         )
317     return pd.DataFrame(rows)

318
319
320 def build_q2_compare(q3_scenario_summary: pd.DataFrame) -> pd.DataFrame:
321     q2_path = Q2_OUT_DIR / "q2_all_scenario_summary.csv"
322     if not q2_path.exists():
323         raise FileNotFoundError(f"Cannot find Q2 scenario summary: {q2_path}")
324     q2_scenario_summary = pd.read_csv(q2_path)
325     q2 = build_weighted_annual_from_scenario(q2_scenario_summary, "q2")
326     q3 = build_weighted_annual_from_scenario(q3_scenario_summary, "q3")
327     compare = q2.merge(q3, on="production_t_per_day", how="inner")
328     compare["equipment_utilization"] = compare["production_t_per_day"] / CAPACITY_DAILY_AMMONIA_T
329     compare["ton_cost_delta_yuan_per_t"] = (
330         compare["q3_annual_average_ton_cost_yuan_per_t"] -
↳ compare["q2_annual_average_ton_cost_yuan_per_t"]
331     )
332     compare["ton_cost_delta_percent"] = (
333         compare["q3_annual_average_ton_cost_yuan_per_t"] /
↳ compare["q2_annual_average_ton_cost_yuan_per_t"] - 1.0
334     )
335     compare["purchase_delta_mwh"] = compare["q3_annual_purchase_mwh"] -
↳ compare["q2_annual_purchase_mwh"]
336     compare["sale_delta_mwh"] = compare["q3_annual_sale_mwh"] - compare["q2_annual_sale_mwh"]
337     compare["self_use_ratio_delta"] = compare["q3_annual_self_use_ratio"] -
↳ compare["q2_annual_self_use_ratio"]
338     compare["green_power_ratio_delta"] = compare["q3_annual_green_power_ratio"] -
↳ compare["q2_annual_green_power_ratio"]
339     compare["sale_ratio_delta"] = compare["q3_annual_sale_ratio"] - compare["q2_annual_sale_ratio"]
340     compare["full_satisfied_days_delta"] = compare["q3_full_satisfied_days"] -
↳ compare["q2_full_satisfied_days"]
341     compare["partial_satisfied_days_delta"] = compare["q3_partial_satisfied_days"] -
↳ compare["q2_partial_satisfied_days"]
342     compare["none_satisfied_days_delta"] = compare["q3_none_satisfied_days"] -
↳ compare["q2_none_satisfied_days"]
343     return compare.sort_values("production_t_per_day", ascending=False).reset_index(drop=True)
344
345
346 def write_markdown_summary(
347     typical_summary: pd.DataFrame,
348     annual: pd.DataFrame,
349     distribution: pd.DataFrame,

```

```

350     compare: pd.DataFrame,
351 ) -> None:
352     best_typical = typical_summary.loc[typical_summary["吨氨成本"].idxmin()]
353     typical_full = typical_summary[typical_summary["satisfaction_type"] == "全满足"]
354     best_typical_full = typical_full.loc[typical_full["吨氨成本"].idxmin()] if not typical_full.empty
355     ↪ else None
356     best_annual = annual.loc[annual["annual_average_ton_cost_yuan_per_t"].idxmin()]
357
358     lines = [
359         "# 问题三结果摘要",
360         "",
361         "## 典型风光场景",
362         "",
363         "| 日产量/t | 等效满负荷小时 | 平均负荷率 | 负荷率范围 | 吨氨成本/(元/t) | 自发自用比例 | 绿电比例",
364         ↪ | 上网比例 | 合格类型 |",
365         "|----:|----:|----:|----:|----:|----:|----:|",
366     ]
367     for _, row in typical_summary.sort_values("production_t_per_day", ascending=False).iterrows():
368         lines.append(
369             "| {production:.0f} | {hours:.2f} | {mean:.2%} | {vmin:.1%}-{vmax:.1%} | {cost:,.2f} |",
370             ↪ {self:.2%} | {green:.2%} | {sale:.2%} | {stype} |".format(
371                 production=row["production_t_per_day"],
372                 hours=row["full_load_equivalent_hours"],
373                 mean=row["load_factor_mean"],
374                 vmin=row["load_factor_min"],
375                 vmax=row["load_factor_max"],
376                 cost=row["吨氨成本"],
377                 self=row["新能源自发自用比例(物理/政策口径)"],
378                 green=row["总用电量绿电比例"],
379                 sale=row["新能源上网电量比例"],
380                 stype=row["satisfaction_type"],
381             )
382         )
383     lines.extend(
384         [
385             "",
386             f"典型风光场景下, 吨氨成本最低的连续调节方案为 **{best_typical['production_t_per_day']:.0f}",
387             ↪ t/d**, 吨氨成本为 **{best_typical['吨氨成本']:.2f} 元/t**.",
388         ]
389     )
390     if best_typical_full is not None:
391         lines.append(
392             f"若要求三项绿电指标全部满足, 典型场景下最低成本方案为",
393             ↪ **{best_typical_full['production_t_per_day']:.0f} t/d**, 吨氨成本为",
394             ↪ **{best_typical_full['吨氨成本']:.2f} 元/t**."
395         )
396     else:
397         lines.append("典型场景下没有日产量方案三项绿电指标全部满足。")
398
399     lines.extend(
400         [
401             "",
402             "## 24 种风光场景年度加权统计",
403             "",
404             "| 日产量/t | 年产氨量/t | 年总成本/万元 | 年均吨氨成本/(元/t) | 年购电量/MWh |",
405             ↪ 年上网电量/MWh | 年度自发自用比例 | 年度绿电比例 | 年度上网比例 | 全满足/天 | 部分满足/天",
406             ↪ | 全不满足/天 |",
407             "|----:|----:|----:|----:|----:|----:|----:|----:|----:|",
408         ]
409     )
410     for _, row in annual.iterrows():
411         lines.append(
412             "| {production:.0f} | {ammonia:,.0f} | {total_cost:,.2f} | {avg_cost:,.2f} |",
413             ↪ {purchase:,.2f} | {sale:,.2f} | {self:.2%} | {green:.2%} | {sale_ratio:.2%} |",
414             ↪ {full_days} | {partial_days} | {none_days} |".format(
415                 production=row["production_t_per_day"],
416                 ammonia=row["annual_ammonia_t"],
417                 total_cost=row["annual_total_cost_yuan"] / 10000,
418                 avg_cost=row["annual_average_ton_cost_yuan_per_t"],
419                 purchase=row["annual_purchase_mwh"],
420                 sale=row["annual_sale_mwh"],
421                 self=row["annual_self_use_ratio"],
422                 green=row["annual_green_power_ratio"],
423                 sale_ratio=row["annual_sale_ratio"],
424                 full_days=int(row["full_satisfied_days"]),
425                 partial_days=int(row["partial_satisfied_days"]),
426                 none_days=int(row["none_satisfied_days"]),
427             )
428         )
429

```

```

420 lines.extend(
421     [
422         """
423         f"24场景年度加权口径下, 年均吨氨成本最低的日产量为
424         ↪ **{best_annual['production_t_per_day']:.0f} t/d**, 年均吨氨成本为
425         ↪ **{best_annual['annual_average_ton_cost_yuan_per_t']:.2f} 元/t**.",
426         """
427         """
428         "## 与问题二离散开停机结果对比",
429         """
430         | 日产量/t | 设备利用率 | Q2吨氨成本 | Q3吨氨成本 | 成本变化 | Q2购电/MWh | Q3购电/MWh |
431         ↪ Q2上网/MWh | Q3上网/MWh | Q2全满足/天 | Q3全满足/天 |",
432         "|---:|---:|---:|---:|---:|---:|---:|",
433     ]
434 )
435 for _, row in compare.iterrows():
436     lines.append(
437         """
438         | {production:.0f} | {util:.2%} | {q2_cost:.2f} | {q3_cost:.2f} | {delta:+.2f} |
439         ↪ {q2_buy:.2f} | {q3_buy:.2f} | {q2_sale:.2f} | {q3_sale:.2f} | {q2_full} |
440         ↪ {q3_full} |""".format(
441             production=row["production_t_per_day"],
442             util=row["equipment_utilization"],
443             q2_cost=row["q2_annual_average_ton_cost_yuan_per_t"],
444             q3_cost=row["q3_annual_average_ton_cost_yuan_per_t"],
445             delta=row["ton_cost_delta_yuan_per_t"],
446             q2_buy=row["q2_annual_purchase_mwh"],
447             q3_buy=row["q3_annual_purchase_mwh"],
448             q2_sale=row["q2_annual_sale_mwh"],
449             q3_sale=row["q3_annual_sale_mwh"],
450             q2_full=int(row["q2_full_satisfied_days"]),
451             q3_full=int(row["q3_full_satisfied_days"]),
452         )
453     )
454 lines.extend(
455     [
456         """
457         """
458         "## 24 场景分布特征",
459         """
460         | 日产量/t | 吨氨成本最小值 | 吨氨成本均值 | 吨氨成本最大值 | 平均日购电量/MWh |
461         ↪ 平均日上网电量/MWh | 平均上网比例 |",
462         "|---:|---:|---:|---:|---:|---:|",
463     ]
464 )
465 for _, row in distribution.iterrows():
466     lines.append(
467         """
468         | {production:.0f} | {min_cost:.2f} | {mean_cost:.2f} | {max_cost:.2f} |
469         ↪ {purchase:.2f} | {sale:.2f} | {sale_ratio:.2%} |""".format(
470             production=row["production_t_per_day"],
471             min_cost=row["ton_cost_min"],
472             mean_cost=row["ton_cost_mean"],
473             max_cost=row["ton_cost_max"],
474             purchase=row["purchase_mwh_mean"],
475             sale=row["sale_mwh_mean"],
476             sale_ratio=row["sale_ratio_mean"],
477         )
478     )
479 lines.extend(
480     [
481         """
482         """
483         "说明: 问题三主模型采用全天连续运行假设, 制氨负荷率满足10%-100%; 每种风光场景代表15天,
484         ↪ 年度统计按360天计算。模型通过购售电互斥二进制变量避免同一小时同时购电和售电。",
485     ]
486 )
487 (OUT_DIR / "q3_result_summary.md").write_text("\n".join(lines), encoding="utf-8")
488
489 def main() -> None:
490     OUT_DIR.mkdir(parents=True, exist_ok=True)
491     params = SystemParams()
492
493     typical_hourly, typical_summary = run_typical_cases(params)
494     scenario_hourly, scenario_summary = run_scenario_cases(params)
495     annual = build_annual_summary(scenario_summary)
496     distribution = build_distribution_summary(scenario_summary)
497     compare = build_q2_compare(scenario_summary)
498
499     typical_hourly.to_csv(OUT_DIR / "q3_typical_hourly_schedule.csv", index=False,
500 ↪ encoding="utf-8-sig")

```

```

491 typical_summary.to_csv(OUT_DIR / "q3_typical_summary.csv", index=False, encoding="utf-8-sig")
492 scenario_hourly.to_csv(OUT_DIR / "q3_all_scenario_hourly_schedule.csv", index=False,
    ↳ encoding="utf-8-sig")
493 scenario_summary.to_csv(OUT_DIR / "q3_all_scenario_summary.csv", index=False,
    ↳ encoding="utf-8-sig")
494 annual.to_csv(OUT_DIR / "q3_annual_summary.csv", index=False, encoding="utf-8-sig")
495 distribution.to_csv(OUT_DIR / "q3_distribution_summary.csv", index=False, encoding="utf-8-sig")
496 compare.to_csv(OUT_DIR / "q3_compare_q2_summary.csv", index=False, encoding="utf-8-sig")
497
498 write_markdown_summary(typical_summary, annual, distribution, compare)
499
500 plot_q3_typical_load_factor(typical_hourly, OUT_DIR)
501 plot_q3_typical_load_factor_lines(typical_hourly, OUT_DIR)
502 plot_q3_q2_cost_compare(compare, OUT_DIR)
503 plot_q3_grid_indicator_compare(compare, OUT_DIR)
504 plot_q3_scenario_cost_box(scenario_summary, OUT_DIR, PRODUCTION_LEVELS_T)
505 plot_q3_satisfaction_stacked(annual, OUT_DIR)
506 plot_q3_cost_duration_curve(scenario_summary, OUT_DIR, PRODUCTION_LEVELS_T, SCENARIO_DAYS)
507 q2_scenario_summary = pd.read_csv(Q2_OUT_DIR / "q2_all_scenario_summary.csv")
508 plot_q3_cost_delta_heatmaps(q2_scenario_summary, scenario_summary, OUT_DIR, PRODUCTION_LEVELS_T)
509
510 typical_full = typical_summary[typical_summary["satisfaction_type"] == "全满足"]
511 selected = typical_full.loc[typical_full["吨氨成本"].idxmin()] if not typical_full.empty else
    ↳ typical_summary.loc[typical_summary["吨氨成本"].idxmin()]
512 selected_hourly = typical_hourly[typical_hourly["production_t_per_day"] ==
    ↳ selected["production_t_per_day"]]
513 plot_q3_typical_power_balance(selected_hourly, OUT_DIR, float(selected["production_t_per_day"]))
514
515 print("Typical summary:")
516 print(
517     typical_summary[
518         [
519             "production_t_per_day",
520             "full_load_equivalent_hours",
521             "load_factor_min",
522             "load_factor_mean",
523             "load_factor_max",
524             "吨氨成本",
525             "新能源自发自用比例（物理/政策口径）",
526             "总用电量绿电比例",
527             "新能源上网电量比例",
528             "satisfaction_type",
529         ]
530     ].to_string(index=False)
531 )
532 print("\nAnnual summary:")
533 print(annual.to_string(index=False))
534 print("\nQ2/Q3 comparison:")
535 print(
536     compare[
537         [
538             "production_t_per_day",
539             "q2_annual_average_ton_cost_yuan_per_t",
540             "q3_annual_average_ton_cost_yuan_per_t",
541             "ton_cost_delta_yuan_per_t",
542             "q2_annual_purchase_mwh",
543             "q3_annual_purchase_mwh",
544             "q2_annual_sale_mwh",
545             "q3_annual_sale_mwh",
546             "q2_full_satisfied_days",
547             "q3_full_satisfied_days",
548         ]
549     ].to_string(index=False)
550 )
551 print(f"\nOutputs written to: {OUT_DIR}")
552
553 if __name__ == "__main__":
554     main()
555

```

E.4 问题四：离网储能配置与电网支撑价值

文件：code/questions/solve_q4.py

说明：建立离网无储能、有储能、储能容量扫描、Pareto 权衡和并网同产量对比模型，

计算储能经济方案与公共电网支撑价值。

```
1 from __future__ import annotations
2
3 import sys
4 from pathlib import Path
5
6 import matplotlib.pyplot as plt
7 import numpy as np
8 import pandas as pd
9 from scipy.optimize import Bounds, LinearConstraint, linprog, milp
10
11 PROJECT_ROOT = Path(__file__).resolve().parents[1]
12 if str(PROJECT_ROOT) not in sys.path:
13     sys.path.insert(0, str(PROJECT_ROOT))
14
15 from model_common.data import SystemParams, load_typical_day, load_wind_solar_scenarios
16 from model_common.metrics import add_process_load
17 from model_common.plotting import CSEE_COLORS, figure_legend_below, save_figure, set_csee_style
18
19
20 OUT_DIR = Path(__file__).resolve().parent / "outputs"
21 SCENARIO_DAYS = 15
22 CAPACITY_DAILY_AMMONIA_T = 72.0
23 MIN_LOAD_FACTOR = 0.10
24 BATTERY_OM_YUAN_PER_KWH = 0.01
25 BATTERY_C_RATE = 1.0
26 CAPACITY_SCAN_MAX_MWH = 300.0
27 CAPACITY_SCAN_STEP_MWH = 5.0
28
29 # Lexicographic-like weights for the off-grid storage dispatch MILP.
30 # Regular-load shedding must be avoided first; ammonia output is then maximized.
31 SHED_PENALTY_YUAN_PER_MWH = 1.0e7
32 AMMONIA_REWARD_YUAN_PER_T = 2.0e5
33 CURTAILMENT_TIE_BREAK_YUAN_PER_MWH = 1.0
34
35
36 def add_regular_load_to_scenarios(params: SystemParams) -> pd.DataFrame:
37     scenario_df = load_wind_solar_scenarios(params=params, root=PROJECT_ROOT)
38     regular = load_typical_day(params=params, root=PROJECT_ROOT)[["hour", "regular_load_mw"]]
39     return scenario_df.merge(regular, on="hour", how="left")
40
41
42 def full_process_om_yuan_per_h(params: SystemParams) -> float:
43     return (
44         params.alk_power_mw(CAPACITY_DAILY_AMMONIA_T) * 1000 * params.alk_om_yuan_per_kwh
45         + params.pem_power_mw(CAPACITY_DAILY_AMMONIA_T) * 1000 * params.pem_om_yuan_per_kwh
46         + params.ammonia_power_mw(CAPACITY_DAILY_AMMONIA_T) * 1000 * params.ammonia_om_yuan_per_kwh
47     )
48
49
50 def add_process_columns(hourly: pd.DataFrame, params: SystemParams, load_factor: np.ndarray) ->
51     ↪ pd.DataFrame:
52     out = hourly.copy()
53     out["production_load_factor"] = np.asarray(load_factor, dtype=float)
54     return add_process_load(
55         out,
56         params=params,
57         daily_ammonia_t=CAPACITY_DAILY_AMMONIA_T,
58         load_factor_col="production_load_factor",
59     )
60
61
62 def compute_offgrid_summary(
63     hourly: pd.DataFrame,
64     params: SystemParams,
65     storage_capacity_mwh: float,
66     mode: str,
67     scenario: str,
68     wind_scenario: int,
69     pv_scenario: int,
70 ) -> dict[str, float | int | str]:
71     wind_mwh = float(hourly["wind_mw"].sum())
72     pv_mwh = float(hourly["pv_mw"].sum())
73     renewable_mwh = wind_mwh + pv_mwh
74     regular_demand_mwh = float(hourly["regular_load_mw"].sum())
75     regular_shed_mwh = float(hourly["regular_shed_mw"].sum())
76     regular_served_mwh = regular_demand_mwh - regular_shed_mwh
77     process_load_mwh = float(hourly["process_load_mw"].sum())
78     served_load_mwh = regular_served_mwh + process_load_mwh
79     demanded_load_mwh = regular_demand_mwh + process_load_mwh
```

```

79     curtailment_mwh = float(hourly["curtailment_mw"].sum())
80     renewable_used_mwh = renewable_mwh - curtailment_mwh
81     storage_charge_mwh = float(hourly["storage_charge_mw"].sum())
82     storage_discharge_mwh = float(hourly["storage_discharge_mw"].sum())
83     storage_throughput_mwh = storage_charge_mwh + storage_discharge_mwh
84     ammonia_t = float(hourly["ammonia_output_t"].sum())
85
86     wind_cost = wind_mwh * 1000 * params.wind_lcoe_yuan_per_kwh
87     pv_cost = pv_mwh * 1000 * params.pv_lcoe_yuan_per_kwh
88     alk_om = float((hourly["alk_load_mw"] * 1000 * params.alk_om_yuan_per_kwh).sum())
89     pem_om = float((hourly["pem_load_mw"] * 1000 * params.pem_om_yuan_per_kwh).sum())
90     ammonia_om = float((hourly["ammonia_load_mw"] * 1000 * params.ammonia_om_yuan_per_kwh).sum())
91     storage_daily_capital = storage_capacity_mwh * 1000 * params.storage_capex_yuan_per_kwh / (
92         params.storage_life_year * 360
93     )
94     storage_om = storage_throughput_mwh * 1000 * BATTERY_OM_YUAN_PER_KWH
95     total_cost = wind_cost + pv_cost + alk_om + pem_om + ammonia_om + storage_daily_capital +
    ↪ storage_om
96
97     storage_loss_mwh = max(renewable_used_mwh - served_load_mwh, 0.0)
98     return {
99         "mode": mode,
100         "scenario": scenario,
101         "wind_scenario": int(wind_scenario),
102         "pv_scenario": int(pv_scenario),
103         "storage_capacity_mwh": float(storage_capacity_mwh),
104         "storage_power_mw": float(storage_capacity_mwh * BATTERY_C_RATE),
105         "日产氨量": ammonia_t,
106         "设备利用率": ammonia_t / CAPACITY_DAILY_AMMONIA_T,
107         "常规负荷需求电量": regular_demand_mwh,
108         "常规负荷供电保障率": regular_served_mwh / regular_demand_mwh,
109         "制氨工艺用电量": process_load_mwh,
110         "离网服务负荷电量": served_load_mwh,
111         "离网需求电量": demanded_load_mwh,
112         "风电发电量": wind_mwh,
113         "光伏发电量": pv_mwh,
114         "新能源可发电量": renewable_mwh,
115         "新能源实际用电量": renewable_used_mwh,
116         "弃风弃光电量": curtailment_mwh,
117         "弃风弃光率": curtailment_mwh / renewable_mwh if renewable_mwh > 0 else np.nan,
118         "风光利用率": renewable_used_mwh / renewable_mwh if renewable_mwh > 0 else np.nan,
119         "储能充电量": storage_charge_mwh,
120         "储能放电量": storage_discharge_mwh,
121         "储能等效吞吐量": storage_throughput_mwh,
122         "储能损耗及自损耗估计": storage_loss_mwh,
123         "新能源上网电量比例": 0.0,
124         "新能源自发自用比例（离网实际消纳口径）": renewable_used_mwh / renewable_mwh if renewable_mwh >
    ↪ 0 else np.nan,
125         "总用电量绿电比例（离网需求保障口径）": served_load_mwh / demanded_load_mwh if demanded_load_mwh
    ↪ > 0 else np.nan,
126         "风电成本": wind_cost,
127         "光伏成本": pv_cost,
128         "碱性电解槽运维成本": alk_om,
129         "PEM 电解槽运维成本": pem_om,
130         "合成氨装置运维成本": ammonia_om,
131         "储能年化投资成本": storage_daily_capital,
132         "储能充放电运维成本": storage_om,
133         "日总成本": total_cost,
134         "吨氨成本": total_cost / ammonia_t if ammonia_t > 1e-9 else np.nan,
135         "能源自治是否满足常规负荷": int(regular_shed_mwh <= 1e-7),
136     }
137
138
139
140 def solve_no_storage_offgrid(
141     hourly: pd.DataFrame,
142     params: SystemParams,
143     scenario: str,
144     wind_scenario: int,
145     pv_scenario: int,
146 ) -> tuple[pd.DataFrame, dict[str, float | int | str]]:
147     full_process_mw = params.process_power_mw(CAPACITY_DAILY_AMMONIA_T)
148     min_process_mw = MIN_LOAD_FACTOR * full_process_mw
149     hourly = hourly.sort_values("hour").reset_index(drop=True).copy()
150     available = hourly["renewable_mw"].to_numpy() - hourly["regular_load_mw"].to_numpy()
151
152     process = np.zeros(len(hourly))
153     curtailment = np.zeros(len(hourly))
154     shed = np.zeros(len(hourly))
155     for i, value in enumerate(available):

```

```

156         if value < 0:
157             shed[i] = -value
158         elif value < min_process_mw:
159             curtailment[i] = value
160         else:
161             process[i] = min(value, full_process_mw)
162             curtailment[i] = max(value - full_process_mw, 0.0)
163
164     load_factor = process / full_process_mw
165     hourly = add_process_columns(hourly, params, load_factor)
166     hourly["production_on"] = (hourly["production_load_factor"] > 1e-9).astype(int)
167     hourly["storage_charge_mw"] = 0.0
168     hourly["storage_discharge_mw"] = 0.0
169     hourly["storage_soc_mwh"] = 0.0
170     hourly["storage_mode_binary"] = 0
171     hourly["curtailment_mw"] = curtailment
172     hourly["regular_shed_mw"] = shed
173     hourly["offgrid_balance_error_mw"] = (
174         hourly["renewable_mw"]
175         + hourly["storage_discharge_mw"]
176         + hourly["regular_shed_mw"]
177         - hourly["regular_load_mw"]
178         - hourly["process_load_mw"]
179         - hourly["storage_charge_mw"]
180         - hourly["curtailment_mw"]
181     )
182     hourly["scenario"] = scenario
183     hourly["wind_scenario"] = int(wind_scenario)
184     hourly["pv_scenario"] = int(pv_scenario)
185     hourly["mode"] = "offgrid_no_storage"
186
187     summary = compute_offgrid_summary(
188         hourly,
189         params=params,
190         storage_capacity_mwh=0.0,
191         mode="offgrid_no_storage",
192         scenario=scenario,
193         wind_scenario=wind_scenario,
194         pv_scenario=pv_scenario,
195     )
196     return hourly, summary
197
198
199 def solve_storage_offgrid(
200     hourly: pd.DataFrame,
201     params: SystemParams,
202     storage_capacity_mwh: float,
203     scenario: str,
204     wind_scenario: int,
205     pv_scenario: int,
206 ) -> tuple[pd.DataFrame, dict[str, float | int | str]]:
207     n_hour = len(hourly)
208     if n_hour != 24:
209         raise ValueError(f"Expected 24 hourly rows, got {n_hour}.")
210
211     hourly = hourly.sort_values("hour").reset_index(drop=True).copy()
212     full_process_mw = params.process_power_mw(CAPACITY_DAILY_AMMONIA_T)
213     full_output_t_per_h = params.ammonia_output_t_per_h(CAPACITY_DAILY_AMMONIA_T)
214     storage_power_mw = storage_capacity_mwh * BATTERY_C_RATE
215
216     idx_u = np.arange(0, n_hour)
217     idx_z = np.arange(n_hour, 2 * n_hour)
218     idx_ch = np.arange(2 * n_hour, 3 * n_hour)
219     idx_dis = np.arange(3 * n_hour, 4 * n_hour)
220     idx_soc = np.arange(4 * n_hour, 5 * n_hour)
221     idx_cur = np.arange(5 * n_hour, 6 * n_hour)
222     idx_shed = np.arange(6 * n_hour, 7 * n_hour)
223     idx_y = np.arange(7 * n_hour, 8 * n_hour)
224     n_var = 8 * n_hour
225
226     lower = np.zeros(n_var)
227     upper = np.full(n_var, np.inf)
228     upper[idx_u] = 1.0
229     upper[idx_z] = 1.0
230     upper[idx_ch] = storage_power_mw
231     upper[idx_dis] = storage_power_mw
232     upper[idx_soc] = storage_capacity_mwh
233     upper[idx_shed] = hourly["regular_load_mw"].to_numpy()
234     upper[idx_y] = 1.0
235
236     c = np.zeros(n_var)
237     c[idx_u] = -AMMONIA_REWARD_YUAN_PER_T * full_output_t_per_h

```

```

238 c[idx_shed] = SHED_PENALTY_YUAN_PER_MWH
239 c[idx_cur] = CURTAILMENT_TIE_BREAK_YUAN_PER_MWH
240 c[idx_ch] = BATTERY_OM_YUAN_PER_KWH * 1000
241 c[idx_dis] = BATTERY_OM_YUAN_PER_KWH * 1000
242
243 rows: list[np.ndarray] = []
244 lb: list[float] = []
245 ub: list[float] = []
246
247 renewable = hourly["renewable_mw"].to_numpy()
248 regular = hourly["regular_load_mw"].to_numpy()
249
250 for t in range(n_hour):
251     row = np.zeros(n_var)
252     row[idx_u[t]] = full_process_mw
253     row[idx_ch[t]] = 1.0
254     row[idx_cur[t]] = 1.0
255     row[idx_dis[t]] = -1.0
256     row[idx_shed[t]] = -1.0
257     rows.append(row)
258     rhs = renewable[t] - regular[t]
259     lb.append(rhs)
260     ub.append(rhs)
261
262 for t in range(n_hour):
263     row = np.zeros(n_var)
264     row[idx_u[t]] = 1.0
265     row[idx_z[t]] = -1.0
266     rows.append(row)
267     lb.append(-np.inf)
268     ub.append(0.0)
269
270     row = np.zeros(n_var)
271     row[idx_u[t]] = -1.0
272     row[idx_z[t]] = MIN_LOAD_FACTOR
273     rows.append(row)
274     lb.append(-np.inf)
275     ub.append(0.0)
276
277 for t in range(n_hour):
278     row = np.zeros(n_var)
279     row[idx_ch[t]] = 1.0
280     row[idx_y[t]] = -storage_power_mw
281     rows.append(row)
282     lb.append(-np.inf)
283     ub.append(0.0)
284
285     row = np.zeros(n_var)
286     row[idx_dis[t]] = 1.0
287     row[idx_y[t]] = storage_power_mw
288     rows.append(row)
289     lb.append(-np.inf)
290     ub.append(storage_power_mw)
291
292 eta_ch = params.storage_charge_eff
293 eta_dis = params.storage_discharge_eff
294 rho = params.storage_self_loss_ratio
295 for t in range(n_hour):
296     prev = (t - 1) % n_hour
297     row = np.zeros(n_var)
298     row[idx_soc[t]] = 1.0
299     row[idx_soc[prev]] = -(1.0 - rho)
300     row[idx_ch[t]] = -eta_ch
301     row[idx_dis[t]] = 1.0 / eta_dis
302     rows.append(row)
303     lb.append(0.0)
304     ub.append(0.0)
305
306 integrality = np.zeros(n_var)
307 integrality[idx_z] = 1
308 integrality[idx_y] = 1
309
310 result = milp(
311     c=c,
312     integrality=integrality,
313     bounds=Bounds(lower, upper),
314     constraints=LinearConstraint(np.vstack(rows), np.array(lb), np.array(ub)),
315     options={"time_limit": 30, "mip_rel_gap": 1e-8},
316 )
317 if not result.success:
318     raise RuntimeError(f"Storage off-grid MILP failed for {scenario},
319         ↳ E={storage_capacity_mwh:.1f} MWh: {result.message}")

```

```

319
320 x = np.asarray(result.x)
321 load_factor = np.clip(x[idx_u], 0, 1)
322 hourly = add_process_columns(hourly, params, load_factor)
323 hourly["production_on"] = np.rint(np.clip(x[idx_z], 0, 1)).astype(int)
324 hourly["storage_charge_mwh"] = np.clip(x[idx_ch], 0, None)
325 hourly["storage_discharge_mwh"] = np.clip(x[idx_dis], 0, None)
326 hourly["storage_soc_mwh"] = np.clip(x[idx_soc], 0, storage_capacity_mwh)
327 hourly["curtailment_mw"] = np.clip(x[idx_cur], 0, None)
328 hourly["regular_shed_mw"] = np.clip(x[idx_shed], 0, None)
329 hourly["storage_mode_binary"] = np.rint(np.clip(x[idx_y], 0, 1)).astype(int)
330 hourly["offgrid_balance_error_mw"] = (
331     hourly["renewable_mw"]
332     + hourly["storage_discharge_mw"]
333     + hourly["regular_shed_mw"]
334     - hourly["regular_load_mw"]
335     - hourly["process_load_mw"]
336     - hourly["storage_charge_mw"]
337     - hourly["curtailment_mw"]
338 )
339 hourly["scenario"] = scenario
340 hourly["wind_scenario"] = int(wind_scenario)
341 hourly["pv_scenario"] = int(pv_scenario)
342 hourly["mode"] = "offgrid_storage"
343 hourly["storage_capacity_mwh"] = storage_capacity_mwh
344
345 summary = compute_offgrid_summary(
346     hourly,
347     params=params,
348     storage_capacity_mwh=storage_capacity_mwh,
349     mode="offgrid_storage",
350     scenario=scenario,
351     wind_scenario=wind_scenario,
352     pv_scenario=pv_scenario,
353 )
354 return hourly, summary
355
356
357 def compute_grid_summary(
358     hourly: pd.DataFrame,
359     params: SystemParams,
360     mode: str,
361     scenario: str,
362     wind_scenario: int,
363     pv_scenario: int,
364 ) -> dict[str, float | int | str]:
365     total_load_mwh = float(hourly["total_load_mw"].sum())
366     regular_load_mwh = float(hourly["regular_load_mw"].sum())
367     process_load_mwh = float(hourly["process_load_mw"].sum())
368     wind_mwh = float(hourly["wind_mw"].sum())
369     pv_mwh = float(hourly["pv_mw"].sum())
370     renewable_mwh = wind_mwh + pv_mwh
371     purchase_mwh = float(hourly["grid_purchase_mw"].sum())
372     sale_mwh = float(hourly["grid_sale_mw"].sum())
373     ammonia_t = float(hourly["ammonia_output_t"].sum())
374     self_use_mwh = renewable_mwh - sale_mwh
375
376     wind_cost = wind_mwh * 1000 * params.wind_lcoe_yuan_per_kwh
377     pv_cost = pv_mwh * 1000 * params.pv_lcoe_yuan_per_kwh
378     purchase_cost = float((hourly["grid_purchase_mw"] * 1000 *
379         ↪ hourly["purchase_price_yuan_per_kwh"]).sum())
380     alk_om = float((hourly["alk_load_mw"] * 1000 * params.alk_om_yuan_per_kwh).sum())
381     pem_om = float((hourly["pem_load_mw"] * 1000 * params.pem_om_yuan_per_kwh).sum())
382     ammonia_om = float((hourly["ammonia_load_mw"] * 1000 * params.ammonia_om_yuan_per_kwh).sum())
383     sale_revenue = sale_mwh * 1000 * params.sell_price_yuan_per_kwh
384     total_cost = wind_cost + pv_cost + purchase_cost + alk_om + pem_om + ammonia_om - sale_revenue
385
386     return {
387         "mode": mode,
388         "scenario": scenario,
389         "wind_scenario": int(wind_scenario),
390         "pv_scenario": int(pv_scenario),
391         "日产氨量": ammonia_t,
392         "设备利用率": ammonia_t / CAPACITY_DAILY_AMMONIA_T,
393         "典型日总用电量": total_load_mwh,
394         "常规电负荷用电量": regular_load_mwh,
395         "制氨工艺用电量": process_load_mwh,
396         "风电发电量": wind_mwh,
397         "光伏发电量": pv_mwh,
398         "新能源发电量": renewable_mwh,
399         "网购电量": purchase_mwh,

```

```

399     " 上网电量": sale_mwh,
400     " 新能源自发自用电量（物理/政策口径）": self_use_mwh,
401     " 新能源自发自用比例（物理/政策口径）": self_use_mwh / renewable_mwh if renewable_mwh > 0 else
    ↪ np.nan,
402     " 总用电量绿电比例": self_use_mwh / total_load_mwh if total_load_mwh > 0 else np.nan,
403     " 新能源上网电量比例": sale_mwh / renewable_mwh if renewable_mwh > 0 else np.nan,
404     " 风电成本": wind_cost,
405     " 光伏成本": pv_cost,
406     " 购电成本": purchase_cost,
407     " 碱性电解槽运维成本": alk_om,
408     " PEM 电解槽运维成本": pem_om,
409     " 合成氨装置运维成本": ammonia_om,
410     " 余电上网收益": sale_revenue,
411     " 日总成本（扣除售电收益后）": total_cost,
412     " 吨氨成本": total_cost / ammonia_t if ammonia_t > 1e-9 else np.nan,
413 }
414
415
416 def solve_grid_connected_same_production(
417     hourly: pd.DataFrame,
418     params: SystemParams,
419     target_ammonia_t: float,
420     scenario: str,
421     wind_scenario: int,
422     pv_scenario: int,
423     big_m_mw: float = 250.0,
424 ) -> tuple[pd.DataFrame, dict[str, float | int | str]]:
425     n_hour = len(hourly)
426     if n_hour != 24:
427         raise ValueError(f"Expected 24 hourly rows, got {n_hour}.")
428
429     hourly = hourly.sort_values("hour").reset_index(drop=True).copy()
430     full_process_mw = params.process_power_mw(CAPACITY_DAILY_AMMONIA_T)
431     full_output_t_per_h = params.ammonia_output_t_per_h(CAPACITY_DAILY_AMMONIA_T)
432
433     idx_u = np.arange(0, n_hour)
434     idx_z = np.arange(n_hour, 2 * n_hour)
435     idx_buy = np.arange(2 * n_hour, 3 * n_hour)
436     idx_sell = np.arange(3 * n_hour, 4 * n_hour)
437     idx_b = np.arange(4 * n_hour, 5 * n_hour)
438     n_var = 5 * n_hour
439
440     lower = np.zeros(n_var)
441     upper = np.full(n_var, np.inf)
442     upper[idx_u] = 1.0
443     upper[idx_z] = 1.0
444     upper[idx_buy] = big_m_mw
445     upper[idx_sell] = big_m_mw
446     upper[idx_b] = 1.0
447
448     c = np.zeros(n_var)
449     c[idx_u] = full_process_om_yuan_per_h(params)
450     c[idx_buy] = hourly["purchase_price_yuan_per_kwh"].to_numpy() * 1000
451     c[idx_sell] = -params.sell_price_yuan_per_kwh * 1000
452
453     rows: list[np.ndarray] = []
454     lb: list[float] = []
455     ub: list[float] = []
456
457     renewable = hourly["renewable_mw"].to_numpy()
458     regular = hourly["regular_load_mw"].to_numpy()
459     for t in range(n_hour):
460         row = np.zeros(n_var)
461         row[idx_u[t]] = full_process_mw
462         row[idx_buy[t]] = -1.0
463         row[idx_sell[t]] = 1.0
464         rows.append(row)
465         rhs = renewable[t] - regular[t]
466         lb.append(rhs)
467         ub.append(rhs)
468
469     row = np.zeros(n_var)
470     row[idx_u] = full_output_t_per_h
471     rows.append(row)
472     lb.append(target_ammonia_t)
473     ub.append(target_ammonia_t)
474
475     for t in range(n_hour):
476         row = np.zeros(n_var)
477         row[idx_u[t]] = 1.0
478         row[idx_z[t]] = -1.0

```

```

479     rows.append(row)
480     lb.append(-np.inf)
481     ub.append(0.0)
482
483     row = np.zeros(n_var)
484     row[idx_u[t]] = -1.0
485     row[idx_z[t]] = MIN_LOAD_FACTOR
486     rows.append(row)
487     lb.append(-np.inf)
488     ub.append(0.0)
489
490     row = np.zeros(n_var)
491     row[idx_buy[t]] = 1.0
492     row[idx_b[t]] = -big_m_mw
493     rows.append(row)
494     lb.append(-np.inf)
495     ub.append(0.0)
496
497     row = np.zeros(n_var)
498     row[idx_sell[t]] = 1.0
499     row[idx_b[t]] = big_m_mw
500     rows.append(row)
501     lb.append(-np.inf)
502     ub.append(big_m_mw)
503
504     integrality = np.zeros(n_var)
505     integrality[idx_z] = 1
506     integrality[idx_b] = 1
507
508     result = milp(
509         c=c,
510         integrality=integrality,
511         bounds=Bounds(lower, upper),
512         constraints=LinearConstraint(np.vstack(rows), np.array(lb), np.array(ub)),
513         options={"time_limit": 30, "mip_rel_gap": 1e-9},
514     )
515     if not result.success:
516         raise RuntimeError(f"Grid-connected MILP failed for {scenario},
517             ↳ target={target_ammonia_t:.4f} t: {result.message}")
518
519     x = np.asarray(result.x)
520     load_factor = np.clip(x[idx_u], 0, 1)
521     hourly = add_process_columns(hourly, params, load_factor)
522     hourly["production_on"] = np.rint(np.clip(x[idx_z], 0, 1)).astype(int)
523     hourly["grid_purchase_mw"] = np.clip(x[idx_buy], 0, None)
524     hourly["grid_sale_mw"] = np.clip(x[idx_sell], 0, None)
525     hourly["grid_exchange_mw"] = hourly["grid_purchase_mw"] - hourly["grid_sale_mw"]
526     hourly["purchase_sale_binary"] = np.rint(np.clip(x[idx_b], 0, 1)).astype(int)
527     hourly["target_ammonia_t"] = target_ammonia_t
528     hourly["scenario"] = scenario
529     hourly["wind_scenario"] = int(wind_scenario)
530     hourly["pv_scenario"] = int(pv_scenario)
531     hourly["mode"] = "grid_connected_same_production"
532
533     summary = compute_grid_summary(
534         hourly,
535         params=params,
536         mode="grid_connected_same_production",
537         scenario=scenario,
538         wind_scenario=wind_scenario,
539         pv_scenario=pv_scenario,
540     )
541     summary["target_ammonia_t"] = target_ammonia_t
542     return hourly, summary
543
544 def run_no_storage_cases(params: SystemParams) -> tuple[pd.DataFrame, pd.DataFrame]:
545     scenarios = add_regular_load_to_scenarios(params)
546     hourly_parts = []
547     records = []
548     for (scenario, wind_id, pv_id), group in scenarios.groupby(["scenario", "wind_scenario",
549         ↳ "pv_scenario"], sort=True):
550         hourly, record = solve_no_storage_offgrid(group, params, str(scenario), int(wind_id),
551             ↳ int(pv_id))
552         hourly_parts.append(hourly)
553         records.append(record)
554     return pd.concat(hourly_parts, ignore_index=True), pd.DataFrame(records)
555
556 def scan_storage_capacity(
557     base_hourly: pd.DataFrame,
558     params: SystemParams,

```

```

558     scenario: str,
559     wind_scenario: int,
560     pv_scenario: int,
561 ) -> pd.DataFrame:
562     capacities = np.arange(0.0, CAPACITY_SCAN_MAX_MWH + 0.1, CAPACITY_SCAN_STEP_MWH)
563     records = []
564     for capacity in capacities:
565         _, record = solve_storage_offgrid(base_hourly, params, float(capacity), scenario,
566                                         ↪ wind_scenario, pv_scenario)
567         records.append(record)
568     scan = pd.DataFrame(records)
569     scan["ammonia_gain_t"] = scan["日产氨量"] - float(scan.loc[scan["storage_capacity_mwh"] == 0.0,
570     ↪ "日产氨量"].iloc[0])
571     scan["curtailment_reduction_mwh"] = float(scan.loc[scan["storage_capacity_mwh"] == 0.0,
572     ↪ "弃风弃光电量"].iloc[0]) - scan["弃风弃光电量"]
573     return scan
574
575 def choose_storage_capacity(scan: pd.DataFrame) -> tuple[float, pd.Series, str]:
576     base_ammonia = float(scan.loc[scan["storage_capacity_mwh"] == 0.0, "日产氨量"].iloc[0])
577     positive = scan[(scan["storage_capacity_mwh"] > 0) & (scan["日产氨量"] > base_ammonia +
578     ↪ 1e-5)].copy()
579     if positive.empty:
580         row = scan.loc[scan["吨氨成本"].idxmin()]
581         return float(row["storage_capacity_mwh"]), row, "未找到可提升日产氨量的正储能容量,
582         ↪ 按全扫描吨氨成本最小选择。"
583     row = positive.loc[positive["吨氨成本"].idxmin()]
584     return float(row["storage_capacity_mwh"]), row, "在能够提升无储能日产氨量的正储能容量中,
585     ↪ 选择吨氨成本最低方案。"
586
587 def build_storage_pareto_summary(scan: pd.DataFrame) -> pd.DataFrame:
588     rows: list[dict[str, float | str]] = []
589     thresholds = [0.20, 0.15, 0.10, 0.05, 0.01, 0.0]
590     for eps in thresholds:
591         if eps == 0.0:
592             feasible = scan[scan["弃风弃光率"] <= 1e-9].copy()
593             label = "零弃电"
594         else:
595             feasible = scan[scan["弃风弃光率"] <= eps].copy()
596             label = f"弃电率不超过 {eps:.0%}"
597         if feasible.empty:
598             continue
599         row = feasible.loc[feasible["吨氨成本"].idxmin()]
600         rows.append(
601             {
602                 "scheme": label,
603                 "epsilon_curtailment_rate": eps,
604                 "storage_capacity_mwh": float(row["storage_capacity_mwh"]),
605                 "storage_power_mw": float(row["storage_power_mw"]),
606                 "daily_ammonia_t": float(row["日产氨量"]),
607                 "curtailment_mwh": float(row["弃风弃光电量"]),
608                 "curtailment_rate": float(row["弃风弃光率"]),
609                 "renewable_utilization": float(row["风光利用率"]),
610                 "ton_cost_yuan_per_t": float(row["吨氨成本"]),
611             }
612         )
613     for label, idx in [
614         ("经济最优正储能", scan[scan["storage_capacity_mwh"] > 0][scan["吨氨成本"].idxmin()]),
615         ("最大日产氨/消纳优先", scan[scan["日产氨量"].idxmax()]),
616     ]:
617         row = scan.loc[idx]
618         rows.append(
619             {
620                 "scheme": label,
621                 "epsilon_curtailment_rate": np.nan,
622                 "storage_capacity_mwh": float(row["storage_capacity_mwh"]),
623                 "storage_power_mw": float(row["storage_power_mw"]),
624                 "daily_ammonia_t": float(row["日产氨量"]),
625                 "curtailment_mwh": float(row["弃风弃光电量"]),
626                 "curtailment_rate": float(row["弃风弃光率"]),
627                 "renewable_utilization": float(row["风光利用率"]),
628                 "ton_cost_yuan_per_t": float(row["吨氨成本"]),
629             }
630         )
631     return pd.DataFrame(rows).drop_duplicates(subset=["scheme"], keep="first")

```

```

631 def run_storage_cases(params: SystemParams, storage_capacity_mwh: float) -> tuple[pd.DataFrame,
↳ pd.DataFrame]:
632     scenarios = add_regular_load_to_scenarios(params)
633     hourly_parts = []
634     records = []
635     for (scenario, wind_id, pv_id), group in scenarios.groupby(["scenario", "wind_scenario",
↳ "pv_scenario"], sort=True):
636         hourly, record = solve_storage_offgrid(group, params, storage_capacity_mwh, str(scenario),
↳ int(wind_id), int(pv_id))
637         hourly_parts.append(hourly)
638         records.append(record)
639     return pd.concat(hourly_parts, ignore_index=True), pd.DataFrame(records)
640
641
642 def run_grid_same_production_cases(
643     params: SystemParams,
644     storage_summary: pd.DataFrame,
645 ) -> tuple[pd.DataFrame, pd.DataFrame]:
646     scenarios = add_regular_load_to_scenarios(params)
647     targets = storage_summary.set_index("scenario")["日产电量"].to_dict()
648     hourly_parts = []
649     records = []
650     for (scenario, wind_id, pv_id), group in scenarios.groupby(["scenario", "wind_scenario",
↳ "pv_scenario"], sort=True):
651         target = float(targets[str(scenario)])
652         hourly, record = solve_grid_connected_same_production(group, params, target, str(scenario),
↳ int(wind_id), int(pv_id))
653         hourly_parts.append(hourly)
654         records.append(record)
655     return pd.concat(hourly_parts, ignore_index=True), pd.DataFrame(records)
656
657
658 def build_offgrid_annual_summary(summary: pd.DataFrame, mode: str) -> pd.DataFrame:
659     total_cost = float((summary["日总成本"] * SCENARIO_DAYS).sum())
660     ammonia = float((summary["日产氨量"] * SCENARIO_DAYS).sum())
661     renewable = float((summary["新能源可发电量"] * SCENARIO_DAYS).sum())
662     curtailment = float((summary["弃风弃光电量"] * SCENARIO_DAYS).sum())
663     shed = float((summary["常规负荷缺供电量"] * SCENARIO_DAYS).sum())
664     regular = float((summary["常规负荷需求电量"] * SCENARIO_DAYS).sum())
665     served_load = float((summary["离网服务负荷电量"] * SCENARIO_DAYS).sum())
666     demand_load = float((summary["离网需求电量"] * SCENARIO_DAYS).sum())
667     storage_capacity = float(summary["storage_capacity_mwh"].iloc[0])
668     return pd.DataFrame(
669         [
670             {
671                 "mode": mode,
672                 "annual_days": int(len(summary) * SCENARIO_DAYS),
673                 "storage_capacity_mwh": storage_capacity,
674                 "storage_power_mw": storage_capacity * BATTERY_C_RATE,
675                 "annual_ammonia_t": ammonia,
676                 "annual_capacity_utilization": ammonia / (CAPACITY_DAILY_AMMONIA_T * len(summary) *
↳ SCENARIO_DAYS),
677                 "annual_total_cost_yuan": total_cost,
678                 "annual_average_ton_cost_yuan_per_t": total_cost / ammonia,
679                 "annual_renewable_mwh": renewable,
680                 "annual_curtailment_mwh": curtailment,
681                 "annual_renewable_utilization": (renewable - curtailment) / renewable,
682                 "annual_regular_shed_mwh": shed,
683                 "annual_regular_supply_ratio": (regular - shed) / regular,
684                 "annual_served_load_mwh": served_load,
685                 "annual_demand_load_mwh": demand_load,
686                 "annual_green_demand_supply_ratio": served_load / demand_load,
687                 "energy_autonomous_days": int((summary["常规负荷缺供电量"] <= 1e-7).sum() *
↳ SCENARIO_DAYS),
688                 "full_production_days": int((summary["日产氨量"] >= CAPACITY_DAILY_AMMONIA_T -
↳ 1e-7).sum() * SCENARIO_DAYS),
689             }
690         ]
691     )
692
693
694 def build_grid_annual_summary(summary: pd.DataFrame) -> pd.DataFrame:
695     total_cost = float((summary["日总成本(扣除售电收益后)"] * SCENARIO_DAYS).sum())
696     ammonia = float((summary["日产氨量"] * SCENARIO_DAYS).sum())
697     renewable = float((summary["新能源发电量"] * SCENARIO_DAYS).sum())
698     purchase = float((summary["网购电量"] * SCENARIO_DAYS).sum())
699     sale = float((summary["上网电量"] * SCENARIO_DAYS).sum())
700     total_load = float((summary["典型日总用电量"] * SCENARIO_DAYS).sum())
701     self_use = renewable - sale
702     return pd.DataFrame(
703         [

```

```

704         {
705             "mode": "grid_connected_same_production",
706             "annual_days": int(len(summary) * SCENARIO_DAYS),
707             "annual_ammonia_t": ammonia,
708             "annual_capacity_utilization": ammonia / (CAPACITY_DAILY_AMMONIA_T * len(summary) *
709             ↪ SCENARIO_DAYS),
710             "annual_total_cost_yuan": total_cost,
711             "annual_average_ton_cost_yuan_per_t": total_cost / ammonia,
712             "annual_renewable_mwh": renewable,
713             "annual_purchase_mwh": purchase,
714             "annual_sale_mwh": sale,
715             "annual_self_use_mwh": self_use,
716             "annual_self_use_ratio": self_use / renewable,
717             "annual_green_power_ratio": self_use / total_load,
718             "annual_sale_ratio": sale / renewable,
719         }
720     ]
721 )
722
723 def build_mode_comparison(
724     no_storage_annual: pd.DataFrame,
725     storage_annual: pd.DataFrame,
726     grid_annual: pd.DataFrame,
727     no_storage_summary: pd.DataFrame,
728     storage_summary: pd.DataFrame,
729     grid_summary: pd.DataFrame,
730 ) -> pd.DataFrame:
731     rows = []
732     for label, annual, summary, cost_col in [
733         ("离网无储能", no_storage_annual, no_storage_summary, "日总成本"),
734         ("离网 + 储能", storage_annual, storage_summary, "日总成本"),
735         ("并网同产量", grid_annual, grid_summary, "日总成本（扣除售电收益后）"),
736     ]:
737         row = annual.iloc[0].to_dict()
738         row["display_mode"] = label
739         row["annual_wind_cost_yuan"] = float((summary["风电成本"] * SCENARIO_DAYS).sum())
740         row["annual_pv_cost_yuan"] = float((summary["光伏成本"] * SCENARIO_DAYS).sum())
741         row["annual_alk_om_yuan"] = float((summary["碱性电解槽运维成本"] * SCENARIO_DAYS).sum())
742         row["annual_pem_om_yuan"] = float((summary["PEM 电解槽运维成本"] * SCENARIO_DAYS).sum())
743         row["annual_ammonia_om_yuan"] = float((summary["合成氨装置运维成本"] * SCENARIO_DAYS).sum())
744         row["annual_total_cost_check_yuan"] = float((summary[cost_col] * SCENARIO_DAYS).sum())
745         if "储能年化投资成本" in summary.columns:
746             row["annual_storage_capital_yuan"] = float((summary["储能年化投资成本"] *
747             ↪ SCENARIO_DAYS).sum())
748             row["annual_storage_om_yuan"] = float((summary["储能充放电运维成本"] *
749             ↪ SCENARIO_DAYS).sum())
750         else:
751             row["annual_storage_capital_yuan"] = 0.0
752             row["annual_storage_om_yuan"] = 0.0
753         if "购电成本" in summary.columns:
754             row["annual_purchase_cost_yuan"] = float((summary["购电成本"] * SCENARIO_DAYS).sum())
755             row["annual_sale_revenue_yuan"] = float((summary["余电上网收益"] * SCENARIO_DAYS).sum())
756         else:
757             row["annual_purchase_cost_yuan"] = 0.0
758             row["annual_sale_revenue_yuan"] = 0.0
759         rows.append(row)
760     comp = pd.DataFrame(rows)
761     off = comp.loc[comp["display_mode"] == "离网 + 储能", "annual_total_cost_yuan"].iloc[0]
762     grid = comp.loc[comp["display_mode"] == "并网同产量", "annual_total_cost_yuan"].iloc[0]
763     ammonia = comp.loc[comp["display_mode"] == "离网 + 储能", "annual_ammonia_t"].iloc[0]
764     comp["grid_support_value_total_yuan"] = off - grid
765     comp["grid_support_value_yuan_per_t"] = (off - grid) / ammonia
766     return comp
767
768 def estimate_min_capacity_for_full_autonomy(params: SystemParams) -> pd.DataFrame:
769     scenarios = add_regular_load_to_scenarios(params)
770     full_process_mw = params.process_power_mw(CAPACITY_DAILY_AMMONIA_T)
771     demand = scenarios["regular_load_mw"].to_numpy() + full_process_mw
772     a_ub = -scenarios[["wind_pu", "pv_pu"]].to_numpy()
773     b_ub = -demand
774     result = linprog(c=np.array([1.0, 1.0]), A_ub=a_ub, b_ub=b_ub, bounds=[(0, None), (0, None)],
775     ↪ method="highs")
776     if not result.success:
777         raise RuntimeError(f"Minimum capacity LP failed: {result.message}")
778     wind_cap, pv_cap = map(float, result.x)
779     min_margin = float((scenarios["wind_pu"] * wind_cap + scenarios["pv_pu"] * pv_cap -
780     ↪ demand).min())
781     current_mix_supply = scenarios["wind_mw"].to_numpy() + scenarios["pv_mw"].to_numpy()
782     fixed_ratio_scale = float(np.max(demand / current_mix_supply))

```

```

780 fixed_wind_cap = params.wind_capacity_mw * fixed_ratio_scale
781 fixed_pv_cap = params.pv_capacity_mw * fixed_ratio_scale
782 fixed_margin = float((current_mix_supply * fixed_ratio_scale - demand).min())
783 return pd.DataFrame(
784     [
785         {
786             "target": "min_total_capacity_unconstrained_mix",
787             "description": " 不固定风光比例、以总装机最小为目标",
788             "wind_capacity_mw": wind_cap,
789             "pv_capacity_mw": pv_cap,
790             "total_capacity_mw": wind_cap + pv_cap,
791             "wind_capacity_multiple": wind_cap / params.wind_capacity_mw,
792             "pv_capacity_multiple": pv_cap / params.pv_capacity_mw if params.pv_capacity_mw > 0
793             ↪ else np.nan,
794             "total_capacity_multiple_vs_current": (wind_cap + pv_cap) / (params.wind_capacity_mw
795             ↪ + params.pv_capacity_mw),
796             "minimum_hourly_margin_mw": min_margin,
797         },
798         {
799             "target": "fixed_current_wind_pv_ratio",
800             "description": " 保持当前 40MW 风电与 64MW 光伏比例",
801             "wind_capacity_mw": fixed_wind_cap,
802             "pv_capacity_mw": fixed_pv_cap,
803             "total_capacity_mw": fixed_wind_cap + fixed_pv_cap,
804             "wind_capacity_multiple": fixed_ratio_scale,
805             "pv_capacity_multiple": fixed_ratio_scale,
806             "total_capacity_multiple_vs_current": fixed_ratio_scale,
807             "minimum_hourly_margin_mw": fixed_margin,
808         },
809     ],
810 )
811
812 def plot_no_storage_heatmaps(summary: pd.DataFrame, output_dir: Path) -> None:
813     set_csee_style()
814     metrics = [
815         (" 日产氨量", " 日产氨量/t", "YlGnBu", "{:.1f}"),
816         (" 弃风弃光电量", " 弃风弃光/MWh", "YlOrRd", "{:.0f}"),
817         (" 常规负荷缺供电量", " 常规缺供/MWh", "Reds", "{:.1f}"),
818         (" 风光利用率", " 风光利用率/%", "YlGn", "{:.0f}"),
819     ]
820     fig, axes = plt.subplots(2, 2, figsize=(8.2, 6.0), sharex=True, sharey=True)
821     for ax, (metric, title, cmap, fmt) in zip(axes.ravel(), metrics):
822         matrix = summary.pivot_table(index="wind_scenario", columns="pv_scenario", values=metric,
823         ↪ aggfunc="first").sort_index()
824         values = matrix.values * 100 if " 率" in metric else matrix.values
825         im = ax.imshow(values, aspect="auto", cmap=cmap)
826         ax.set_title(title, fontsize=9)
827         ax.set_xticks(np.arange(matrix.shape[1]))
828         ax.set_xticklabels([str(int(c)) for c in matrix.columns])
829         ax.set_yticks(np.arange(matrix.shape[0]))
830         ax.set_yticklabels([str(int(i)) for i in matrix.index])
831         vmax = np.nanmax(values)
832         vmin = np.nanmin(values)
833         for i in range(matrix.shape[0]):
834             for j in range(matrix.shape[1]):
835                 val = values[i, j]
836                 color = "white" if vmax > vmin and val > vmin + 0.62 * (vmax - vmin) else "#222222"
837                 ax.text(j, i, fmt.format(val), ha="center", va="center", fontsize=6.6, color=color)
838         cbar = fig.colorbar(im, ax=ax, fraction=0.044, pad=0.018)
839         ax.tick_params(labelsize=7)
840     for ax in axes[:, 0]:
841         ax.set_ylabel(" 风电场景")
842     for ax in axes[-1, :]:
843         ax.set_xlabel(" 光伏场景")
844     fig.suptitle(" 离网无储能下 24 种风光场景运行结果", y=0.985, fontsize=10)
845     fig.subplots_adjust(left=0.08, right=0.96, top=0.90, bottom=0.09, wspace=0.22, hspace=0.28)
846     save_figure(fig, output_dir / "q4_fig1_no_storage_heatmaps_csee")
847     plt.close(fig)
848
849 def plot_storage_capacity_scan(scan: pd.DataFrame, selected_capacity: float, output_dir: Path) ->
850     ↪ None:
851     set_csee_style()
852     df = scan.sort_values("storage_capacity_mwh")
853     fig, axes = plt.subplots(2, 1, figsize=(7.6, 6.0), sharex=True)
854     ax = axes[0]
855     ax.plot(df["storage_capacity_mwh"], df["日产氨量"], color=CSEE_COLORS["green"], marker="o",
856     ↪ label="日产氨量")
857     ax.set_ylabel(" 日产氨量/t")
858     ax.grid(True, axis="y", linestyle="--", alpha=0.4)

```

```

856 ax2 = ax.twinx()
857 ax2.plot(df["storage_capacity_mwh"], df["弃风弃光电量"], color=CSEE_COLORS["red"], marker="s",
↪ label="弃风弃光")
858 ax2.set_ylabel(" 弃风弃光/MWh")
859 ax.axvline(selected_capacity, color="black", linestyle="--", linewidth=0.9)
860 handles1, labels1 = ax.get_legend_handles_labels()
861 handles2, labels2 = ax2.get_legend_handles_labels()
862 figure_legend_below(fig, handles1 + handles2, labels1 + labels2, ncol=2, y=0.49)
863
864 ax = axes[1]
865 ax.plot(df["storage_capacity_mwh"], df["吨氨成本"], color=CSEE_COLORS["blue"], marker="^",
↪ label="吨氨成本")
866 ax.axvline(selected_capacity, color="black", linestyle="--", linewidth=0.9, label=" 选定容量")
867 ax.set_xlabel(" 储能容量/MWh (功率按 1C 配置)")
868 ax.set_ylabel(" 吨氨成本/(元/t)")
869 ax.grid(True, axis="y", linestyle="--", alpha=0.4)
870 ax.legend(frameon=False, ncol=2, loc="upper center", bbox_to_anchor=(0.5, -0.22))
871 fig.suptitle(" 最大弃电场景下储能容量扫描结果", y=0.985, fontsize=10)
872 fig.subplots_adjust(left=0.10, right=0.90, top=0.90, bottom=0.16, hspace=0.42)
873 save_figure(fig, output_dir / "q4_fig2_storage_capacity_scan_csee")
874 plt.close(fig)
875
876
877 def plot_storage_pareto(scan: pd.DataFrame, pareto: pd.DataFrame, selected_capacity: float,
↪ output_dir: Path) -> None:
878     set_csee_style()
879     df = scan.sort_values("storage_capacity_mwh")
880     fig, ax1 = plt.subplots(figsize=(7.6, 4.8))
881     sc = ax1.scatter(
882         df[" 弃风弃光率"] * 100,
883         df[" 吨氨成本"],
884         c=df["storage_capacity_mwh"],
885         cmap="YlGnBu",
886         s=34,
887         edgecolor="#444444",
888         linewidth=0.35,
889         label=" 容量扫描点",
890     )
891     selected = df.loc[df["storage_capacity_mwh"].eq(selected_capacity)].iloc[0]
892     ax1.scatter(
893         [selected[" 弃风弃光率"] * 100],
894         [selected[" 吨氨成本"]],
895         s=70,
896         marker="*",
897         color=CSEE_COLORS["red"],
898         edgecolor="black",
899         linewidth=0.5,
900         label=" 经济方案",
901         zorder=4,
902     )
903     zero = df.loc[df[" 弃风弃光电量"].idxmin()]
904     ax1.scatter(
905         [zero[" 弃风弃光率"] * 100],
906         [zero[" 吨氨成本"]],
907         s=64,
908         marker="D",
909         color=CSEE_COLORS["green"],
910         edgecolor="black",
911         linewidth=0.5,
912         label=" 消纳优先",
913         zorder=4,
914     )
915     label_offsets = {
916         " 弃电率不超过 20%": (-8, 14, "center"),
917         " 弃电率不超过 15%": (-6, 18, "center"),
918         " 弃电率不超过 10%": (0, 18, "center"),
919         " 弃电率不超过 5%": (0, 22, "center"),
920         " 弃电率不超过 1%": (22, 36, "left"),
921         " 零弃电": (10, 8, "left"),
922     }
923     for _, row in pareto[pareto["scheme"].str.contains("弃电率不超过|零弃电", regex=True)].iterrows():
924         label = row["scheme"].replace(" 弃电率不超过", "<")
925         dx, dy, ha = label_offsets.get(row["scheme"], (0, 16, "center"))
926         ax1.annotate(
927             label,
928             xy=(row["curtailment_rate"] * 100, row["ton_cost_yuan_per_t"]),
929             xytext=(dx, dy),
930             textcoords="offset points",
931             ha=ha,
932             va="center",

```

```

933         fontsize=7,
934         arrowprops={"arrowstyle": "-", "color": "#555555", "lw": 0.45, "shrinkA": 0, "shrinkB":
          ↪ 4},
935     )
936     ax1.set_xlabel(" 弃风弃光率/%")
937     ax1.set_ylabel(" 吨氨成本/(元/t)")
938     ax1.set_title(" 最大弃电场景储能容量的成本-消纳权衡")
939     ax1.grid(True, linestyle="--", alpha=0.4)
940     ax1.legend(frameon=False, ncol=3, loc="upper center", bbox_to_anchor=(0.5, -0.18))
941     cbar = fig.colorbar(sc, ax=ax1, fraction=0.035, pad=0.02)
942     cbar.set_label(" 储能容量/MWh")
943     fig.subplots_adjust(left=0.11, right=0.92, top=0.88, bottom=0.24)
944     save_figure(fig, output_dir / "q4_fig6_storage_pareto_csee")
945     plt.close(fig)
946
947
948 def plot_selected_storage_dispatch(hourly: pd.DataFrame, output_dir: Path) -> None:
949     set_csee_style()
950     df = hourly.sort_values("hour")
951     x = df["hour"]
952     fig, axes = plt.subplots(3, 1, figsize=(7.8, 7.0), sharex=True, gridspec_kw={"height_ratios":
          ↪ [1.15, 1.0, 0.85]})
953
954     ax = axes[0]
955     ax.stackplot(x, df["wind_mw"], df["pv_mw"], colors=[CSEE_COLORS["light_blue"], "#f7c97f"],
          ↪ labels=["风电", "光伏"], alpha=0.82)
956     ax.plot(x, df["regular_load_mw"] + df["process_load_mw"], color=CSEE_COLORS["red"], marker="o",
          ↪ label="总用电")
957     ax.plot(x, df["process_load_mw"], color=CSEE_COLORS["purple"], marker="s", label=" 制氮负荷")
958     ax.set_ylabel(" 功率/MW")
959     ax.grid(True, axis="y", linestyle="--", alpha=0.4)
960     ax.legend(frameon=False, ncol=4, loc="upper center", bbox_to_anchor=(0.5, -0.10))
961
962     ax = axes[1]
963     ax.bar(x - 0.18, df["storage_charge_mw"], width=0.34, color=CSEE_COLORS["light_blue"],
          ↪ edgecolor=CSEE_COLORS["blue"], linewidth=0.5, label="充电")
964     ax.bar(x + 0.18, -df["storage_discharge_mw"], width=0.34, color=CSEE_COLORS["light_red"],
          ↪ edgecolor=CSEE_COLORS["red"], linewidth=0.5, label="放电")
965     ax.plot(x, df["curtailment_mw"], color=CSEE_COLORS["orange"], marker="^", label=" 弃电")
966     ax.axhline(0, color="black", linewidth=0.7)
967     ax.set_ylabel(" 功率/MW")
968     ax.grid(True, axis="y", linestyle="--", alpha=0.4)
969     ax.legend(frameon=False, ncol=3, loc="upper center", bbox_to_anchor=(0.5, -0.12))
970
971     ax = axes[2]
972     ax.plot(x, df["storage_soc_mwh"], color=CSEE_COLORS["green"], marker="o", label="SOC")
973     ax2 = ax.twinx()
974     ax2.plot(x, df["production_load_factor"] * 100, color=CSEE_COLORS["purple"], linestyle="--",
          ↪ marker="s", label="负荷率")
975     ax.set_xlabel(" 时段/h")
976     ax.set_ylabel("SOC/MWh")
977     ax2.set_ylabel(" 制氮负荷率/%")
978     ax.set_xticks(range(0, 24, 2))
979     ax.set_xlim(-0.4, 23.4)
980     ax.grid(True, axis="y", linestyle="--", alpha=0.4)
981     handles1, labels1 = ax.get_legend_handles_labels()
982     handles2, labels2 = ax2.get_legend_handles_labels()
983     ax.legend(handles1 + handles2, labels1 + labels2, frameon=False, ncol=2, loc="upper center",
          ↪ bbox_to_anchor=(0.5, -0.24))
984
985     scenario = str(df["scenario"].iloc[0])
986     cap = float(df["storage_capacity_mwh"].iloc[0])
987     fig.suptitle(f" 最大弃电场景 {scenario} 的储能调度结果 ({cap:.0f} MWh)", y=0.99, fontsize=10)
988     fig.subplots_adjust(left=0.10, right=0.90, top=0.91, bottom=0.12, hspace=0.42)
989     save_figure(fig, output_dir / "q4_fig3_selected_storage_dispatch_csee")
990     plt.close(fig)
991
992
993 def plot_no_storage_vs_storage(no_storage: pd.DataFrame, storage: pd.DataFrame, output_dir: Path) ->
          ↪ None:
994     set_csee_style()
995     left = no_storage[["scenario", " 日产氨量", " 弃风弃光电量", " 吨氨成本"]].rename(
996         columns={"日产氨量": "no_storage_ammonia", "弃风弃光电量": "no_storage_curtailment",
          ↪ "吨氨成本": "no_storage_ton_cost"})
997
998     right = storage[["scenario", " 日产氨量", " 弃风弃光电量", " 吨氨成本"]].rename(
999         columns={"日产氨量": "storage_ammonia", "弃风弃光电量": "storage_curtailment", "吨氨成本":
          ↪ "storage_ton_cost"})
1000
1001     df = left.merge(right, on="scenario").sort_values("scenario")

```

```

1002 x = np.arange(len(df))
1003 fig, axes = plt.subplots(3, 1, figsize=(8.6, 7.0), sharex=True)
1004 width = 0.36
1005 axes[0].bar(x - width / 2, df["no_storage_ammonia"], width, color=CSEE_COLORS["light_blue"],
1006 ↪ edgecolor=CSEE_COLORS["blue"], linewidth=0.5, label="无储能")
1007 axes[0].bar(x + width / 2, df["storage_ammonia"], width, color="#f7c97f",
1008 ↪ edgecolor=CSEE_COLORS["orange"], linewidth=0.5, label="有储能")
1009 axes[0].set_ylabel("日产氨量/t")
1010 axes[0].grid(True, axis="y", linestyle="--", alpha=0.4)
1011 axes[0].legend(frameon=False, ncol=2, loc="upper center", bbox_to_anchor=(0.5, -0.10))
1012 axes[1].bar(x - width / 2, df["no_storage_curtailement"], width, color=CSEE_COLORS["light_blue"],
1013 ↪ edgecolor=CSEE_COLORS["blue"], linewidth=0.5, label="无储能")
1014 axes[1].bar(x + width / 2, df["storage_curtailement"], width, color="#f7c97f",
1015 ↪ edgecolor=CSEE_COLORS["orange"], linewidth=0.5, label="有储能")
1016 axes[1].set_ylabel("弃风弃光/MWh")
1017 axes[1].grid(True, axis="y", linestyle="--", alpha=0.4)
1018 axes[2].plot(x, df["no_storage_ton_cost"], color=CSEE_COLORS["blue"], marker="o",
1019 ↪ linestyle="--", label="无储能")
1020 axes[2].plot(x, df["storage_ton_cost"], color=CSEE_COLORS["orange"], marker="s", label="有储能")
1021 axes[2].set_ylabel("吨氨成本/(元/t)")
1022 axes[2].set_xlabel("风光组合场景")
1023 axes[2].set_xticks(x)
1024 axes[2].set_xticklabels(df["scenario"], rotation=45, ha="right")
1025 axes[2].grid(True, axis="y", linestyle="--", alpha=0.4)
1026 axes[2].legend(frameon=False, ncol=2, loc="upper center", bbox_to_anchor=(0.5, -0.36))
1027 fig.suptitle("离网无储能与有储能运行结果对比", y=0.99, fontsize=10)
1028 fig.subplots_adjust(left=0.09, right=0.98, top=0.91, bottom=0.24, hspace=0.35)
1029 save_figure(fig, output_dir / "q4_fig4_no_storage_vs_storage_csee")
1030 plt.close(fig)
1031
1032 def plot_mode_cost_breakdown(mode_compare: pd.DataFrame, output_dir: Path) -> None:
1033     set_csee_style()
1034     df = mode_compare.copy()
1035     labels = df["display_mode"].tolist()
1036     x = np.arange(len(df))
1037     components = [
1038         ("annual_wind_cost_yuan", "风电", CSEE_COLORS["light_blue"]),
1039         ("annual_pv_cost_yuan", "光伏", "#f7c97f"),
1040         ("annual_purchase_cost_yuan", "购电", CSEE_COLORS["light_red"]),
1041         ("annual_alk_om_yuan", "ALK 运维", CSEE_COLORS["purple"]),
1042         ("annual_pem_om_yuan", "PEM 运维", CSEE_COLORS["blue"]),
1043         ("annual_ammonia_om_yuan", "合成氨运维", CSEE_COLORS["gray"]),
1044         ("annual_storage_capital_yuan", "储能年化", "#b7b7b7"),
1045         ("annual_storage_om_yuan", "储能运维", CSEE_COLORS["green"]),
1046     ]
1047     fig, ax = plt.subplots(figsize=(7.8, 4.9))
1048     bottom = np.zeros(len(df))
1049     for col, name, color in components:
1050         values = df[col].fillna(0).to_numpy() / 10000
1051         ax.bar(x, values, bottom=bottom, color=color, edgecolor="black", linewidth=0.35, width=0.58,
1052 ↪ label=name)
1053         bottom += values
1054     sale = df["annual_sale_revenue_yuan"].fillna(0).to_numpy() / 10000
1055     ax.bar(x, -sale, color="#d9ead3", edgecolor=CSEE_COLORS["green"], linewidth=0.35, width=0.58,
1056 ↪ label="售电收益")
1057     ax.axhline(0, color="black", linewidth=0.8)
1058     positive_limit = max(float(np.max(bottom)) * 1.14, 1.0)
1059     negative_limit = min(float(np.min(-sale)) * 1.18, -1.0) if np.max(sale) > 0 else -1.0
1060     ax.set_ylim(negative_limit, positive_limit)
1061     ax.set_xticks(x)
1062     ax.set_xticklabels(labels)
1063     ax.set_ylabel("年度成本/万元")
1064     ax.grid(True, axis="y", linestyle="--", alpha=0.4)
1065     ax2 = ax.twinx()
1066     ax2.plot(x, df["annual_average_ton_cost_yuan_per_t"], color=CSEE_COLORS["red"], marker="o",
1067 ↪ label="吨氨成本")
1068     ax2.set_ylabel("吨氨成本/(元/t)")
1069     handles1, labels1 = ax.get_legend_handles_labels()
1070     handles2, labels2 = ax2.get_legend_handles_labels()
1071     figure_legend_below(fig, handles1 + handles2, labels1 + labels2, ncol=5, y=0.01)
1072     ax.set_title("离网与并网同产量方案年度成本分项对比")
1073     fig.subplots_adjust(left=0.10, right=0.90, top=0.88, bottom=0.25)
1074     save_figure(fig, output_dir / "q4_fig5_mode_cost_breakdown_csee")
1075     plt.close(fig)

```

```

1073 def write_markdown_summary(
1074     no_storage_summary: pd.DataFrame,
1075     no_storage_annual: pd.DataFrame,
1076     capacity_scan: pd.DataFrame,
1077     pareto_summary: pd.DataFrame,
1078     selected_capacity: float,
1079     selected_row: pd.Series,
1080     selection_note: str,
1081     storage_summary: pd.DataFrame,
1082     storage_annual: pd.DataFrame,
1083     grid_annual: pd.DataFrame,
1084     mode_compare: pd.DataFrame,
1085     capacity_estimate: pd.DataFrame,
1086 ) -> None:
1087     max_curtail = no_storage_summary.loc[no_storage_summary["弃风弃光电量"].idxmax()]
1088     no_ann = no_storage_annual.iloc[0]
1089     st_ann = storage_annual.iloc[0]
1090     grid_ann = grid_annual.iloc[0]
1091     support_value = float(mode_compare["grid_support_value_yuan_per_t"].iloc[0])
1092     utilization_max_row = capacity_scan.loc[capacity_scan["日产氨量"].idxmax()]
1093
1094     lines = [
1095         "# 问题四结果摘要",
1096         "",
1097         "## 离网无储能运行",
1098         "",
1099         "| 指标 | 数值 |",
1100         "|---|---|",
1101         f"| 年产氨量/t | {no_ann['annual_ammonia_t']:.2f} |",
1102         f"| 年产能利用率 | {no_ann['annual_capacity_utilization']:.2%} |",
1103         f"| 年均吨氨成本/(元/t) | {no_ann['annual_average_ton_cost_yuan_per_t']:.2f} |",
1104         f"| 年弃风弃光电量/MWh | {no_ann['annual_curtailment_mwh']:.2f} |",
1105         f"| 年风光利用率 | {no_ann['annual_renewable_utilization']:.2%} |",
1106         f"| 年常规负荷供电量/MWh | {no_ann['annual_regular_shed_mwh']:.2f} |",
1107         f"| 常规负荷完全自给代表天数/d | {int(no_ann['energy_autonomous_days'])} |",
1108         "",
1109         f"无储能时弃风弃光最大的场景为 **{max_curtail['scenario']}**, 日弃电量为
1110         ↳ **{max_curtail['弃风弃光电量']:.2f} MWh**, 日产氨量为 **{max_curtail['日产氨量']:.2f} t**.
1111         ↳ ",
1112         "",
1113         "## 储能容量配置",
1114         "",
1115         f"容量选择规则: {selection_note}",
1116         "",
1117         "| 储能容量/MWh | 储能功率/MW | 日产氨量/t | 弃风弃光/MWh | 风光利用率 | 吨氨成本/(元/t) |",
1118         "|---|---|---|---|---|---|",
1119         f"| {selected_capacity:.2f} | {selected_capacity * BATTERY_C_RATE:.2f} |
1120         ↳ {selected_row['日产氨量']:.2f} | {selected_row['弃风弃光电量']:.2f} |
1121         ↳ {selected_row['风光利用率']:.2%} | {selected_row['吨氨成本']:.2f} |",
1122         "",
1123         "作为工程对照, 若以最大化风光消纳和日产氨量为主目标, 容量扫描中 **{cap:.2f} MWh**
1124         ↳ 可将该场景弃风弃光降至 **{cur:.2f} MWh**, 日产氨量提高至 **{nh3:.2f} t**, 但吨氨成本升至
1125         ↳ **{cost:.2f} 元/t**, 因此未作为主经济方案.".format(
1126             cap=utilization_max_row["storage_capacity_mwh"],
1127             cur=utilization_max_row["弃风弃光电量"],
1128             nh3=utilization_max_row["日产氨量"],
1129             cost=utilization_max_row["吨氨成本"],
1130         ),
1131         "",
1132         "### 经济-消纳权衡方案",
1133         "",
1134         "| 方案 | 储能容量/MWh | 日产氨量/t | 弃风弃光率 | 风光利用率 | 吨氨成本/(元/t) |",
1135         "|---|---|---|---|---|---|",
1136     ]
1137
1138     for _, row in pareto_summary.iterrows():
1139         lines.append(
1140             "| {scheme} | {cap:.2f} | {nh3:.2f} | {cur:.2%} | {use:.2%} | {cost:.2f} |".format(
1141                 scheme=row["scheme"],
1142                 cap=row["storage_capacity_mwh"],
1143                 nh3=row["daily_ammonia_t"],
1144                 cur=row["curtailment_rate"],
1145                 use=row["renewable_utilization"],
1146                 cost=row["ton_cost_yuan_per_t"],
1147             )
1148         )
1149
1150     lines.extend(
1151         [
1152             "",
1153             "## 24 场景储能调度年度结果",
1154             "",
1155         ]
1156     )

```

```

1147     "| 指标 | 无储能 | 有储能 |",
1148     "|---|---:|---:|",
1149     f"| 年产量/t | {no_ann['annual_ammonia_t']:.2f} | {st_ann['annual_ammonia_t']:.2f} |",
1150     f"| 年产能利用率 | {no_ann['annual_capacity_utilization']:.2%} |",
1151     f"| 年吨氨成本/(元/t) | {no_ann['annual_average_ton_cost_yuan_per_t']:.2f} |",
1152     f"| 年弃风弃光/MWh | {no_ann['annual_curtailment_mwh']:.2f} |",
1153     f"| 年风光利用率 | {no_ann['annual_renewable_utilization']:.2%} |",
1154     f"| 年常规负荷缺供/MWh | {no_ann['annual_regular_shed_mwh']:.2f} |",
1155     f"| 年吨氨成本/(元/t) | {st_ann['annual_average_ton_cost_yuan_per_t']:.2f} |",
1156     f"| 年弃风弃光/MWh | {st_ann['annual_curtailment_mwh']:.2f} |",
1157     f"| 年风光利用率 | {st_ann['annual_renewable_utilization']:.2%} |",
1158     f"| 年常规负荷缺供/MWh | {st_ann['annual_regular_shed_mwh']:.2f} |",
1159     f"| 年吨氨成本/(元/t) | {st_ann['annual_average_ton_cost_yuan_per_t']:.2f} |",
1160     "### 并网同产量对比",
1161     "模式 | 年产量/t | 年总成本/万元 | 年吨氨成本/(元/t) |",
1162     "|---|---:|---:|---:|",
1163 ]
1164 )
1165 for _, row in mode_compare.iterrows():
1166     lines.append(
1167         f"| {row['display_mode']} | {row['annual_ammonia_t']:.2f} |",
1168         f"| {row['annual_total_cost_yuan'] / 10000:.2f} |",
1169         f"| {row['annual_average_ton_cost_yuan_per_t']:.2f} |",
1170     )
1171     lines.extend(
1172         [
1173             "",
1174             f"以离网+储能方案的年产量为同产量约束，公共电网支撑价值为 **{support_value:.2f} 元/t**",
1175             f"（离网+储能成本减并网同产量成本）。",
1176             "",
1177             "### 无储能满产能源自治装机估算",
1178             "",
1179             f"| 估算口径 | 风电容量/MW | 光伏容量/MW | 总容量/MW | 相对当前总容量倍数 | 最小时段裕度/MW |",
1180             f"|---|---:|---:|---:|---:|",
1181         ]
1182     )
1183 )
1184 for _, cap in capacity_estimate.iterrows():
1185     lines.append(
1186         f"| {cap['description']} | {cap['wind_capacity_mw']:.2f} | {cap['pv_capacity_mw']:.2f} |",
1187         f"| {cap['total_capacity_mw']:.2f} | {cap['total_capacity_multiple_vs_current']:.2f} |",
1188         f"| {cap['minimum_hourly_margin_mw']:.4f} |",
1189     )
1190     lines.extend(
1191         [
1192             "",
1193             "说明：储能容量扫描范围为0-300 MWh，步长5 MWh；储能功率按1C配置。",
1194             f"无储能满产能源自治装机估算要求24个风光场景每个小时均可满足常规负荷和72 t/d满负荷制氨负荷，",
1195             f"因此属于保守的瞬时自给容量下界估算。",
1196         ]
1197     )
1198 )
1199 (OUT_DIR / "q4_result_summary.md").write_text("\n".join(lines), encoding="utf-8")
1200
1201 def validate_results(
1202     no_storage_hourly: pd.DataFrame,
1203     storage_hourly: pd.DataFrame,
1204     grid_hourly: pd.DataFrame,
1205     storage_capacity_mwh: float,
1206 ) -> pd.DataFrame:
1207     rows = [
1208         {
1209             "check": "no_storage_offgrid_power_balance_max_abs_mw",
1210             "value": float(no_storage_hourly["offgrid_balance_error_mw"].abs().max()),
1211         },
1212         {
1213             "check": "storage_offgrid_power_balance_max_abs_mw",
1214             "value": float(storage_hourly["offgrid_balance_error_mw"].abs().max()),
1215         },
1216         {
1217             "check": "storage_soc_min_mwh",
1218             "value": float(storage_hourly["storage_soc_mwh"].min()),
1219         },
1220         {
1221             "check": "storage_soc_max_mwh",
1222             "value": float(storage_hourly["storage_soc_mwh"].max()),
1223         },
1224     ]
1225     return pd.DataFrame(rows)

```

```

1214         "check": "storage_capacity_mwh",
1215         "value": float(storage_capacity_mwh),
1216     },
1217     {
1218         "check": "storage_charge_discharge_simultaneous_max_mw2",
1219         "value": float((storage_hourly["storage_charge_mw"] *
1220             ↪ storage_hourly["storage_discharge_mw"]).max()),
1221     },
1222     {
1223         "check": "grid_purchase_sale_simultaneous_max_mw2",
1224         "value": float((grid_hourly["grid_purchase_mw"] * grid_hourly["grid_sale_mw"]).max()),
1225     },
1226 ]
1227 cyclic_errors = []
1228 for _, group in storage_hourly.groupby("scenario", sort=False):
1229     group = group.sort_values("hour")
1230     soc = group["storage_soc_mwh"].to_numpy()
1231     ch = group["storage_charge_mw"].to_numpy()
1232     dis = group["storage_discharge_mw"].to_numpy()
1233     prev = np.roll(soc, 1)
1234     err = soc - (1 - SystemParams().storage_self_loss_ratio) * prev -
1235         ↪ SystemParams().storage_charge_eff * ch + dis / SystemParams().storage_discharge_eff
1236     cyclic_errors.append(np.max(np.abs(err)))
1237 rows.append({"check": "storage_soc_dynamic_max_abs_mwh", "value": float(np.max(cyclic_errors))})
1238 return pd.DataFrame(rows)
1239
1240 def main() -> None:
1241     OUT_DIR.mkdir(parents=True, exist_ok=True)
1242     params = SystemParams()
1243
1244     no_storage_hourly, no_storage_summary = run_no_storage_cases(params)
1245     no_storage_annual = build_offgrid_annual_summary(no_storage_summary, "offgrid_no_storage")
1246
1247     max_curtail = no_storage_summary.loc[no_storage_summary["弃风弃光电量"].idxmax()]
1248     scenarios = add_regular_load_to_scenarios(params)
1249     max_group = scenarios[scenarios["scenario"] == max_curtail["scenario"]].copy()
1250     capacity_scan = scan_storage_capacity(
1251         max_group,
1252         params,
1253         str(max_curtail["scenario"]),
1254         int(max_curtail["wind_scenario"]),
1255         int(max_curtail["pv_scenario"]),
1256     )
1257     selected_capacity, selected_row, selection_note = choose_storage_capacity(capacity_scan)
1258     pareto_summary = build_storage_pareto_summary(capacity_scan)
1259
1260     storage_selected_hourly, _ = solve_storage_offgrid(
1261         max_group,
1262         params,
1263         selected_capacity,
1264         str(max_curtail["scenario"]),
1265         int(max_curtail["wind_scenario"]),
1266         int(max_curtail["pv_scenario"]),
1267     )
1268     storage_hourly, storage_summary = run_storage_cases(params, selected_capacity)
1269     storage_annual = build_offgrid_annual_summary(storage_summary, "offgrid_storage")
1270
1271     grid_hourly, grid_summary = run_grid_same_production_cases(params, storage_summary)
1272     grid_annual = build_grid_annual_summary(grid_summary)
1273     mode_compare = build_mode_comparison(
1274         no_storage_annual,
1275         storage_annual,
1276         grid_annual,
1277         no_storage_summary,
1278         storage_summary,
1279         grid_summary,
1280     )
1281     capacity_estimate = estimate_min_capacity_for_full_autonomy(params)
1282     validation = validate_results(no_storage_hourly, storage_hourly, grid_hourly, selected_capacity)
1283
1284     no_storage_hourly.to_csv(OUT_DIR / "q4_no_storage_hourly.csv", index=False,
1285         ↪ encoding="utf-8-sig")
1286     no_storage_summary.to_csv(OUT_DIR / "q4_no_storage_summary.csv", index=False,
1287         ↪ encoding="utf-8-sig")
1288     no_storage_annual.to_csv(OUT_DIR / "q4_no_storage_annual_summary.csv", index=False,
1289         ↪ encoding="utf-8-sig")
1290     capacity_scan.to_csv(OUT_DIR / "q4_storage_capacity_scan.csv", index=False,
1291         ↪ encoding="utf-8-sig")
1292     pareto_summary.to_csv(OUT_DIR / "q4_storage_pareto_summary.csv", index=False,
1293         ↪ encoding="utf-8-sig")

```

```

1288 storage_selected_hourly.to_csv(OUT_DIR / "q4_storage_selected_hourly.csv", index=False,
1289 ↪ encoding="utf-8-sig")
1289 storage_hourly.to_csv(OUT_DIR / "q4_storage_all_scenario_hourly.csv", index=False,
1290 ↪ encoding="utf-8-sig")
1290 storage_summary.to_csv(OUT_DIR / "q4_storage_all_scenario_summary.csv", index=False,
1291 ↪ encoding="utf-8-sig")
1291 storage_annual.to_csv(OUT_DIR / "q4_storage_annual_summary.csv", index=False,
1292 ↪ encoding="utf-8-sig")
1292 grid_hourly.to_csv(OUT_DIR / "q4_grid_comparison_hourly.csv", index=False, encoding="utf-8-sig")
1293 grid_summary.to_csv(OUT_DIR / "q4_grid_comparison_summary.csv", index=False,
1294 ↪ encoding="utf-8-sig")
1294 grid_annual.to_csv(OUT_DIR / "q4_grid_annual_summary.csv", index=False, encoding="utf-8-sig")
1295 mode_compare.to_csv(OUT_DIR / "q4_mode_comparison_summary.csv", index=False,
1296 ↪ encoding="utf-8-sig")
1296 capacity_estimate.to_csv(OUT_DIR / "q4_min_capacity_estimate.csv", index=False,
1297 ↪ encoding="utf-8-sig")
1297 validation.to_csv(OUT_DIR / "q4_validation_summary.csv", index=False, encoding="utf-8-sig")
1298
1299 write_markdown_summary(
1300     no_storage_summary,
1301     no_storage_annual,
1302     capacity_scan,
1303     pareto_summary,
1304     selected_capacity,
1305     selected_row,
1306     selection_note,
1307     storage_summary,
1308     storage_annual,
1309     grid_annual,
1310     mode_compare,
1311     capacity_estimate,
1312 )
1313
1314 plot_no_storage_heatmaps(no_storage_summary, OUT_DIR)
1315 plot_storage_capacity_scan(capacity_scan, selected_capacity, OUT_DIR)
1316 plot_storage_pareto(capacity_scan, pareto_summary, selected_capacity, OUT_DIR)
1317 plot_selected_storage_dispatch(storage_selected_hourly, OUT_DIR)
1318 plot_no_storage_vs_storage(no_storage_summary, storage_summary, OUT_DIR)
1319 plot_mode_cost_breakdown(mode_compare, OUT_DIR)
1320
1321 print("Q4 no-storage annual summary:")
1322 print(no_storage_annual.to_string(index=False))
1323 print("\nMax curtailment scenario:")
1324 print(max_curtail[["scenario", "日产氨量", "弃风弃光电量", "风光利用率", "吨氨成本"]].to_string())
1325 print("\nSelected storage capacity:")
1326 print(f"{selected_capacity:.2f} MWh / {selected_capacity * BATTERY_C_RATE:.2f} MW")
1327 print(selected_row[["日产氨量", "弃风弃光电量", "风光利用率", "吨氨成本"]].to_string())
1328 print(f"Selection note: {selection_note}")
1329 print("\nQ4 storage annual summary:")
1330 print(storage_annual.to_string(index=False))
1331 print("\nGrid-connected same-production annual summary:")
1332 print(grid_annual.to_string(index=False))
1333 print("\nMode comparison:")
1334 print(
1335     mode_compare[
1336         [
1337             "display_mode",
1338             "annual_ammonia_t",
1339             "annual_total_cost_yuan",
1340             "annual_average_ton_cost_yuan_per_t",
1341             "grid_support_value_yuan_per_t",
1342         ]
1343     ].to_string(index=False)
1344 )
1345 print("\nMinimum capacity estimate:")
1346 print(capacity_estimate.to_string(index=False))
1347 print("\nValidation:")
1348 print(validation.to_string(index=False))
1349 print(f"\nOutputs written to: {OUT_DIR}")
1350
1351 if __name__ == "__main__":
1352     main()
1353

```

E.5 问题五：系统影响与政策证据矩阵

文件：code/questions/solve_q5.py

说明：从前四问结果中提取 E1-E8 证据，生成系统影响分析表、政策建议矩阵和政策依据索引。

```
1 from __future__ import annotations
2
3 import sys
4 from pathlib import Path
5
6 import matplotlib.pyplot as plt
7 import numpy as np
8 import pandas as pd
9
10 PROJECT_ROOT = Path(__file__).resolve().parents[1]
11 if str(PROJECT_ROOT) not in sys.path:
12     sys.path.insert(0, str(PROJECT_ROOT))
13
14 from model_common.metrics import get_metric
15 from model_common.plotting import CSEE_COLORS, annotate_bars, legend_below, save_figure,
16     ↪ set_csee_style
17
18 OUT_DIR = Path(__file__).resolve().parent / "outputs"
19 Q1_OUT = PROJECT_ROOT / " 问题 1_ 典型风光场景指标分析" / "outputs"
20 Q2_OUT = PROJECT_ROOT / " 问题 2_ 离散制氮调节优化" / "outputs"
21 Q3_OUT = PROJECT_ROOT / " 问题 3_ 连续制氮调节优化" / "outputs"
22 Q4_OUT = PROJECT_ROOT / " 问题 4_ 离网储能配置优化" / "outputs"
23
24
25 def pct(value: float) -> str:
26     return f"{value:.2%}"
27
28
29 def load_prior_results() -> dict[str, pd.DataFrame]:
30     results = {
31         "q1_summary": pd.read_csv(Q1_OUT / "q1_summary.csv"),
32         "q2_annual": pd.read_csv(Q2_OUT / "q2_annual_summary.csv"),
33         "q3_annual": pd.read_csv(Q3_OUT / "q3_annual_summary.csv"),
34         "q3_compare": pd.read_csv(Q3_OUT / "q3_compare_q2_summary.csv"),
35         "q4_no_storage_annual": pd.read_csv(Q4_OUT / "q4_no_storage_annual_summary.csv"),
36         "q4_storage_annual": pd.read_csv(Q4_OUT / "q4_storage_annual_summary.csv"),
37         "q4_grid_annual": pd.read_csv(Q4_OUT / "q4_grid_annual_summary.csv"),
38         "q4_mode_compare": pd.read_csv(Q4_OUT / "q4_mode_comparison_summary.csv"),
39         "q4_capacity": pd.read_csv(Q4_OUT / "q4_min_capacity_estimate.csv"),
40     }
41     pareto_path = Q4_OUT / "q4_storage_pareto_summary.csv"
42     results["q4_pareto"] = pd.read_csv(pareto_path) if pareto_path.exists() else pd.DataFrame()
43     return results
44
45
46 def build_model_evidence(results: dict[str, pd.DataFrame]) -> pd.DataFrame:
47     q1 = results["q1_summary"]
48     q2 = results["q2_annual"]
49     q3 = results["q3_annual"]
50     q3c = results["q3_compare"]
51     q4_no = results["q4_no_storage_annual"].iloc[0]
52     q4_st = results["q4_storage_annual"].iloc[0]
53     q4_grid = results["q4_grid_annual"].iloc[0]
54     q4_comp = results["q4_mode_compare"].iloc[0]
55     q4_cap = results["q4_capacity"]
56     q4_pareto = results["q4_pareto"]
57
58     q3_72 = q3.loc[q3["production_t_per_day"] == 72].iloc[0]
59     q3_36 = q3.loc[q3["production_t_per_day"] == 36].iloc[0]
60     q3c_36 = q3c.loc[q3c["production_t_per_day"] == 36].iloc[0]
61     q2_36 = q2.loc[q2["production_t_per_day"] == 36].iloc[0]
62     if not q4_pareto.empty:
63         p10 = q4_pareto.loc[q4_pareto["scheme"] == " 弃电率不超过 10%"].iloc[0]
64         zero = q4_pareto.loc[q4_pareto["scheme"] == " 零弃电"].iloc[0]
65         econ = q4_pareto.loc[q4_pareto["scheme"] == " 经济最优正储能"].iloc[0]
66         pareto_text = (
67             f" 最大弃电场景下，经济方案为 {econ['storage_capacity_mwh']:.0f}MWh； "
68             f" 若要求弃电率≤10%，
69             ↪ {p10['storage_capacity_mwh']:.0f}MWh且吨氨成本{p10['ton_cost_yuan_per_t']:.2f}元/t；
70             ↪ "
```

```

69         f"零弃电需{zero['storage_capacity_mwh']:.0f}MWh且吨氨成本{zero['ton_cost_yuan_per_t']:.2f}
        ↪ }元/t。"
70     )
71 else:
72     pareto_text = " 储能容量需要在经济性、弃电率和可靠性之间进行权衡。"
73
74 rows = [
75     {
76         "evidence_id": "E1",
77         "source_problem": "Q1",
78         "quantitative_result": "典型日上网比例35.92%，超过20%约束；同时购电172.04 MWh、上网216.77
        ↪ MWh。",
79         "system_impact": " 源荷曲线不同步会同时形成购电缺口和反送电压力。",
80         "policy_implication": " 绿电直连项目不能只看年绿电量，应考核小时级源荷匹配和最大交换功率。",
81     },
82     {
83         "evidence_id": "E2",
84         "source_problem": "Q2",
85         "quantitative_result": (
86             f"离散开停36 t/d年均吨氨成本{q2_36['annual_average_ton_cost_yuan_per_t']:.2f}元/t最低，
            ↪ "
87             f"但全年0天全满足；72
            ↪ t/d全满足270天但成本{q3_72['annual_average_ton_cost_yuan_per_t']:.2f}元/t。"
88         ),
89         "system_impact": " 低产量降低购电成本但推高余电上网比例，高产量提高消纳但增加用电和成本。",
90         "policy_implication": "运行考核应同时约束成本、产能利用率和绿电指标，避免单纯追求最低吨成本。",
91     },
92     {
93         "evidence_id": "E3",
94         "source_problem": "Q3",
95         "quantitative_result": (
96             f"连续调节36 t/d较离散开停降本{abs(q3c_36['ton_cost_delta_yuan_per_t']):.2f}元/t，"
97             f"购电减少{abs(q3c_36['purchase_delta_mwh']):.2f}MWh，
            ↪ 上网减少{abs(q3c_36['sale_delta_mwh']):.2f}MWh，"
98             f" 全满足天数由 0 增至 165 天。"
99         ),
100        "system_impact": " 电解槽和合成氨负荷连续调节可显著降低公共电网交换电量。",
101        "policy_implication": " 应把可调负荷能力作为绿电直连项目接入评审和运行考核指标。",
102    },
103    {
104        "evidence_id": "E4",
105        "source_problem": "Q3",
106        "quantitative_result": (
107            f"连续调节72 t/d日上网比例{pct(q3_72['annual_sale_ratio'])}, 36
            ↪ t/d上网比例{pct(q3_36['annual_sale_ratio'])}。"
108        ),
109        "system_impact": " 负荷利用率下降会降低本地消纳能力并增加新能源外送压力。",
110        "policy_implication": "新能源装机应按以荷定源原则滚动校核，
        ↪ 并设置上网比例和消纳困难时段反送约束。",
111    },
112    {
113        "evidence_id": "E5",
114        "source_problem": "Q4",
115        "quantitative_result": (
116            f"离网无储能年产氨{q4_no['annual_ammonia_t']:.2f}t，
            ↪ 产能利用率{pct(q4_no['annual_capacity_utilization'])}, "
117            f"年弃电{q4_no['annual_curtailment_mwh']:.2f}MWh；
            ↪ 5MWh储能后年产氨{q4_st['annual_ammonia_t']:.2f}t，"
118            f" 弃电降至 {q4_st['annual_curtailment_mwh']:.2f}MWh 且缺供为 0。"
119        ),
120        "system_impact": " 离网模式提高物理绿电属性，但对储能、冗余装机和运行安全要求更高。",
121        "policy_implication": " 离网项目应建立自给能力、缺供风险、弃电率和储能配置的专项评估。",
122    },
123    {
124        "evidence_id": "E6",
125        "source_problem": "Q4",
126        "quantitative_result": (
127            f"同产量下离网+储能吨氨成本{q4_st['annual_average_ton_cost_yuan_per_t']:.2f}元/t，"
128            f" 并网同产量 {q4_grid['annual_average_ton_cost_yuan_per_t']:.2f} 元/t，"
129            f" 系统支撑价值 {q4_comp['grid_support_value_yuan_per_t']:.2f} 元/t。"
130        ),
131        "system_impact": " 公共电网提供备用、互济和消纳服务，降低园区自建储能和冗余容量成本。",
132        "policy_implication": " 并网型项目应公平承担输配电费、系统运行费、备用和辅助服务成本。",
133    },
134    {
135        "evidence_id": "E7",
136        "source_problem": "Q4",
137        "quantitative_result": (
138            " 无储能满产逐小时自治若不固定风光比例需总装机"

```

```

139         f"{q4_cap.iloc[0]['total_capacity_mw']:.2f}MW;
        ↳ 保持当前风光比例需{q4_cap.iloc[1]['total_capacity_mw']:.2f}MW。"
140     ),
141     "system_impact": " 完全离网满产自治需要极大冗余装机,经济性和土地资源约束突出。",
142     "policy_implication": " 应鼓励弱联网、源网荷储协同和市场化备用,而不是简单追求完全离网。",
143 },
144 {
145     "evidence_id": "E8",
146     "source_problem": "Q4",
147     "quantitative_result": pareto_text,
148     "system_impact": " 储能容量存在成本-消纳 Pareto 权衡,经济最优容量不等于消纳最优容量。",
149     "policy_implication": " 储能配置考核应采用经济性、弃电率和可靠性多目标评估,
        ↳ 而不是只规定单一容量比例。",
150 },
151 ]
152 return pd.DataFrame(rows)
153
154 def build_policy_matrix() -> pd.DataFrame:
155     rows = [
156         {
157             "source_id": "P01",
158             "source_name": " 发改能源〔2026〕688 号:多用户绿电直连",
159             "local_file":
160                 ↳ "参考资料/01_政策与官方报告/P01_2026_多用户绿电直连发展通知_发改能源688号.html",
161             "url": "https://www.ndrc.gov.cn/xxgk/zcfb/tz/202605/t20260520_1405313.html",
162             "use_in_q5": "支持绿色氢氨醇开展绿电直连;提出以荷定源、自发自用比例、上网比例、小时级溯源、
        ↳ 接网容量和系统费用分担。",
163         },
164         {
165             "source_id": "P02",
166             "source_name": " 发改能源〔2025〕650 号:绿电直连",
167             "local_file": "参考资料/01_政策与官方报告/P02_2025_绿电直连发展通知_发改能源650号.html",
168             "url": "https://www.nea.gov.cn/20250530/2d67a6e49c044f2eacabe1fddf48d20f/c.html",
169             "use_in_q5": "三项绿电直连指标、源荷匹配、系统友好性、储能和负荷灵活性、分表计量与费用承担依据。
        ↳ ",
170         },
171         {
172             "source_id": "P03",
173             "source_name": " 发改价格〔2025〕136 号:新能源上网电价市场化改革",
174             "local_file":
175                 ↳ "参考资料/01_政策与官方报告/P03_2025_新能源上网电价市场化改革_发改价格136号.html",
176             "url": "https://www.ndrc.gov.cn/xxgk/zcfb/tz/202502/t20250209_1396066.html",
177             "use_in_q5": " 余电上网价格、市场化交易和新能源消纳责任的价格背景。",
178         },
179         {
180             "source_id": "P04",
181             "source_name": " 发改能源〔2024〕1537 号:可再生能源替代行动",
182             "local_file": "参考资料/01_政策与官方报告/P04_2024_可再生能源替代行动指导意见.html",
183             "url": "https://www.ndrc.gov.cn/xxgk/zcfb/tz/202410/t20241030_1394119.html",
184             "use_in_q5": " 工业绿电直供、源网荷储、低碳氢替代、风光氢氨醇一体化基地和需求侧调节依据。",
185         },
186         {
187             "source_id": "P05",
188             "source_name": " 国能发发改〔2024〕93 号:电力领域新型经营主体",
189             "local_file":
190                 ↳ "参考资料/01_政策与官方报告/P05_2024_电力领域新型经营主体创新发展指导意见.html",
191             "url": "https://www.gov.cn/zhengce/zhengceku/202412/content_6991420.htm",
192             "use_in_q5": " 虚拟电厂、负荷聚合、智能微电网等新型主体参与调节和市场机制。",
193         },
194         {
195             "source_id": "P06",
196             "source_name": " 国能综通科技〔2025〕91 号:能源领域氢能试点",
197             "local_file":
198                 ↳ "参考资料/01_政策与官方报告/P06_2025_能源领域氢能试点通知_国能综通科技91号.pdf",
199             "url": "https://www.nea.gov.cn/20250610/472b12c43f534aab9a95de81034dcd92/20250610472b12_
        ↳ c43f534aab9a95de81034dcd92_2288580504e06346d39eb8d268c8e70947.pdf",
200             "use_in_q5": " 柔性离网制氢、氢氨燃料供能、氢储能和示范试点依据。",
201         },
202         {
203             "source_id": "P08",
204             "source_name": " 发改能源规〔2021〕280 号:源网荷储一体化和多能互补",
205             "local_file": "参考资料/01_政策与官方报告/P08_2021_源网荷储一体化和多能互补指导意见.html",
206             "url": "https://www.gov.cn/zhengce/zhengceku/2021-03/06/content_5590895.htm",
207             "use_in_q5": " 园区级源网荷储、负荷侧调节、储能参与调峰和多能互补依据。",
208         },
209         {
210             "source_id": "R01",
211             "source_name": "IEA Global Hydrogen Review 2025",

```

```

209         "local_file": "参考资料/01_政策与官方报告/R01_IEA_Global_Hydrogen_Review_2025.pdf",
210         "url": "https://iea.blob.core.windows.net/assets/12d92ecc-e960-40f3-aff5-b2de6690ab6b/Glob
↪ alHydrogenReview2025.pdf",
211         "use_in_q5": "低排放氢产业、项目落地、政策支持和需求创造的国际背景。",
212     },
213     {
214         "source_id": "R02",
215         "source_name": "IRENA Innovation Outlook: Renewable Ammonia",
216         "local_file":
↪ "参考资料/01_政策与官方报告/R02_IRENA_Innovation_Outlook_Renewable_Ammonia_2022.pdf",
217         "url": "https://www.irena.org/-/media/Files/IRENA/Agency/Publication/2022/May/IRENA_Inn
↪ ovation_Outlook_Ammonia_2022.pdf",
218         "use_in_q5": "绿氨技术路线、可再生电力制氢制氨和成本构成依据。",
219     },
220     {
221         "source_id": "R05",
222         "source_name": "EU 2023/1184 RFNBO Delegated Regulation",
223         "local_file":
↪ "参考资料/01_政策与官方报告/R05_EU_RFNBO_Delegated_Regulation_2023_1184.pdf",
224         "url": "https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:32023R1184",
225         "use_in_q5": "绿氢绿氨电力来源的额外性、时间匹配和地域匹配国际经验。",
226     },
227 ]
228 return pd.DataFrame(rows)
229
230
231 def build_recommendation_matrix() -> pd.DataFrame:
232     rows = [
233         {
234             "recommendation": "建立小时级绿电溯源与分表计量",
235             "model_basis": "Q1显示同一典型日既购电172.04MWh又上网216.77MWh;
↪ Q3低产量方案上网比例可达35.76%。",
236             "policy_basis": "P01 要求小时级新能源发用电量匹配, P02 要求分表计量和源荷匹配。",
237             "expected_effect": "避免年累计绿电量掩盖小时错配, 提高绿电认证可信度。",
238         },
239         {
240             "recommendation": "将上网比例、最大交换功率和反送时段纳入运行考核",
241             "model_basis": "Q1 上网比例 35.92% 不达标, Q4 并网同产量上网比例 49.82%。",
242             "policy_basis": "P01/P02 均强调上网比例一般不超过 20%, 消纳困难时段不应反送。",
243             "expected_effect": "降低局部潮流反转和新能源消纳压力。",
244         },
245         {
246             "recommendation": "把柔性负荷能力作为准入和补偿对象",
247             "model_basis": "Q3连续调节使36t/d方案较离散开停减少购电16216.93MWh、上网16216.93MWh,
↪ 并增加165个全满足代表日。",
248             "policy_basis": "P04、P08 均强调需求侧资源和源网荷储互动。",
249             "expected_effect": "提升园区对电网的友好性, 减少外部调峰需求。",
250         },
251         {
252             "recommendation": "建立储能配置的经济性、弃电率和可靠性多目标评估",
253             "model_basis": "Q4中5MWh储能为经济方案, 80MWh可使最大弃电场景弃电率不超过10%,
↪ 155MWh可零弃电但吨氨成本更高。",
254             "policy_basis": "P01/P02 要求合理配置储能, P08 强调优化储能规模和系统平衡能力。",
255             "expected_effect": "避免过度储能或储能不足, 兼顾消纳、可靠性和成本。",
256         },
257         {
258             "recommendation": "完善系统运行费、备用和辅助服务费用分担",
259             "model_basis": "Q4 测算并网支撑价值为 735.81 元/t。",
260             "policy_basis": "P01/P02 要求并网型项目公平承担输配电费、系统运行费等费用。",
261             "expected_effect": "防止成本向普通用户转移, 反映公共电网支撑价值。",
262         },
263         {
264             "recommendation": "推进风光氢氨醇一体化与弱联网示范",
265             "model_basis": "Q4 完全离网满产自治需极高冗余装机, 弱联网更具经济性。",
266             "policy_basis": "P01 优先支持绿色氢氨醇, P04 支持风光氢氨醇一体化基地, P06 支持氢能试点。",
267             "expected_effect": "在产业脱碳和电力系统安全之间取得更优平衡。",
268         },
269     ],
270 ]
271 return pd.DataFrame(rows)
272
273 def plot_q3_grid_exchange(results: dict[str, pd.DataFrame], output_dir: Path) -> None:
274     set_csee_style()
275     q3 = results["q3_annual"].sort_values("production_t_per_day")
276     x = np.arange(len(q3))
277     width = 0.34
278     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7.8, 6.1), sharex=True,
↪ gridspec_kw={"height_ratios": [1.15, 1.0]})
279     bars1 = ax1.bar(

```

```

280     x - width / 2,
281     q3["annual_purchase_mwh"],
282     width,
283     color=CSEE_COLORS["light_red"],
284     edgecolor=CSEE_COLORS["red"],
285     linewidth=0.5,
286     label=" 年购电量",
287 )
288 bars2 = ax1.bar(
289     x + width / 2,
290     q3["annual_sale_mwh"],
291     width,
292     color=CSEE_COLORS["light_blue"],
293     edgecolor=CSEE_COLORS["blue"],
294     linewidth=0.5,
295     label=" 年上网电量",
296 )
297 ax1.set_ylabel(" 年度电量/MWh")
298 ax1.grid(True, axis="y", linestyle="--", alpha=0.4)
299 ax1.legend(frameon=False, ncol=2, loc="upper center", bbox_to_anchor=(0.5, -0.10))
300
301 ax2.plot(x, q3["annual_sale_ratio"] * 100, color=CSEE_COLORS["red"], marker="o",
302     ↪ label="上网比例")
303 ax2.axhline(20, color=CSEE_COLORS["red"], linestyle=":", linewidth=0.9, label="20% 约束")
304 ax2.set_ylabel(" 上网比例/%")
305 ax2.set_xlabel(" 日产量/t")
306 ax2.set_xticks(x)
307 ax2.set_xticklabels([f"{int(v)}" for v in q3["production_t_per_day"]])
308 ax2.grid(True, axis="y", linestyle="--", alpha=0.4)
309 ax3 = ax2.twinx()
310 bars3 = ax3.bar(
311     x,
312     q3["full_satisfied_days"],
313     width=0.26,
314     color="#d9ead3",
315     edgecolor=CSEE_COLORS["green"],
316     linewidth=0.5,
317     alpha=0.75,
318     label=" 全满足天数",
319 )
320 ax3.set_ylabel(" 全满足天数/d")
321 ax3.set_ylim(0, 360)
322 handles2, labels2 = ax2.get_legend_handles_labels()
323 handles3, labels3 = ax3.get_legend_handles_labels()
324 ax2.legend(handles2 + handles3, labels2 + labels3, frameon=False, ncol=3, loc="upper center",
325     ↪ bbox_to_anchor=(0.5, -0.24))
326 fig.suptitle(" 连续调节下日产量对电网交换和指标合格性的影响", y=0.985, fontsize=10)
327 fig.subplots_adjust(left=0.10, right=0.90, top=0.91, bottom=0.20, hspace=0.35)
328 save_figure(fig, output_dir / "q5_fig1_grid_exchange_policy_evidence_csee")
329 plt.close(fig)
330
331 def plot_q4_grid_support(results: dict[str, pd.DataFrame], output_dir: Path) -> None:
332     set_csee_style()
333     comp = results["q4_mode_compare"].copy()
334     x = np.arange(len(comp))
335     labels = comp["display_mode"].tolist()
336     fig, ax = plt.subplots(figsize=(7.2, 4.6))
337     bars = ax.bar(
338         x,
339         comp["annual_average_ton_cost_yuan_per_t"],
340         color=[CSEE_COLORS["light_blue"], "#f7c97f", CSEE_COLORS["light_red"]],
341         edgecolor=[CSEE_COLORS["blue"], CSEE_COLORS["orange"], CSEE_COLORS["red"]],
342         linewidth=0.6,
343         width=0.58,
344     )
345     ax.set_xticks(x)
346     ax.set_xticklabels(labels)
347     ax.set_ylabel(" 年均吨氨成本/(元/t)")
348     ax.set_title(" 离网与并网同产量的系统支撑价值")
349     ax.grid(True, axis="y", linestyle="--", alpha=0.4)
350     ax.set_ylim(0, comp["annual_average_ton_cost_yuan_per_t"].max() * 1.18)
351     annotate_bars(ax, bars, fmt="{:.0f}", dy_frac=0.012)
352     off_cost = float(comp.loc[comp["display_mode"] == "离网+储能",
353         ↪ "annual_average_ton_cost_yuan_per_t"].iloc[0])
354     grid_cost = float(comp.loc[comp["display_mode"] == "并网同产量",
355         ↪ "annual_average_ton_cost_yuan_per_t"].iloc[0])
356     support = off_cost - grid_cost
357     ax.annotate(
358         f" 系统支撑价值: {support:.0f} 元/t",
359         xy=(2, grid_cost),

```

```

357     xytext=(1.25, off_cost * 0.72),
358     arrowprops={"arrowstyle": "->", "lw": 0.8, "color": CSEE_COLORS["gray"]},
359     fontsize=8,
360     color="#222222",
361 )
362 fig.subplots_adjust(left=0.12, right=0.98, top=0.88, bottom=0.16)
363 save_figure(fig, output_dir / "q5_fig2_grid_support_value_csee")
364 plt.close(fig)
365
366
367 def write_summary(
368     evidence: pd.DataFrame,
369     policy_matrix: pd.DataFrame,
370     recommendations: pd.DataFrame,
371     results: dict[str, pd.DataFrame],
372 ) -> None:
373     q4_comp = results["q4_mode_compare"]
374     support_value = float(q4_comp["grid_support_value_yuan_per_t"].iloc[0])
375     lines = [
376         "# 问题五结果摘要",
377         "",
378         "## 前四问支撑结论",
379         "",
380         "| 编号 | 来源 | 数值证据 | 系统影响 | 政策含义 |",
381         "| ---|---|---|---|---|",
382     ]
383     for _, row in evidence.iterrows():
384         lines.append(
385             f"| {row['evidence_id']} | {row['source_problem']} | {row['quantitative_result']} | "
386             f"↪ {row['system_impact']} | {row['policy_implication']} |"
387         )
388     lines.extend(
389         [
390             "",
391             "## 绿电直连园区高渗透的主要影响",
392             "",
393             "有利影响：促进新能源就近消纳；推动化工行业绿氢绿氨替代；形成可调负荷和储能资源；"
394             "↪ 带动源网荷储一体化投资。",
395             "",
396             "不利影响或风险：增加公共电网净负荷波动和反送压力；可能转移备用、调峰和系统运行成本；"
397             "↪ 弱联网或离网项目对安全稳定、储能和冗余容量要求高；绿电溯源和多主体结算复杂度提高；"
398             "↪ 电氢氨安全与电力安全耦合增强。",
399             "",
400             "## 政策建议",
401             "",
402             "| 建议 | 模型依据 | 政策依据 | 预期效果 |",
403             "| ---|---|---|---|",
404         ]
405     )
406     for _, row in recommendations.iterrows():
407         lines.append(
408             f"| {row['recommendation']} | {row['model_basis']} | {row['policy_basis']} | "
409             f"↪ {row['expected_effect']} |"
410         )
411     lines.extend(
412         [
413             "",
414             f"本文将并网同产量相对离网+储能的成本差定义为公共电网支撑价值，本算例为 **{support_value:.2f}"
415             f"↪ 元/t**。",
416             "",
417             "## 主要参考资料",
418             "",
419             "| 编号 | 资料 | 用途 | 链接 |",
420             "| ---|---|---|---|",
421         ]
422     )
423     for _, row in policy_matrix.iterrows():
424         lines.append(f"| {row['source_id']} | {row['source_name']} | {row['use_in_q5']} | "
425                     f"↪ {row['url']} |")
426     (OUT_DIR / "q5_result_summary.md").write_text("\n".join(lines), encoding="utf-8")
427
428
429 def main() -> None:
430     OUT_DIR.mkdir(parents=True, exist_ok=True)
431     results = load_prior_results()
432     evidence = build_model_evidence(results)
433     policy_matrix = build_policy_matrix()
434     recommendations = build_recommendation_matrix()
435
436     evidence.to_csv(OUT_DIR / "q5_model_evidence_summary.csv", index=False, encoding="utf-8-sig")

```

```

430 policy_matrix.to_csv(OUT_DIR / "q5_policy_reference_matrix.csv", index=False,
    ↪ encoding="utf-8-sig")
431 recommendations.to_csv(OUT_DIR / "q5_policy_recommendation_matrix.csv", index=False,
    ↪ encoding="utf-8-sig")
432 write_summary(evidence, policy_matrix, recommendations, results)
433 plot_q3_grid_exchange(results, OUT_DIR)
434 plot_q4_grid_support(results, OUT_DIR)
435
436 print("Q5 model evidence:")
437 print(evidence.to_string(index=False))
438 print("\nQ5 recommendations:")
439 print(recommendations.to_string(index=False))
440 print(f"\nOutputs written to: {OUT_DIR}")
441
442
443 if __name__ == "__main__":
444     main()

```